

2026 상반기 DFIR 기초 역량 강화 세미나

파일시스템

2026-03-06

김은채, 조민혁



고려대학교 정보보호대학원
Korea University
School of Cybersecurity



DFRC Korea University
Digital Forensic Research Center

- 개요
- FAT32
- NTFS
- The Sleuth Kit (TSK)
- X-Ways

Keywords

파일 시스템

FAT32

NTFS

개요

Windows 파일시스템 종류 및 변화

■ 파일시스템이란?

- 디지털 데이터를 효과적으로 관리하기 위해 파일을 체계적으로 기록하는 방식

■ Windows 파일시스템 정리

파일시스템	최초 등장	설명
FAT (File Allocation Table)	1977	• MS – DOS 시기부터 현재까지 USB, SD Card 등 광범위하게 사용 • 파일 위치를 FAT 테이블로 관리하는 체인 구조 • 저널링, 보안 기능 부재 및 대용량 파일 처리 한계
HPFS (High Performance File System)	1988	• OS / 2와 초기 Windows NT용으로 설계된 고성능 파일 시스템 • 디스크 용량 및 파일 크기 증가로 FAT 사용 한계 → 긴 파일명, 큰 볼륨 지원하는 HPFS 등장 • B + 트리 기반 디렉터리 구조 사용 • NTFS 등장으로 현대 Windows에서 사실상 사용 X
NTFS (New Technology File System)	1993	• Windows NT 계열의 표준 파일 시스템 • 기존 HPFS의 권한, 보안 기능 부재 해결 • MFT 기반 구조와 저널링으로 안정성과 복구 기능 제공 • 보안, 압축, 암호화(EFS), 대용량 파일 지원 등 고급 기능 지원
ReFS (Resilient File System)	2012	• 대규모 스토리지 환경을 위해 설계된 차세대 파일 시스템 • NTFS의 한계를 보완하기 위해 개발 → 데이터 무결성 및 자동 복구 기능 강화 • 주로 Windows Server와 일부 고급 워크스테이션 에디션에서 사용

FAT32 vs NTFS

■ 주요 차이점

■ FAT32

- 구조: FAT (File Allocation Table) 기반 클러스터 체인 관리 구조
- 안정성 / 보안성: 저널링, 파일 권한, 접근제어 등 고급 기능 미지원
- 호환성: 다양한 OS 및 저장 장치 호환 가능 → USB, SD-Card 등 범용적 사용

■ NTFS

- 구조: MFT (Master File Table) 기반 메타데이터 중심 관리 구조
- 안정성 / 보안성: 트랜잭션 로그 (\$LogFile) 기반 저널링, ACL 기반 제어 기능 지원
- 호환성: Windows 기반 설계/ 타 OS에서는 읽기 위주 지원 → 쓰기 지원을 위해 추가 드라이버 등 필요

FAT32 vs NTFS

■ 주요 차이점

구분	FAT32	NTFS
구조	FAT 기반	MFT 기반
저널링	X	O
권한 설정 기능	X	O (ACL)
암호화	X	O (EFS)
최대 볼륨 크기	2 TB	이론상 8 YB
최대 파일 크기	4 GB	이론상 16 EB
호환성	Windows, macOS, Linux 등 대부분 OS에서 읽기 / 쓰기 지원	Windows 중심 다른 OS는 쓰기 드라이버 등 추가 장치 필요
주사용 용도	USB, SD Card, 임베디드 장치 등	데스크탑, PC, 서버 등

FAT32

소개

- **FAT (File Allocation Table) 파일 시스템이란?**
 - Microsoft에서 개발
 - 파일이 사용하는 Cluster 번호를 테이블에 기록
 - 다음 Cluster를 체인처럼 연결
 - 장점
 - 높은 호환성: Windows, macOS, Linux 등 다양한 운영체제 지원
 - 단순한 구조: 임베디드 장치, USB 등 널리 사용
 - 단점
 - 파일 크기 제한: 단일 파일 최대 4 GB로 제한 → 대용량 데이터 저장 불가
 - 보안 및 안정성 부족: 파일 권한 관리, 암호화, 저널링 기능 부재

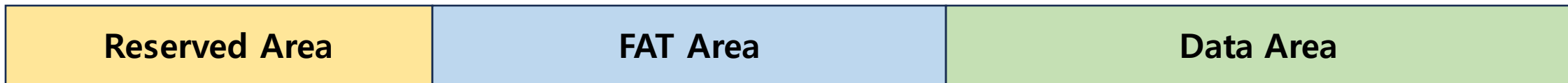
소개

■ FAT 파일 시스템 종류

- FAT12: 1980년대 도입 / MS-DOS용으로 개발, 플로피 디스크 및 저용량 저장 장치 목적 설계
- FAT16: 1984년 도입 / Cluster 표현 비트 수가 16 bits로 확장
- FAT32: 1996년 도입 / 실제 Cluster 표현 비트 수를 32 bits (실제 28 bits만 사용)로 확장
- exFAT: 2006년 도입 / FAT32의 4 GB 파일 크기 제한을 개선 → 고용량 Flash Memory 최적화

구분	Cluster 표현 비트 수	최대 볼륨 크기	최대 파일 크기	디렉토리 당 최대 파일 수
FAT12	12 bits	16 MB	볼륨 크기	4,096
FAT16	16 bits	2 GB	볼륨 크기	65,535
FAT32	32 bits	2 TB	4 GB	65,535
exFAT	64 bits	16 EB	12 PB	2,796,202

전체 구조



■ Reserved Area

- Boot Sector, FSINFO, Boot Strap 등 파일 시스템 부팅 및 관리 정보 저장
- 볼륨 최상단에 위치 / 32개의 섹터로 구성

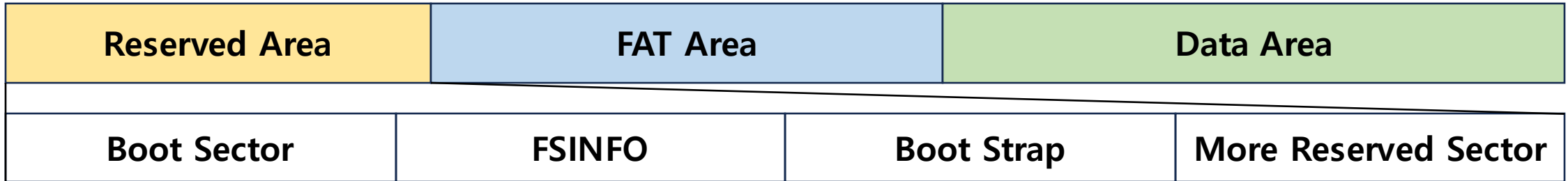
■ FAT Area

- 파일 / 폴더가 저장되는 위치 정보를 포함한 FAT이 저장되는 영역
- 각 클러스터 번호에 대응하는 FAT Entry 존재 → 파일 / 폴더가 여러 클러스터 사용할 경우 클러스터 체인 형성

■ Data Area

- 실제 파일 / 폴더의 데이터 및 속성 (Directory Entry)이 저장되는 영역

전체 구조 - Reserved Area



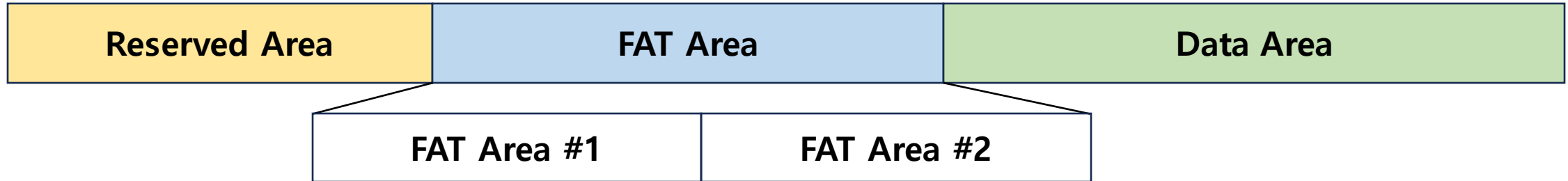
- **Boot Sector (0번 섹터 / 백업: 6번 섹터)**
 - BPB (BIOS Parameter Block), Boot Code and Error Message 부팅 및 파일 시스템 관리 정보 저장
- **FSINFO (1번 섹터 / 백업: 7번 섹터)**
 - 사용 가능 Cluster 수, 사용 가능 Cluster 위치 등 저장
- **Boot Strap (2번 섹터 / 백업: 8번 섹터)**
 - Boot Code and Error Message 영역이 부족할 경우 추가 사용 가능한 공간

전체 구조 - Reserved Area

■ Boot Sector 구조

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00	Jump Command			OEM ID									Bytes per Sector		Sector per Cluster	Reserved Sector Count	
10	Number of FAT Tables	Root Directory Entry		Total Sector		Media Type	FAT Size 16		Sector per Track		Number of Heads		Hidden Sectors				
20	Total Sector 32				FAT Size 32				Ext Flags		File System Version		Root Directory Cluster				
30	File System Info		Backup Boot Sector		Reserved												
40	Drive Num	Reserved	Boot Signature	Volume ID				Volume Label									
50	Volume Label		File System Type														
...	Boot Code and Error Message																
1F0															Signature		

전체 구조 - FAT Area



■ FAT Area

- FAT Area #1: 원본 FAT / FAT Area #2: 복사본
- 연속된 FAT Entry로 구성
 - 각 Entry는 4 bytes / Cluster 당 1개씩 존재
- 연결 리스트 구조: Cluster 번호를 통해 다음 파일의 위치를 추적

전체 구조 - FAT Area

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0x00	Media Type				Partition Status				Cluster 2				Cluster 3			
0x10	Cluster 4				Cluster 5				Cluster 6				Cluster 7			
0x20	Cluster 8				Cluster 9				Cluster 10				Cluster 11			
0x30	...															

■ FAT Area

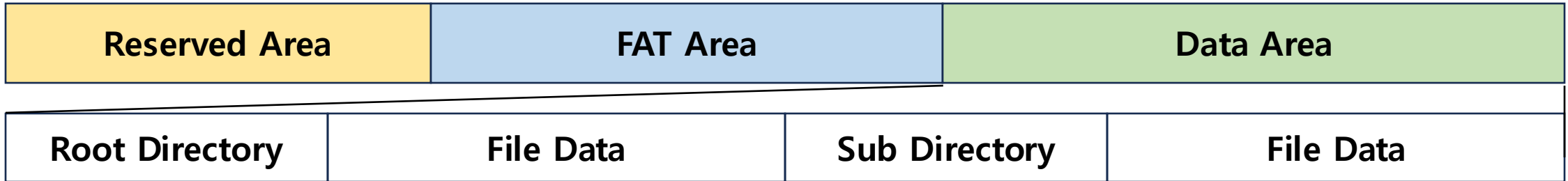
- Cluster 0, 1은 예약 상태 → 실제 데이터 저장은 Cluster 2부터 진행
- Cluster 0 (Media Type)
 - 저장매체 종류 의미, DOS 시절 플로피 디스크 (0xF0) / 하드 디스크 (0xF8) 구분을 위한 필드
- Cluster 1 (Partition Status)
 - 해당 파티션 부팅 가능 여부 표시

전체 구조 - FAT Area

■ FAT Entry Value 정리

FAT Entry Value	Description
0x?0000000	비어 있는 Cluster, 사용 가능
0x?0000001	예약된 값, 사용 X
0x?0000002 - 0x?FFFFFFEF	다음 Cluster 번호 지시
0x?FFFFFFF0 - 0x?FFFFFFF6	예약된 값, 사용 X
0x?FFFFFFF7	Bad Cluster, 데이터 저장 불가
0x?FFFFFFF8 - 0x?FFFFFFF	Cluster 체인의 마지막 의미 (EOC, End Of Chain)

전체 구조 - Data Area



■ Data Area

■ Root Directory

- 파일 시스템 최상위 Directory → 위치는 Boot Sector에서 지정 (일반적으로 Cluster 2)

■ Sub Directory

- Root Directory 하위 Directory

■ File Data

- Directory Entry가 가리키는 파일의 실제 데이터가 저장된 영역
 - Directory Entry: Root / Sub Directory에 순차적으로 저장되는 메타데이터 구조체

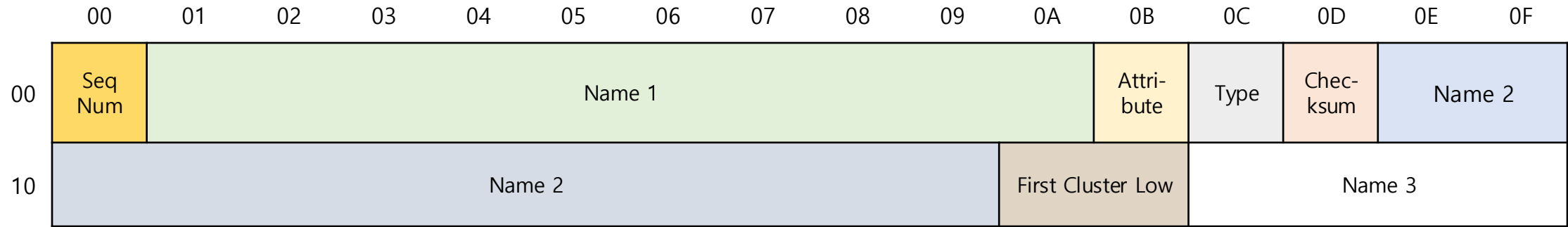
메타데이터 및 구조체 분석 – SFN

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	Status	Name							Extension			Attri- bute	Reser- ved	Create Time Tenth	Create Time	
10	Create Date		Last Accessed Time		Starting Cluster High		Last Modified Time		Last Modified Date		Starting Cluster Low		File Size			

■ SFN (Short File Name) Entry

- 파일 / 폴더의 메타데이터를 담은 32 bytes 구조체
- 파일 이름이 8.3 형식을 만족하는 경우: 32 bytes SFN Entry 1개 사용
 - 8.3 형식: 8글자 Name + 3글자 Extension

메타데이터 및 구조체 분석 – LFN



■ LFN (Long File Name) Entry

- 파일 / 폴더의 메타데이터를 담은 32 bytes 구조체
- 파일 이름이 8.3 형식을 만족하지 않는 경우: LFN (Long File Name) Entry 추가 사용
 - SFN Entry 1개 + LFN Entry 여러 개 → 최대 255자 표현 가능

메타데이터 및 구조체 분석 – Attribute

■ Attribute Field 속성 정리

- 파일, 폴더의 속성을 정의하는 값
- OR 연산으로 여러 속성 동시 표현 가능
- LFN의 Attribute: 0x0F 고정 → 나머지 Attribute는 같이 쓰이는 SFN에서 표현

Value	Type	Description
0x01	Read only	읽기 전용 파일
0x02	Hidden	숨김 파일
0x04	System	운영체제 관련 시스템 파일
0x08	Volume Label	볼륨 이름
0x0F	Long file name	LFN 의미
0x10	Sub Directory	디렉터리
0x20	Archive	일반 파일

메타데이터 및 구조체 분석

■ SFN 구조체 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004000E0	44	46	52	43	20	20	20	20	20	20	20	10	08	80	31	9E	DFRC ..€1ž
004000F0	5B	5C	5B	5C	00	00	32	9E	5B	5C	05	00	00	00	00	00	[\[\..2ž[\.....

Offset	Field Name	Size	Description / Value
0x00	Status	1 bytes	파일, 폴더 이름 앞 첫 글자 / 삭제된 경우 0xE5 표기
0x01	Name	7 bytes	파일, 폴더 앞 첫 글자를 제외한 나머지 글자
0x08	Extension	3 bytes	확장자 / 폴더인 경우 공백
0x0B	Attribute	1 bytes	파일, 폴더의 속성 / 0x10: Directory
0x0C	Reserved	1 bytes	예약된 영역
0x14	Statring Cluster High	2 bytes	파일, 폴더 데이터 시작 클러스터 상위 2 bytes
0x1A	Starting Cluster Low	2 bytes	파일, 폴더 데이터 시작 클러스터 하위 2 bytes
0x1C	File Size	4 bytes	파일 크기 / Diretory의 경우 0

메타데이터 및 구조체 분석

■ SFN 구조체 분석

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
004000E0 44 46 52 43 20 20 20 20 20 20 20 10 08 80 31 9E DFRC ..€1ž
004000F0 5B 5C 5B 5C 00 00 32 9E 5B 5C 05 00 00 00 00 00 [\[\...\2ž[\.....

Offset	Field Name	Size	Description / Value
0x0D	Create Time Tenth	1 bytes	파일 생성 시각 1 / 100 단위 값 저장
0x0E	Create Time	2 bytes	파일 생성 시 / 분 / 초 값 저장 0x9E31 → 10011 / 110001 / 10001 → 19 / 49 / 17 → 19시 49분 34초
0x10	Create Date	2 bytes	파일 생성 년 / 월 / 일 값 저장 (1980년 기준) 0x5C5B → 0101110 / 0010 / 11011 → 46 / 2 / 27 → 2026년 2월 27일
0x12	Last Accessed Date	2 bytes	마지막 파일 접근 날짜 (1980년 기준)
0x16	Last Modified Time	2 bytes	마지막 파일 수정 시 / 분 / 초 값 저장 0x9E32 → 10011 / 110001 / 10010 → 19 / 49 / 18 → 19시 49분 36초
0x18	Last Modified Date	2 bytes	마지막 파일 수정 년 / 월 / 일 값 저장 (1980년 기준)

메타데이터 및 구조체 분석

■ LFN 구조체 분석

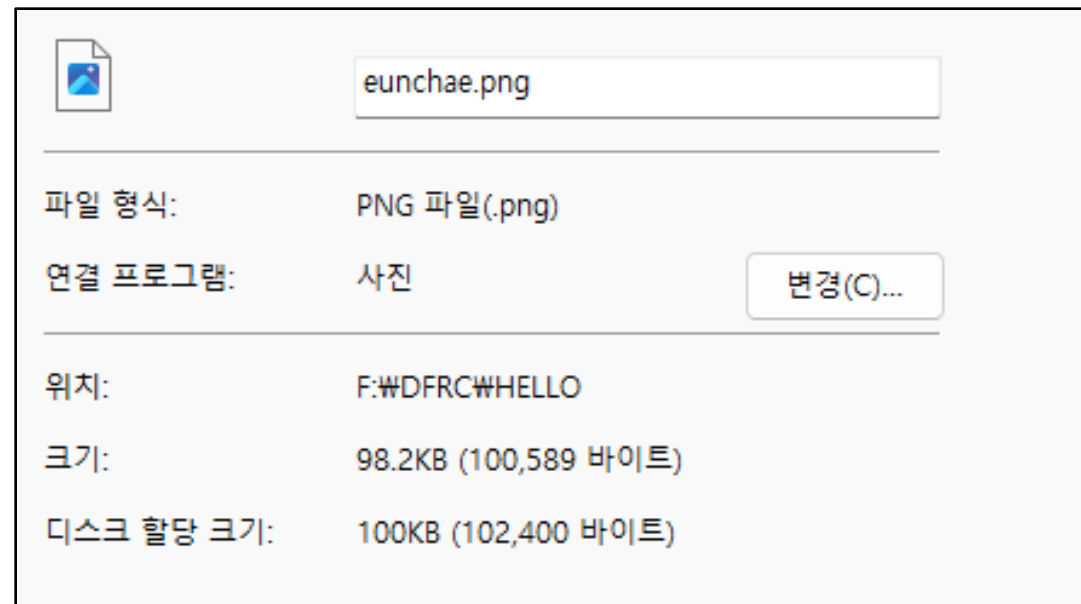
	Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
LFN Entry 2	00400140	42	77	00	65	00	6C	00	63	00	6F	00	0F	00	A4	6D	00	Bw.e.l.c.o...m.
	00400150	65	00	74	00	6F	00	64	00	66	00	00	00	72	00	63	00	e.t.o.d.f...r.c.
LFN Entry 1	00400160	01	77	00	65	00	6C	00	63	00	6F	00	0F	00	A4	6D	00	.w.e.l.c.o...m.
	00400170	65	00	74	00	6F	00	64	00	66	00	00	00	72	00	63	00	e.t.o.d.f...r.c.
SFN Entry 1	00400180	57	45	4C	43	4F	4D	7E	31	20	20	20	10	00	4A	35	9E	WELCOM~1 ..J5ž
	00400190	5B	5C	5B	5C	00	00	36	9E	5B	5C	08	00	00	00	00	00	[\[\..6ž[\.....

Offset	Field Name	Size	Description / Value
0x00	Sequence Number	1 bytes	해당 LFN Entry의 순서 지시 (0x42: 2번째, 마지막 LFN 지시)
0x01	Name 1	10 bytes	파일 이름 저장 / UNICODE 2 bytes 씩 총 10 bytes 저장
0x08	Attribute	1 bytes	파일, 폴더의 속성 / LFN 0x0F 고정
0x0B	Type	1 bytes	예약된 영역
0x0C	Checksum	1 bytes	SFN Entry 1에서 계산한 1 bytes 체크섬 / 동일 파일 LFN Entry는 같은 값 저장
0x14	Name2	12 bytes	파일 이름 저장 / UNICODE 2 bytes 씩 총 12 bytes 저장
0x20	First Cluster Low	2 bytes	LFN Entry에서 실사용 X / 실제 파일 데이터 시작 위치는 SFN Entry 위치
0x40	Name 3	4 bytes	파일 이름 저장 / UNICODE 2 bytes 씩 총 4 bytes 저장

메타데이터 및 구조체 분석 - [실습] 파일 찾기

■ 임의의 png 파일 생성

- 1GB의 가상 하드디스크 생성 → 디스크 초기화 후 FAT32 파일 시스템 할당
- F:\WDFRC\HELLO 경로에 eunchae.png 파일 생성
 - png 파일의 디스크 할당 크기는 100 KB



[eunchae.png 파일 생성]

메타데이터 및 구조체 분석 - [실습] 파일 찾기

- Boot Sector에서 Data Area를 찾아가기 위해 필요한 정보 확인
 - FAT Area 시작 Offset: $0x203400$ (Reserved Sector Count * Bytes per Sector)
 - Data Area 시작 offset: $0x400000$
 - FAT Area 시작 Offset + (FAT Size 32 x Number of FAT Tables) * Bytes per Sector
 - Sector per Cluster: $0x08$ / Root Directory Cluster: $0x2$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	EB	58	90	4D	53	44	4F	53	35	2E	30	00	02	08	1A	10	ëX.MSDOS5.0.....
00000010	02	00	00	00	00	F8	00	00	3F	00	FF	00	80	00	00	00	ø..?.ÿ.€...
00000020	00	E8	1F	00	F3	07	00	00	00	00	00	00	02	00	00	00	.è..ó.....
00000030	01	00	06	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	80	00	29	94	A6	C3	44	4E	4F	20	4E	41	4D	45	20	20	€.)”!ÃDNO NAME
00000050	20	20	46	41	54	33	32	20	20	20	33	C9	8E	D1	BC	F4	FAT32 3ÉŽÑ%ô

메타데이터 및 구조체 분석 - [실습] 파일 찾기

■ DFRC Directory가 저장되어 있는 Cluster 번호 확인 및 시작 offset 계산

■ Root Directory (클러스터 2)로 이동해서 DFRC Directory Entry 정보 파악

- Starting Cluster High: 0x0 / Starting Cluster Low: 0x6 → 클러스터 6에 위치

- DFRC Directory 시작 Offset

- $0x400000 + (6 - 2) * \text{Sector per Cluster} * \text{Bytes per Sector} = 0x404000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00400000	BB	F5	20	BA	BC	B7	FD	20	20	20	20	08	00	00	00	00	»õ ¼·ý
00400010	00	00	00	00	00	00	27	82	59	5C	00	00	00	00	00	00	',Y\.....
--- 중략 ---																	
004000E0	44	46	52	43	20	20	20	20	20	20	20	10	00	04	2B	82	DFRC ...+,
004000F0	59	5C	59	5C	00	00	2C	82	59	5C	06	00	00	00	00	00	Y\Y\...,Y\.....

메타데이터 및 구조체 분석 - [실습] 파일 찾기

- **HELLO Directory가 저장되어 있는 Cluster 번호 확인 및 시작 offset 계산**
 - DFRC Directory (클러스터 6)로 이동해서 HELLO Directory Entry 정보 파악
 - Starting Cluster High: 0x0 / Starting Cluster Low: 0x9 → 클러스터 9에 위치
 - HELLO Directory 시작 Offset
 - $0x400000 + (9 - 2) * \text{Sector per Cluster} * \text{Bytes per Sector} = 0x407000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00404000	2E	20	20	20	20	20	20	20	20	20	20	10	00	04	2B	82	+,
00404010	59	5C	59	5C	00	00	2C	82	59	5C	06	00	00	00	00	00	Y\Y\...,Y\.....
--- 중략 ---																	
00404080	48	45	4C	4C	4F	20	20	20	20	20	20	10	00	5E	2F	82	HELLO ..^/,
00404090	59	5C	59	5C	00	00	30	82	59	5C	09	00	00	00	00	00	Y\Y\..0,Y\.....

메타데이터 및 구조체 분석 - [실습] 파일 찾기

- png 파일이 저장되어 있는 Cluster 번호 확인 및 시작 offset 계산
 - HELLO Directory (클러스터 9)로 이동해서 eunchae.png Entry 정보 파악
 - Starting Cluster High: 0x0 / Starting Cluster Low: 0xA → 클러스터 10에 위치
 - eunchae.png 시작 Offset
 - $0x400000 + (10 - 2) * \text{Sector per Cluster} * \text{Bytes per Sector} = 0x408000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00407000	2E	20	20	20	20	20	20	20	20	20	10	00	5E	2F	82		. ..^/,
00407010	59	5C	59	5C	00	00	30	82	59	5C	09	00	00	00	00	00	Y\Y\..0,Y\.....
--- 중략 ---																	
00407060	45	55	4E	43	48	41	45	20	50	4E	47	20	18	3D	63	82	EUNCHAE PNG .=c,
00407070	59	5C	59	5C	00	00	64	82	59	5C	0A	00	ED	88	01	00	Y\Y\..d,Y\..í^..

메타데이터 및 구조체 분석 - [실습] 파일 찾기

- png 파일 Hex 데이터 복사 후 복사본 생성 및 기존 파일과 hash 값 비교
 - eunchae.png (클러스터 10)로 이동 → PNG 파일 존재 확인
 - 해당 섹터부터 25개의 클러스터 (100 KB) 데이터 복사 → F:\WDFRC\HELLO\eunchae1.png 저장

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00408000	89	50	4E	47	0D	0A	1A	0A	00	00	00	0D	49	48	44	52	%PNGIHDR
00408010	00	00	02	86	00	00	01	61	08	06	00	00	00	ED	23	6B	...t...a.....í#k
00408020	DE	00	00	00	01	73	52	47	42	00	AE	CE	1C	E9	00	00	p....sRGB.®Î.é..
00408030	00	04	67	41	4D	41	00	00	B1	8F	0B	FC	61	05	00	00	..gAMA..±...üa...
00408040	00	09	70	48	59	73	00	00	0E	C3	00	00	0E	C3	01	C7	..pHYs...Ã...Ã.ç

메타데이터 및 구조체 분석 - [실습] 파일 찾기

- png 파일 Hex 데이터 복사 후 복사본 생성 및 기존 파일과 hash 값 비교
 - Hash Calc를 이용해 eunchae.png, eunchae1.png hash 값 비교 → 동일 hash 값 / 두 파일 내용 동일
 - MD5: 9f8325f477942879b7ece56ebf672da3

SHA1: 836d872149746fef0d5bee044f6017c591666f9

Data Format:	Data:
File	F:\WDFRC\WHELLO\weunchae.png
☐ HMAC	Key: Text string
☒ MD5	9f8325f477942879b7ece56ebf672da3
☐ MD4	
☒ SHA1	e836d872149746fef0d5bee044f6017c591666f9

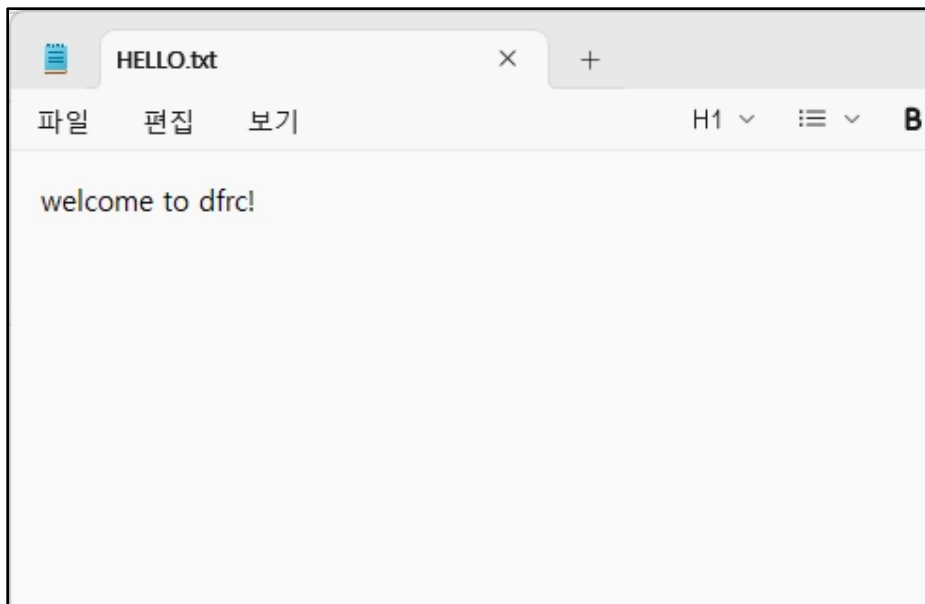
MD5: 9f8325f477942879b7ece56ebf672da3

Data Format:	Data:
File	F:\WDFRC\WHELLO\weunchae1.png
☐ HMAC	Key: Text string
☒ MD5	9f8325f477942879b7ece56ebf672da3
☐ MD4	
☒ SHA1	e836d872149746fef0d5bee044f6017c591666f9

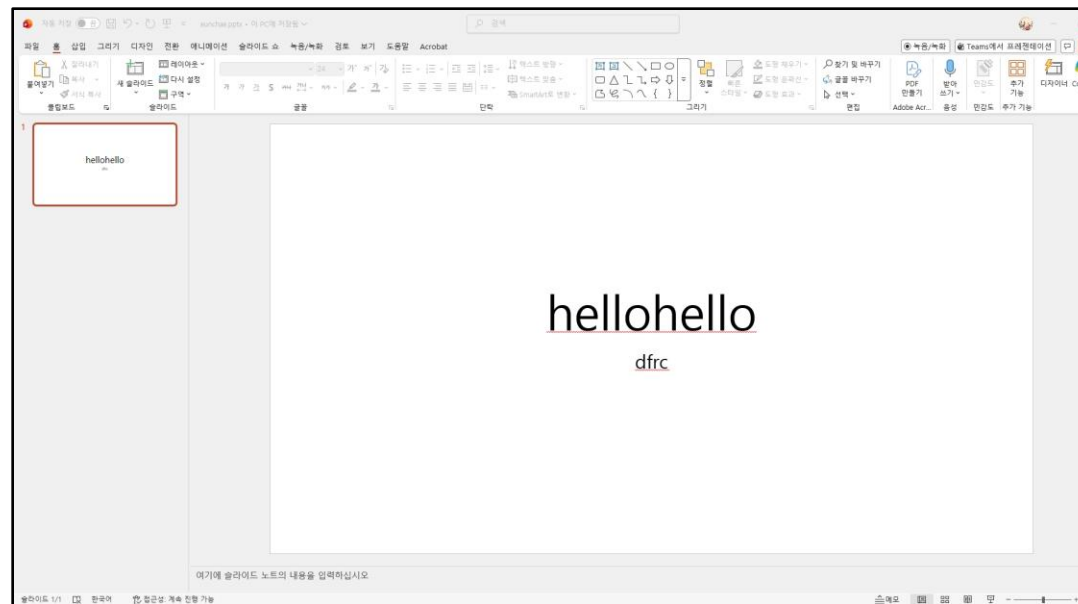
[HashCalc를 통한 해시 비교]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 임의의 텍스트 파일 및 PPT 파일 생성
 - 1GB의 가상 하드디스크 생성 → 디스크 초기화 후 FAT32 파일 시스템 할당
 - 새 볼륨 (F:)에 HELLO.txt, eunchae.pptx 생성



[HELLO.txt 내용]



[eunchae.pptx 내용]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 삭제 전 FAT Area 확인
 - FAT Area로 이동해서 할당된 클러스터 확인
 - 17개의 클러스터 할당 확인
 - 예약된 클러스터 0, 1 제외, 클러스터 2 ~ 클러스터 17 할당

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00203400	F8	FF	FF	0F	FF	FF	FF	FF	FF	FF	FF	0F	FF	FF	FF	0F	øÿÿ.ÿÿÿÿÿÿÿ.ÿÿÿ.
00203410	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	ÿÿÿ.ÿÿÿ.ÿÿÿ.ÿÿÿ.
00203420	00	00	00	00	0A	00	00	00	0B	00	00	00	0C	00	00	00	
00203430	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00	
00203440	11	00	00	00	FF	FF	FF	0F	00	00	00	00	00	00	00	00	ÿÿÿ.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 삭제 전 Directory Entry 확인

■ HELLO.txt

- Starting Cluster High: 0x0 / Starting Cluster Low: 0x7 / File size: 0x10 (16 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004000E0	48	45	4C	4C	4F	20	20	20	54	58	54	20	10	A9	AA	99	HELLO TXT .@™
004000F0	59	5C	59	5C	00	00	B5	99	59	5C	07	00	10	00	00	00	Y\Y\..μ™Y\.....

■ eunchae.pptx

- Starting Cluster High: 0x0 / Starting Cluster Low: 0x9 / File size: 0x87DE (34,872 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004002A0	45	55	4E	43	48	41	7E	31	50	50	54	20	00	63	B8	99	EUNCHA~1PPT .c,™
004002B0	59	5C	59	5C	00	00	C4	99	59	5C	09	00	DE	87	00	00	Y\Y\..Ä™Y\..p‡..

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 일반 삭제 후 FAT Area 확인

- FAT Area로 이동해서 할당된 클러스터 확인
 - 17개의 클러스터 할당 확인
 - 예약된 클러스터 0, 1 제외, 클러스터 2 ~ 클러스터 17 할당

■ 일반 삭제 전 후 FAT Area 차이 X

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00203400	F8	FF	FF	0F	FF	FF	FF	FF	FF	FF	FF	0F	FF	FF	FF	0F	øÿÿ.ÿÿÿÿÿÿÿ.ÿÿÿ.
00203410	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	ÿÿÿ.ÿÿÿ.ÿÿÿ.ÿÿÿ.
00203420	00	00	00	00	0A	00	00	00	0B	00	00	00	0C	00	00	00	
00203430	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00	
00203440	11	00	00	00	FF	FF	FF	0F	00	00	00	00	00	00	00	00	ÿÿÿ.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 일반 삭제 후 Directory Entry 확인

- HELLO.txt → 파일명 맨 앞이 0xE5로 변경, 나머지 동일
 - Starting Cluster High: 0x0 / Starting Cluster Low: 0x7 / File size: 0x10 (16 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004000E0	48	45	4C	4C	4F	20	20	20	54	58	54	20	10	A9	AA	99	HELLO TXT .@™
004000F0	59	5C	59	5C	00	00	B5	99	59	5C	07	00	10	00	00	00	Y\Y\..μ™Y\.....



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004000E0	E5	45	4C	4C	4F	20	20	20	54	58	54	20	10	A9	AA	99	âELLO TXT .@™
004000F0	59	5C	59	5C	00	00	B5	99	59	5C	07	00	10	00	00	00	Y\Y\..μ™Y\.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 일반 삭제 후 Directory Entry 확인

- eunchae.pptx → 파일명 맨 앞이 0xE5로 변경, 나머지 동일
 - Starting Cluster High: 0x0 / Starting Cluster Low: 0x9 / File size: 0x87DE (34,872 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004002A0	45	55	4E	43	48	41	7E	31	50	50	54	20	00	63	B8	99	EUNCHA~1PPT .c ,™
004002B0	59	5C	59	5C	00	00	C4	99	59	5C	09	00	DE	87	00	00	Y\Y\..Ä™Y\..p‡..



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004002A0	E5	55	4E	43	48	41	7E	31	50	50	54	20	00	63	B8	99	åUNCHA~1PPT .c ,™
004002B0	59	5C	59	5C	00	00	C4	99	59	5C	09	00	DE	87	00	00	Y\Y\..Ä™Y\..p‡..

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 일반 삭제 후 \$RECYCLE.BIN Directory 확인
 - \$IR992YZ.pptx, \$RR992YZ.pptx, \$IOR8H8G.txt, \$ROR8H8G.txt 생성 확인
 - 해당 파일 데이터 영역 이동

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00403000	2E	20	20	20	20	20	20	20	20	20	20	10	00	55	2E	8E	. ..U.Ž
00403010	62	5C	62	5C	00	00	2F	8E	62	5C	05	00	00	00	00	00	b\b\../Žb\.....
--- 중략 ---																	
00403060	41	24	00	49	00	52	00	39	00	39	00	0F	00	44	32	00	A\$.I.R.9.9...D2.
00403070	59	00	5A	00	2E	00	70	00	70	00	00	00	74	00	78	00	Y.Z...p.p...t.x.
00403080	24	49	52	39	39	32	7E	31	50	50	54	20	00	10	46	8E	\$IR992~1PPT ..FŽ
00403090	62	5C	62	5C	00	00	47	8E	62	5C	08	00	3C	00	00	00	b\b\..GŽb\...<...
004030A0	41	24	00	52	00	52	00	39	00	39	00	0F	00	7B	32	00	A\$.R.R.9.9...{2.
004030B0	59	00	5A	00	2E	00	70	00	70	00	00	00	74	00	78	00	Y.Z...p.p...t.x.
004030C0	24	52	52	39	39	32	7E	31	50	50	54	20	00	2E	36	8E	\$RR992~1PPT ..6Ž
004030D0	62	5C	62	5C	00	00	3D	8E	62	5C	09	00	DD	87	00	00	b\b\..=Žb\..Ý‡..
004030E0	24	49	30	52	38	48	38	47	54	58	54	20	10	9D	46	8E	\$I0R8H8GTXt ..FŽ
004030F0	62	5C	62	5C	00	00	47	8E	62	5C	12	00	36	00	00	00	b\b\..GŽb\..6...
00403100	24	52	30	52	38	48	38	47	54	58	54	20	10	40	2E	8E	\$R0R8H8GTXt .@.Ž
00403110	62	5C	62	5C	00	00	34	8E	62	5C	07	00	10	00	00	00	b\b\..4Žb\.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 일반 삭제 후 \$RECYCLE.BIN Directory 확인
 - \$IR992YZ.pptx
 - 파일 삭제 전, 원래 파일명 확인 가능 (eunchae.pptx)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00406000	02	00	00	00	00	00	00	00	DD	87	00	00	00	00	00	00	Ý#.....
00406010	00	64	D0	94	21	AA	DC	01	10	00	00	00	46	00	3A	00	.dĐ”!ªÜ.....F.:.:
00406020	5C	00	65	00	75	00	6E	00	63	00	68	00	61	00	65	00	\.e.u.n.c.h.a.e.
00406030	2E	00	70	00	70	00	74	00	78	00	00	00	00	00	00	00	..p.p.t.x.....
00406040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 일반 삭제 후 \$RECYCLE.BIN Directory 확인
 - \$RR992YZ.pptx
 - 파일 삭제 전, 원래 파일이 가지고 있던 데이터 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00407000	50	4B	03	04	14	00	06	00	08	00	00	00	21	00	DF	CC	PK.....!.ßİ
00407010	18	F5	AD	01	00	00	46	0C	00	00	13	00	08	02	5B	43	.õ....F.....[C
00407020	6F	6E	74	65	6E	74	5F	54	79	70	65	73	5D	2E	78	6D	ontent_Types].xm
00407030	6C	20	A2	04	02	28	A0	00	02	00	00	00	00	00	00	00	1 ¢..(.....
00407040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 일반 삭제 후 \$RECYCLE.BIN Directory 확인

■ \$IOR8H8G.txt

- 파일 삭제 전, 원래 파일명 확인 가능 (HELLO.txt)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00410000	02	00	00	00	00	00	00	00	10	00	00	00	00	00	00	00	
00410010	50	FF	A7	95	21	AA	DC	01	0D	00	00	00	46	00	3A	00	Pÿ\$•!ªÜ.....F.:.:
00410020	5C	00	48	00	45	00	4C	00	4C	00	4F	00	2E	00	74	00	\.H.E.L.L.O...t.
00410030	78	00	74	00	00	00	00	00	00	00	00	00	00	00	00	00	x.t.....
00410040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 일반 삭제 후 \$RECYCLE.BIN Directory 확인
 - \$ROR8H8G.txt
 - 파일 삭제 전, 원래 파일이 가지고 있던 데이터 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00405000	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
00405010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00405020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00405030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00405040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 완전 삭제 후 FAT Area 확인

■ FAT Area로 이동해서 할당된 클러스터 확인

- 6개의 클러스터 할당 확인: 예약된 클러스터 0, 1 제외, 클러스터 2 ~ 클러스터 7 할당

■ 완전 삭제 후, 할당 받았던 클러스터 해제 확인

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00203400	F8	FF	FF	0F	FF	FF	FF	FF	FF	FF	FF	0F	FF	FF	FF	0F	øÿÿ.ÿÿÿÿÿÿÿ.ÿÿÿ.
00203410	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	ÿÿÿ.ÿÿÿ.ÿÿÿ.ÿÿÿ.
00203420	00	00	00	00	0A	00	00	00	0B	00	00	00	0C	00	00	00	
00203430	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00	
00203440	11	00	00	00	FF	FF	FF	0F	00	00	00	00	00	00	00	00	ÿÿÿ.....



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00203400	F8	FF	FF	0F	FF	FF	FF	FF	FF	FF	FF	0F	FF	FF	FF	0F	øÿÿ.ÿÿÿÿÿÿÿ.ÿÿÿ.
00203410	FF	FF	FF	0F	FF	FF	FF	0F	FF	FF	FF	0F	00	00	00	00	ÿÿÿ.ÿÿÿ.ÿÿÿ.....
00203420	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00203430	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00203440	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 완전 삭제 후 Directory Entry 확인

■ HELLO.txt → 일반 삭제와 차이 X

- Starting Cluster High: 0x0 / Starting Cluster Low: 0x7 / File size: 0x10 (16 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004000E0	E5	45	4C	4C	4F	20	20	20	54	58	54	20	10	A9	AA	99	HELLO TXT .@™
004000F0	59	5C	59	5C	00	00	B5	99	59	5C	07	00	10	00	00	00	Y\Y\...μ™Y\.....

■ Eunhae.pptx → 일반 삭제와 차이 X

- Starting Cluster High: 0x0 / Starting Cluster Low: 0x9 / File size: 0x87DE (34,872 bytes)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
004002A0	E5	55	4E	43	48	41	7E	31	50	50	54	20	00	63	B8	99	UNCH~1PPT .c,™
004002B0	59	5C	59	5C	00	00	C4	99	59	5C	09	00	DE	87	00	00	Y\Y\...Ä™Y\...p#..

- 완전 삭제 이후에도 Starting Cluster High, Starting Cluster Low 값이 존재 → 복구 가능성 존재

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 완전 삭제 후 복원

■ HELLO.txt Data Area 이동

- $0x400000 + (7 - 2) * \text{Sector per Cluster} * \text{Bytes per Sector} = 0x405000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00405000	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
00405010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

■ Eunhae.pptx Data Area 이동

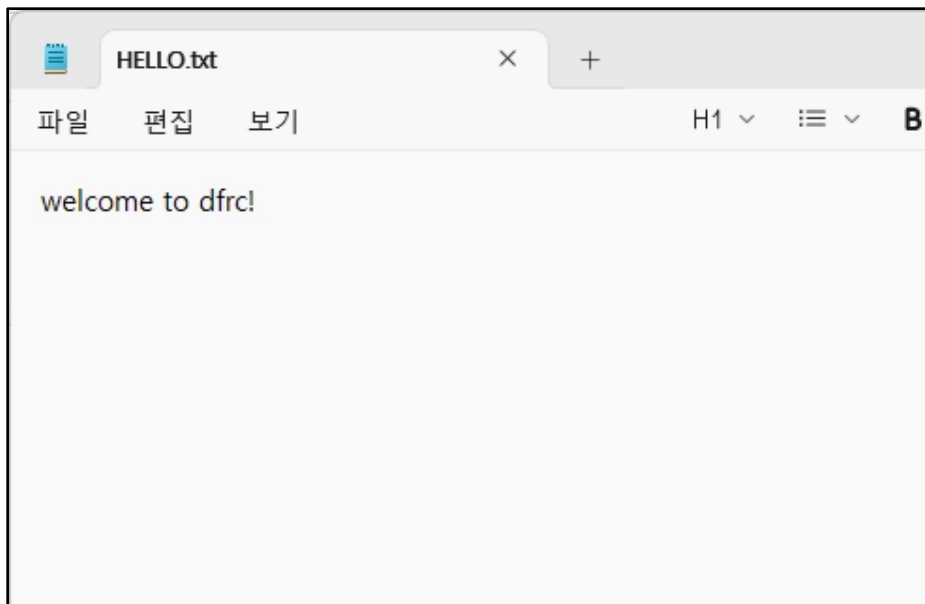
- $0x400000 + (9 - 2) * \text{Sector per Cluster} * \text{Bytes per Sector} = 0x407000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00407000	50	4B	03	04	14	00	06	00	08	00	00	00	21	00	DF	CC	PK.....!.ßİ
00407010	18	F5	AD	01	00	00	46	0C	00	00	13	00	08	02	5B	43	.õ....F.....[C

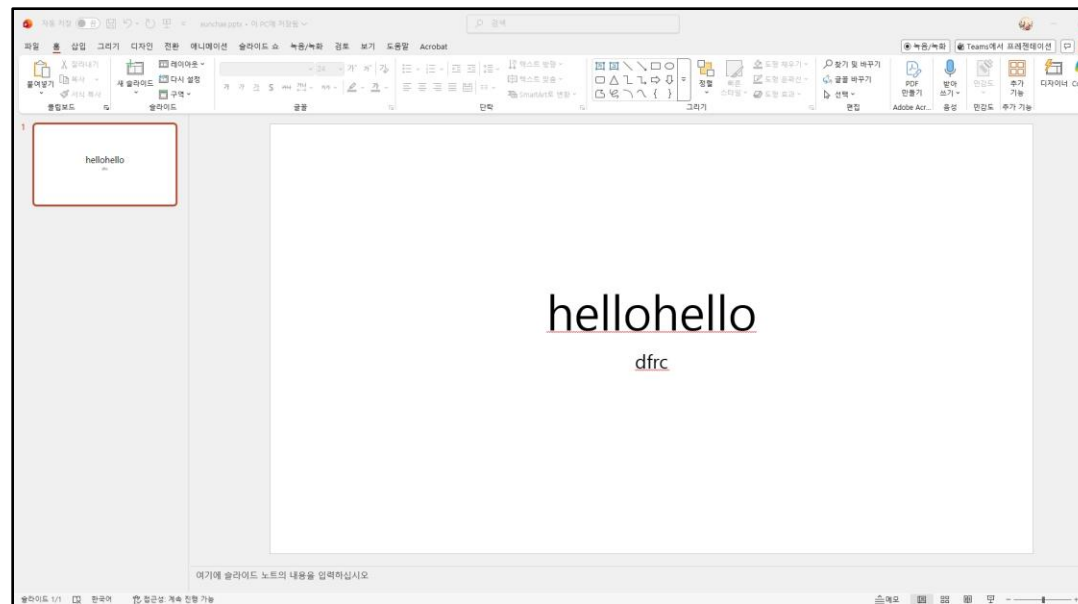
- 완전 삭제 이후에도 클러스터 내 데이터 존재 → 복원 가능

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

- 완전 삭제 후 복원
 - 클러스터 내 데이터 복사 → HxD로 새 파일 생성
 - 동일 파일 내용 확인 가능 → 복원 성공



[HELLO.txt 복원]



[eunchae.pptx 복원]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 및 복구

■ 실습 결과

■ 파일 일반 삭제 후

- FAT Area: 클러스터 할당 변화 관찰 X
- Directory Entry: 파일명 첫 글자가 0xE5로 변경
- Data Area: 데이터 잔존

■ 파일 완전 삭제 후

- FAT Area: 할당된 클러스터 해제
- Directory Entry: 파일명 첫 글자가 0xE5로 변경
- Data Area: 데이터 잔존

■ 파일 영구 삭제 이후에도 클러스터 내 데이터 잔존 → 복구 가능성 존재

NTFS

소개

■ NTFS (New Technology File System) 개요

- Windows의 기본 파일 시스템
 - Windows 11, 10, 7.1, 8, 7, Vista, XP
 - Windows Server 계열: 2022, 2019, 2016, 2012, 2008, 2003, 2000 등
 - Windows NT
- MFT (Master File Table) 중심의 데이터베이스 구조
- FAT 파일 시스템의 한계를 개선하기 위해 설계

FAT 한계	NTFS 개선
파일 크기 제한 (4GB)	대용량 파일 지원
보안 기능 부재	ACL 기반 파일 권한
단순 연결 구조	MFT 기반 구조
충돌 시 데이터 손상	저널링 기반 \$LogFile을 이용한 볼륨 보호
암호화 / 압축 기능 부재	파일 압축 및 암호화 지원

소개

■ 버전 별 NTFS

NFTS 버전	내용	출시 OS
NTFS 1.0 (1993)	• NTFS 최초 버전 • 1.1 이후 버전과 호환 불가	Windows NT 3.1
NTFS 1.1 (1995)	• ADS, ACL, 압축 파일 지원 • Windows NT 3.51과 함께 출시	Windows NT 3.51
NTFS 1.2 (1996)	• Security Descriptor 지원 • Windows NT 4.0과 함께 출시	Windows NT 4.0
NTFS 3.0 (NTFS 5.0) (2000)	• 디스크 쿼터, EFS, Sparse 파일, Reparse 포인트, USN 저널링, 분산 링크 추적, \$Extend 폴더 • 서비스 팩 업데이트를 통해 Windows NT 4.0과 호환 • Windows NT 5.0과 함께 출시	Windows 2000
NTFS 3.1 (NTFS 5.1) (2001)	• MFT Entry, Record 개선 • Windows XP 이후 NTFS 버전은 3.1 사용	Windows XP

소개

■ 기존 파일 시스템 대비 발전된 특징

특징	세부 기능 및 지원 요소
확장성 및 호환성	• 대용량 파일 지원 • Unicode 지원 • ADS (Alternate Data Streams)
저장 공간 유연성	• 압축 기능 지원 • Sparse 파일 속성
안정성	• 저널링 기반 데이터 복구 • 동적 Bad Cluster 탐지 및 재할당 • VSS (Volume Shadow Copy Service) 지원
보안성	• ACL 기반 세부 권한을 통한 보안 기능 • 디스크 쿼터 • EFS 암호화

소개

■ 대용량 파일 지원[추가]

- 클러스터 크기가 최대 파일 및 볼륨 크기에 영향

- 이론상 최대 볼륨 및 파일 크기

구분	볼륨	파일
이론상 최대 크기	8 YB	16 EB
현실상 최대 크기	256 TB	256 TB

- 최대 볼륨 크기: $512 \text{ KB} * 2^{64} = 8 \text{ YB}$ (최대 클러스터 수: 2^{64} / 클러스터 크기 512 KB 기준)

- 최대 파일 크기: 16 EB

- Non - Resident Attribute Header의 Allocated Size of Attribute Content (8 bytes 크기, 2^{64})

- 현실상 최대 볼륨 및 파일 크기

- 최대 볼륨 크기: $64 \text{ KB} * 2^{32} = 256 \text{ TB}$

- LCN 표현 필드 8 bytes 중 실사용은 4 bytes → 클러스터 수 2^{32}

- I/O 매핑 및 캐싱 최대 64 KB 제한 → 클러스터 크기 64 KB로 제한

- 최대 파일 크기: 256 TB (볼륨 최대 크기 초과 불가)

소개

■ Unicode

- NTFS는 UTF-16 인코딩을 사용하여 파일 및 디렉터리 이름을 저장
- NTFS에 저장되는 스트림은 모두 유니코드 지원하며 다국어 지원 가능

■ ADS (Alternate Data Streams)

- NTFS에서 하나의 파일에 여러 개의 데이터 스트림을 저장할 수 있는 기능 → 확장성
- 파일 크기나 탐색기에서 확인 불가
- ADS 설정 파일은 file.txt:hidden과 같이 명시된 스트림에 접근
- ADS 파일의 이동 / 복사
 - 다른 NTFS 볼륨으로 이동 / 복사 시 ADS 유지
 - NTFS가 아닌 볼륨 이동 / 복사 시 ADS 제거, 메인 스트림만 유지

소개

■ ADS 사용 이유[추가]

■ Resource Fork

- macOS의 HFS (Hierarchical File System)에서 사용
- 하나의 파일을 Data Fork, Resource Fork로 구성
 - Data Fork: 파일의 실제 본문 내용
 - Resource Fork: 파일 관련 메타데이터, 설정 등 부가 정보 저장

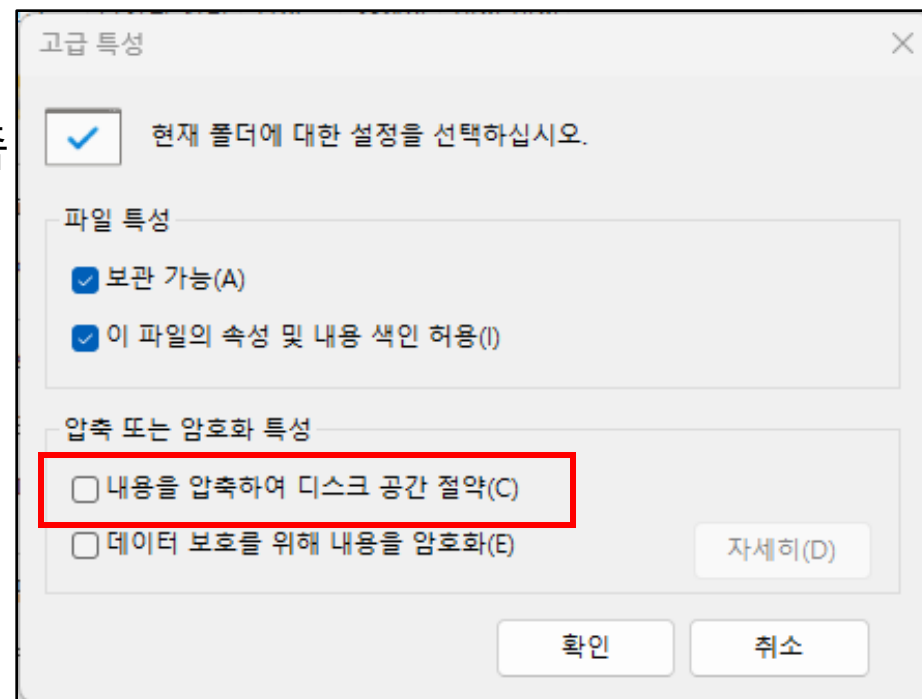
■ Windows 서버 환경에서 macOS 클라이언트와의 호환성을 위해 도입

- HFS 파일을 NTFS로 복사 → Resource Fork 손상
- Resource Fork의 손상을 막기 위해 ADS 사용 가능

소개

■ 압축 기능 지원[추가]

- NTFS는 파일 단위뿐 아니라 폴더 단위에서도 압축 기능 제공
- 압축된 데이터는 사용 시 실시간으로 해제되며, 저장 시 자동으로 다시 압축
- 기본 압축 방식은 LZNT1 알고리즘 이용
 - Windows NT 계열에서 사용하는 NTFS 전용 압축 알고리즘
 - 무손실 압축을 지원 → 안정성 있는 압축 알고리즘
 - 데이터는 16개 클러스터 단위의 청크 구조로 압축 처리

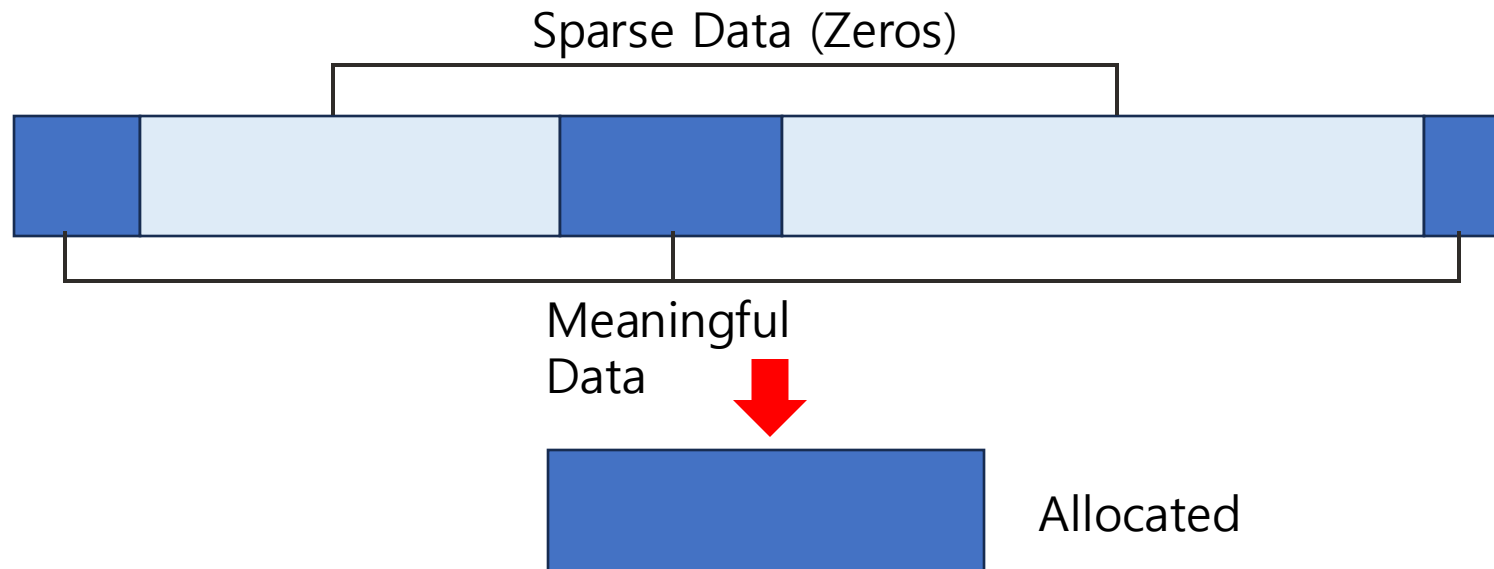


[파일 / 폴더 속성 → 일반 → 고급
탭]

소개

■ Sparse 속성[추가]

- NTFS 5.0 이후 추가된 속성 / \$DATA Attribute에서만 사용
- 파일 데이터가 대부분 0인 경우 실제 데이터 저장 X, 크기만 기록 → 저장장치 효율성 증가
- 클러스터 할당 X → 실제 데이터가 기록되지 않고 메타데이터만 존재



소개

- 동적 Bad Cluster 재할당[추가]
 - 읽기 / 쓰기 중 물리적 오류 발생 시 디스크가 OS에 전달
 - OS는 \$BadClus:\$Bad에 불량 클러스터 등록 및 데이터는 정상 클러스터로 재배치
 - 추가 손상 확산 방지 및 데이터 무결성 유지
- VSS (Volume Shadow Copy Service)
 - 실행 중인 시스템에서도 일관된 시점의 스냅샷 생성
 - Copy - On - Write 방식으로 변경 직전 데이터를 별도 Shadow 영역에 보존 후 변경 수행
 - 파일 단위가 아닌 블록 단위 동작
 - 시스템 속성 → 시스템 보호를 통해 설정 가능

소개

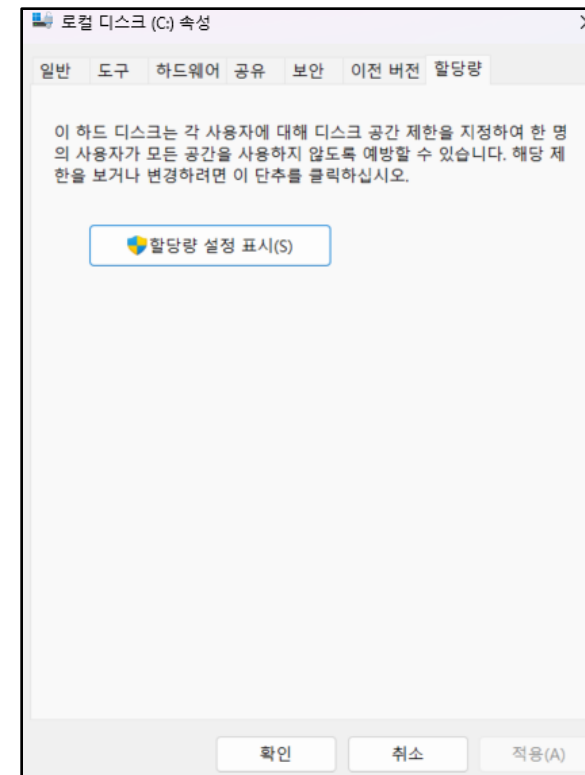
■ 저널링[추가]

- NTFS 볼륨 / 파일 및 폴더에 변경이 생길 때마다 변경 사항을 기록하는 행위
- \$LogFile: NTFS 파일 시스템 변경에 대한 트랜잭션 로그 기록
 - LSN (Log Sequence Number) 기준 순차적 저장
 - 시스템 장애 발생 시 로그를 기반으로 자동 복구 수행 → 파일 시스템 일관성 유지
- \$UsnJrnl: NTFS 볼륨 내 파일 / 폴더 변경 로그 기록
 - USN (Update Sequence Number) 기준 순차적 저장
 - 변경 추적 목적 → 그 자체로 복구 기능 X
 - VSS 스냅샷 생성 → 해당 시점의 \$UsnJrnl, 파일 데이터가 함께 포함될 수 있음 → 데이터 복구 가능성 존재

소개

■ 디스크 쿼터

- NTFS 볼륨에서 사용자 단위 디스크 사용량을 추적 및 제한
- 할당량 정보는 `₩$Extend₩$Quota`에 저장
- 폴더 단위가 아닌 볼륨 단위로 적용
- 동작 방식
 - 사용자가 파일 생성, 수정 시 소유자 SID 기준으로 용량 누적
 - 논리적 파일 크기 기준 계산
 - 압축 파일 또한 압축 전 논리 크기를 기준으로 집계
 - 임계값 초과 시 경고 로그 또는 쓰기 차단

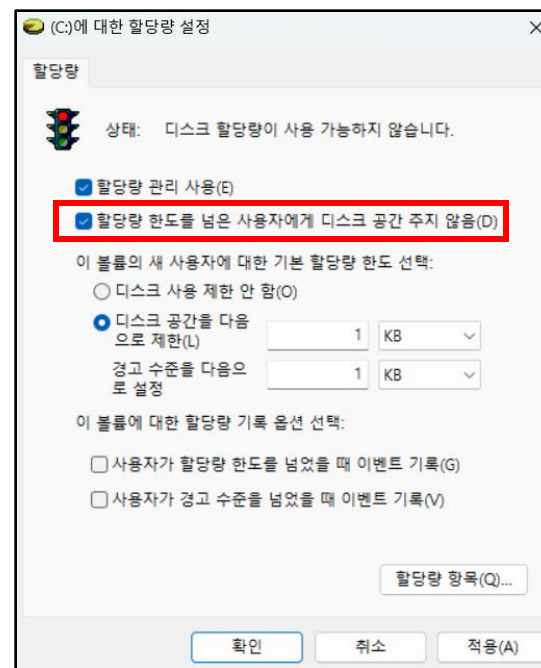
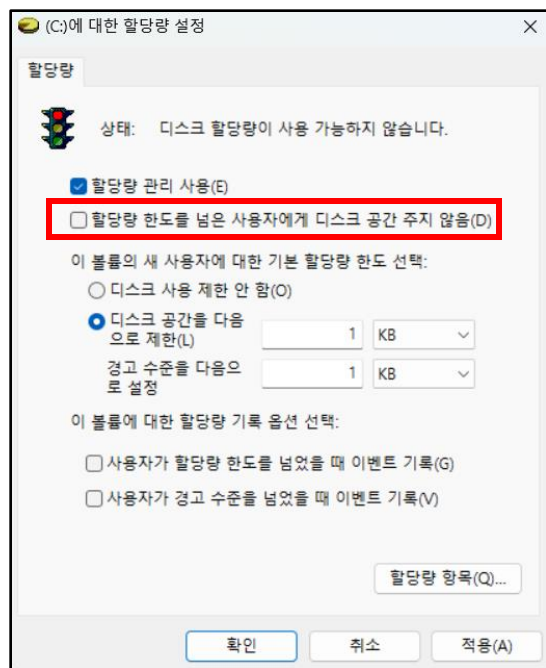


[로컬 디스크 (C:) 속성 - 할당량]

소개

■ 디스크 쿼터

- '할당량 한도를 넘은 사용자에게 디스크 공간 주지 않음' 체크 여부에 따른 변화
 - 체크 X: 경고만 주며 저장 가능
 - 체크 O: 경고와 함께 저장 불가능



[(C:)에 대한 할당량 설정]

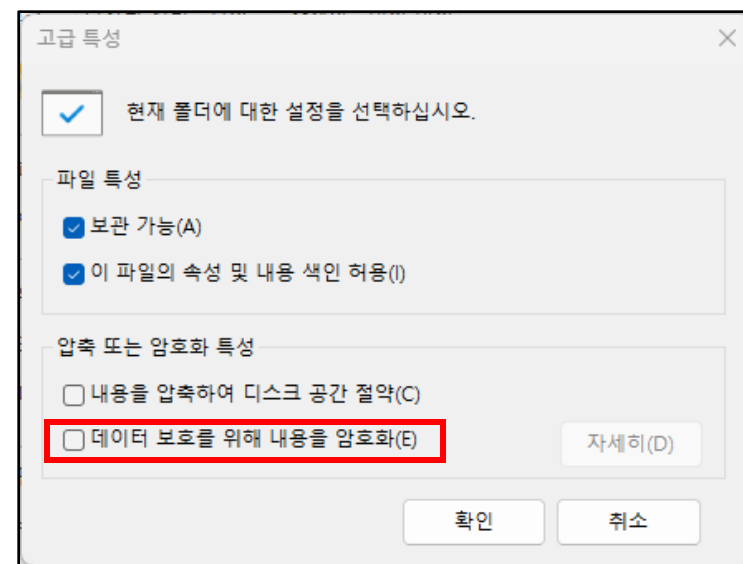
소개

■ 암호화 파일 시스템 (Encrypting File System, EFS)[추가]

- NTFS에서 제공하는 파일 / 폴더 단위 데이터 암호화 기능
- 권한 있는 사용자 계정에서만 자동 복호화 되어 접근 가능
- 암호화된 상태로 이동 또는 복사 시
 - NTFS 볼륨 내 → 암호화 유지
 - EFS가 지원되지 않는 파일시스템 이동 또는 복사 → 암호화 제거

■ 동작 과정

- 파일 데이터는 FEK(File Encryption Key)로 암호화
- FEK 키를 사용자 RSA 공개키로 다시 암호화
- 암호화된 FEK는 각 파일 / 폴더 MFT Entry \$LOGGED_UTILITY_STREAM에 저장 (Attribute Type ID 0x100)



[파일 / 폴더 속성 → 일반 → 고급 탭]

소개

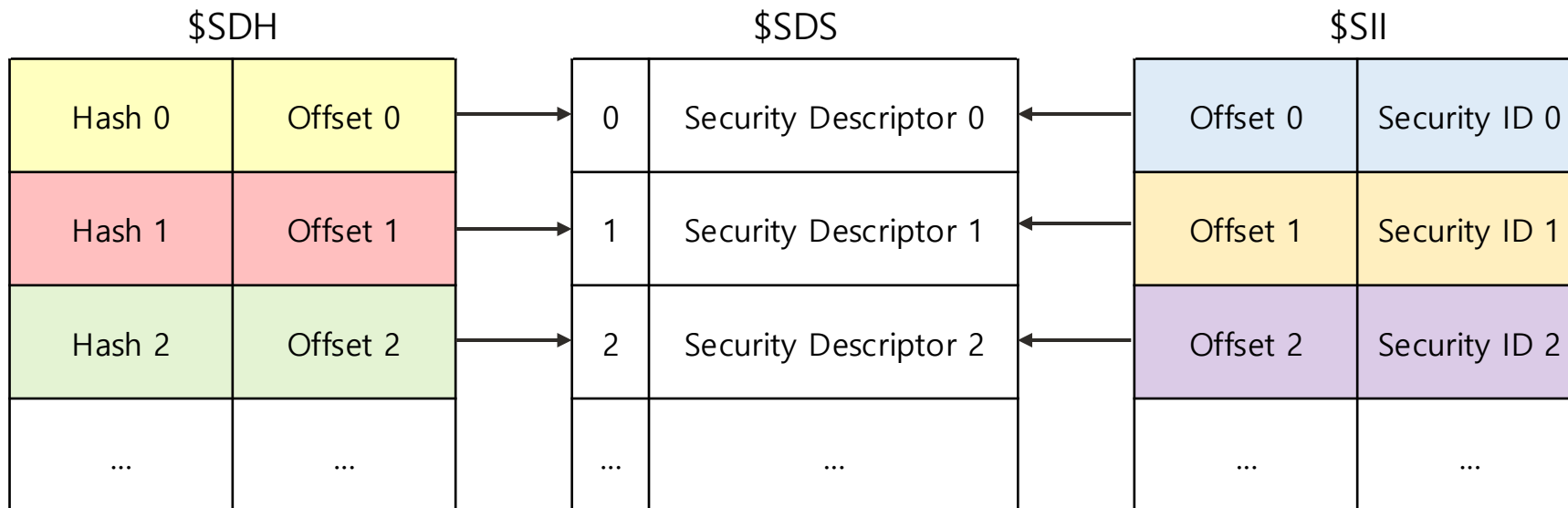
■ ACL 기반 접근 제어

- NTFS는 파일 / 폴더 단위로 접근 권한 분리
- 각 객체 보안 설명자를 보유
- 사용자 및 그룹의 SID (Security Identifier) 기준으로 권한 적용
- ACL 구성 요소
 - DACL (Discretionary ACL): 실제 접근 허용 / 거부 권한 정의된 ACE (Access Control Entry)들의 집합
 - SACL (System ACL): 접근 시도에 대한 감사 설정
- ACL 동작 방식
 - 사용자가 객체 접근 시, 시스템이 사용자 SID 및 그룹 SID 확인
 - DACL의 ACE 순서대로 확인 → 명시적 거부가 우선 적용, 허용 조건 충족 시 접근 승인
 - 접근 시도 성공 / 실패에 대한 감사 기록

소개

■ ACL 접근 방식[추가]

- \$Secure: MFT Entry 9번에 위치 → 각 파일의 보안 / 접근 권한 정보 포함
 - \$SDS (Security Descriptor Stream): 볼륨에 존재하는 모든 파일 / 폴더 Security Descriptor 저장
 - \$SDH (Security Descriptor Hash): Security Descriptor에 대한 해시 값 저장
 - \$SII (Security ID Index): 파일 / 폴더 MFT Entry의 \$STANDARD_INFORMATION의 Security ID 값 저장



소개

■ Zone.Identifier

- Windows에서 인터넷 등 외부 출처에서 다운로드된 파일에 자동으로 부여되는 보안 정보
- NTFS의 ADS 기능 이용하여 저장
- 동작 과정
 - 브라우저 / 메일 클라이언트가 파일 저장
 - Windows Attachment Manager가 출처 영역 판단
 - 해당 파일에 Zone.Identifier ADS 생성
 - 실행 시 Windows가 Zone 정보 확인
 - Internet Zone이면 경고창 표시

소개

■ Zone.Identifier ID[추가]

■ Zone ID 및 설명

Zone ID	유입 경로	설명
0	Local machine	로컬 컴퓨터에서 생성 된 파일
1	Local intranet	내부 네트워크에서 다운로드 된 파일
2	Trusted sites	신뢰할 수 있는 사이트에서 다운로드 된 파일
3	Internet	인터넷에서 다운로드 된 파일
4	Restricted sites	제한된 사이트에서 다운로드 된 파일

■ 인터넷 속성 → 보안 탭에서 설정 가능

- 인터넷, 로컬 인트라넷, 신뢰할 수 있는 사이트, 제한된 사이트



[인터넷 속성 - 보안]

전체 구조

■ 개요



■ VBR (Volume Boot Record)

- NTFS의 가장 앞부분에 위치하는 영역
- 부팅에 필요한 코드와 NTFS 핵심 파라미터 저장
- 클러스터 크기에 따라 VBR 크기 결정

■ MFT (Master File Table)

- 볼륨 내 모든 파일과 디렉터리 정보를 엔트리 단위로 저장

■ Data Area

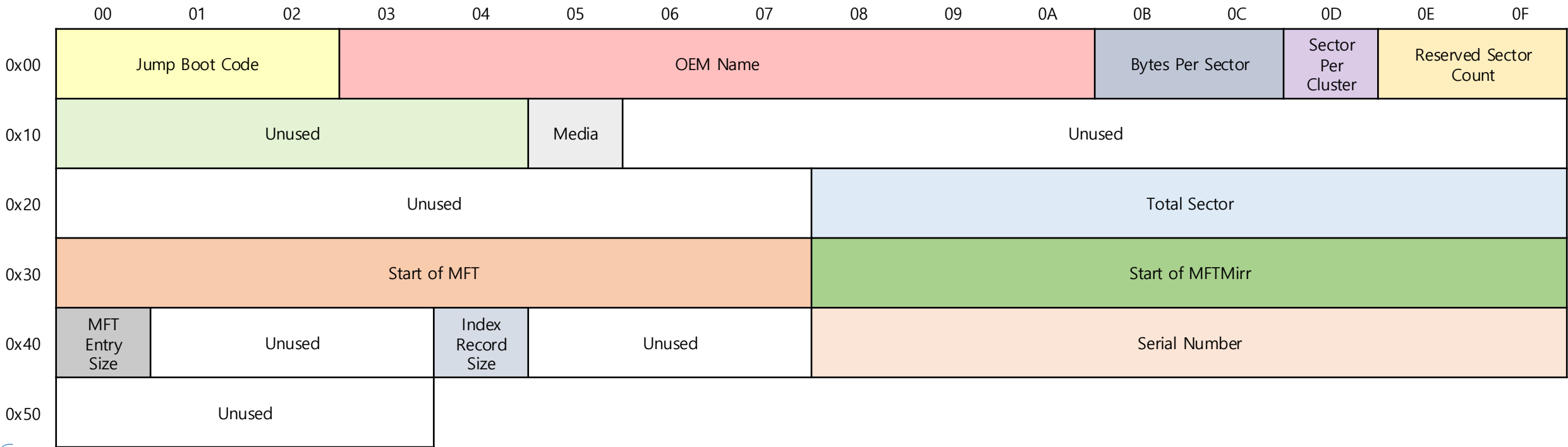
- 실제 데이터가 존재하는 영역 → Resident 여부에 따라 데이터 위치가 파일 내 / 외 결정

전체 구조

■ VBR (Volume Boot Record)

■ Boot Sector

- NTFS 볼륨의 시작 부분에 위치하는 부트 영역 / BIOS, UEFI가 파티션 부팅 시 가장 먼저 읽는 영역
- 파일 시스템 식별 정보와 부트 코드 포함



전체 구조

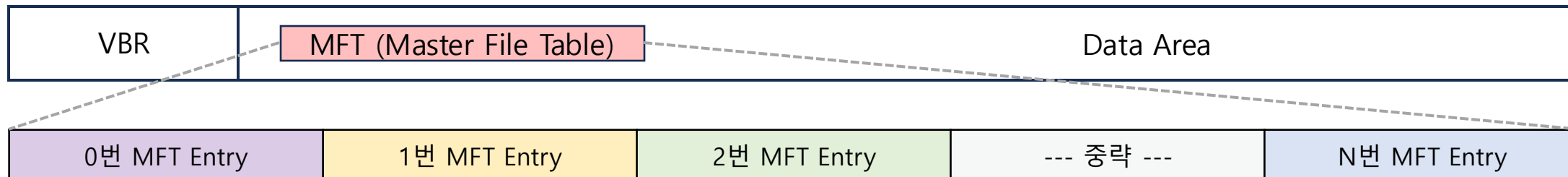
■ VBR (Volume Boot Record)

■ Bootstrap Code Area

- 부트 섹터 이후의 실행 코드 영역
- NTFS 볼륨에서 부트 로더 (BOOTMGR 또는 NTLDR)를 찾아 실행하는 코드
- 동작 과정
 - Windows XP / 2003: VBR → NTLDR 실행 → boot.ini 파일 읽기 → OS 선택 → 커널 로드
 - Vista 이후: VBR → BOOTMGR 실행 → BOOTMGR이 BCD 읽기 → winload.exe 실행 → 커널 로드

구분	boot.ini	BCD
적용 버전	XP / 2003	Vista 이후
저장 형태	텍스트 파일	레지스트리 형식 DB
저장 위치	루트 디렉터리	\\Boot\\BCD
관리 방식	수동 편집	bcdedit 도구

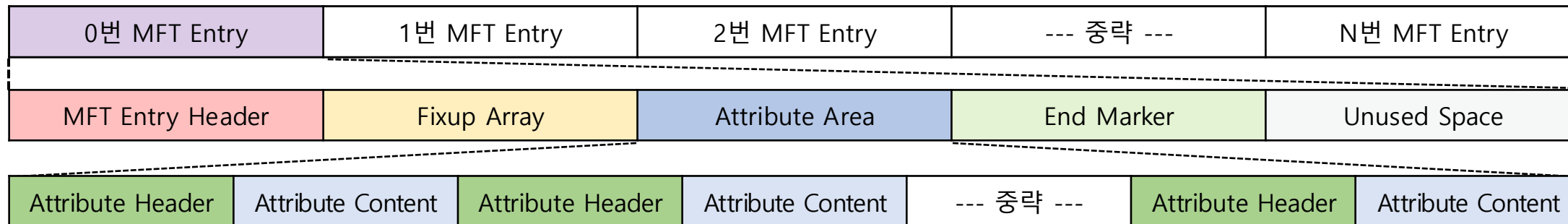
전체 구조



■ MFT (Master File Table)[추가]

- MFT 자체도 하나의 파일 (MFT Entry #0)으로 관리되며 파일 증가에 따라 동적으로 확장
- MFT의 크기가 부족할 경우 대비해 MFT Zone을 통해 MFT 추가 공간 예약
 - Registry 값 조절을 통해 MFT Zone 크기 조절 가능 (1: 12.5%, 2: 25%, 3: 37.5%, 4: 50% / 볼륨 비율)
 - HKLM\System\CurrentControlSet\Control\FileSystem\MFTZoneReservation

전체 구조



■ MFT Entry 구조

- MFT Entry Header: 48 bytes를 통해 MFT Entry의 메타 정보를 표현
- Fixup Array: 데이터 읽기·쓰기 시 무결성 검증에 사용되며, 섹터 단위의 손상 탐지
- Attribute: 각 파일은 End Marker가 나오기 전까지 여러 개의 속성을 가지는 것 가능
 - Attribute Header: 각각의 파일 메타 정보를 가지며, 속성마다 공통 헤더, Res / Non-Res 헤더를 보유
 - Attribute Content: 속성의 내용 저장, 속성마다 저장되는 내용 상이

전체 구조

■ Fixup Array의 목적 및 동작 과정[추가]

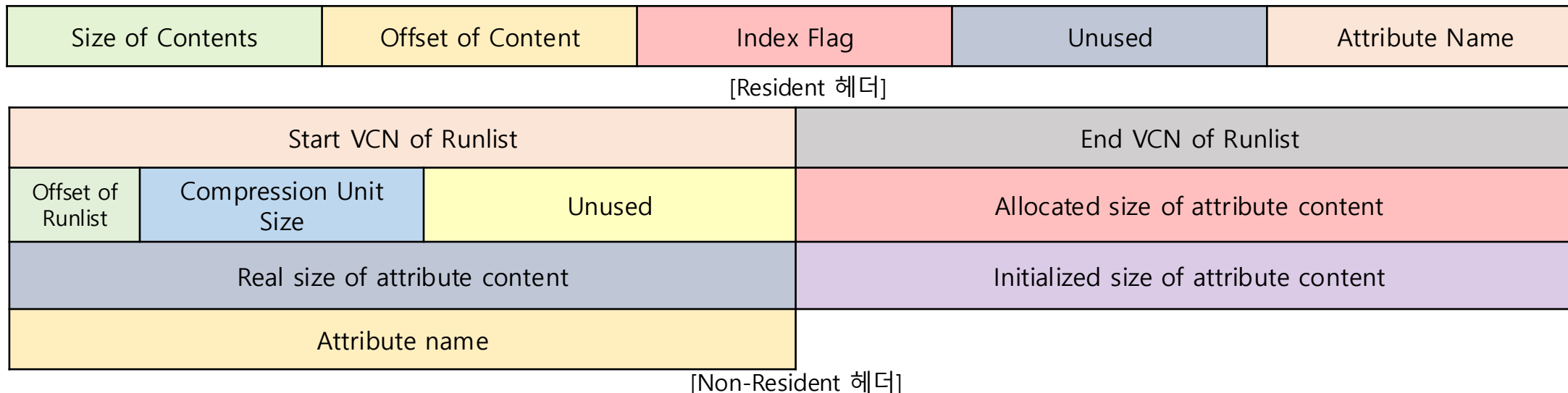
■ 목적

- 한 MFT Entry (1,024 bytes)가 중간 손상 없이 온전히 기록되었는지에 대한 무결성 검사
- Entry 손상 시 오류 반환하고, 추후 복구 로직을 통해 복구 시도
 - 읽기 재시도, chkdsk, \$LogFile

■ 동작 과정

- Step 1: Fixup Array의 특정 시그니처 생성
- Step 2: 각 섹터의 마지막 2 bytes를 Fixup Array에 저장 후 시그니처로 덮어쓰기
- Step 3: 읽기 검사 시 각 섹터 시그니처와 일치하는 지 확인
- Step 4: 시그니처와 일치 시 기존 값으로 복원
- Step 5: 시그니처와 불일치 시 Entry 손상 판단

전체 구조



■ Data Area

- 클러스터 단위로 관리하며, 데이터 저장 시 하나 이상의 클러스터가 할당
- Resident vs. Non-Resident
 - Resident: 데이터가 MFT entry 내부에 직접 저장되는 방식
 - Non-Resident: 데이터가 Data Area에 클러스터로 저장 → Run list를 통해 클러스터 위치 계산

전체 구조

■ Cluster Run

- 하나의 연속된 클러스터 구간으로 LCN (Logical Cluster Number)으로 표현
- 파일의 데이터는 Cluster Run의 집합

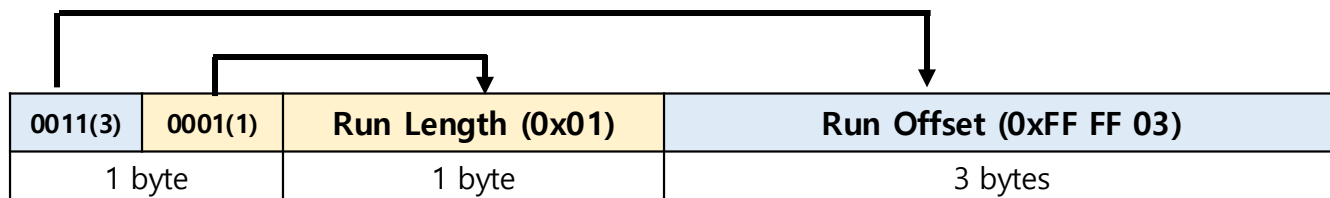
■ Run List

- Cluster Run을 나열한 목록 / 첫 바이트를 4 bits씩 나눈 후, Run Length와 Run Offset 크기 표현
 - 파일이 연속된 공간에 저장되지 않고 조각나 있는 경우, 여러 개의 Run List로 구성
 - 2번 Run List 부터 이전 Starting LCN + Run Offset으로 현재 Starting LCN 계산
 - 음수의 경우 2의 보수 취한 후 계산 필요

전체 구조

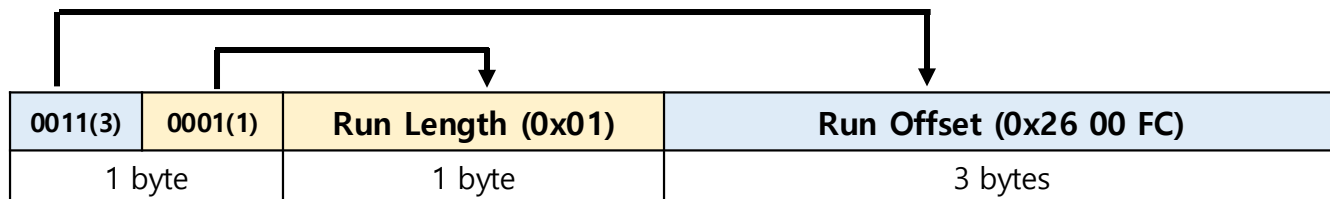
■ Run List 계산 방법[추가]

- Run Offset이 양수일 때 계산 방식: 이전 Starting LCN + 현재 Run Offset
 - 이전 Starting LCN 없을 경우 0 + 현재 Run Offset



1. 이전 Starting LCN 파악: 0
2. 첫 번째 Cluster Run Offset 계산:
 $0 + 0x3FFFF * 0x1000 \text{ (Cluster Size)} = 0x3FFFF000$

- Run Offset이 음수일 때 계산 방식: 2의 보수 계산 후 이전 Starting LCN - Run Offset
 - 음수의 경우 이전 Starting LCN 없을 경우는 논리적으로 불가 → 반드시 이전 Starting Run 존재



1. Run Offset의 2의 보수 계산: 0x3FFDA
2. 이전 Starting LCN 파악: 0x3FFFF000
3. 두 번째 Cluster LCN Offset 계산:
 $0x3FFFF000 - 0x3FFDA * 0x1000 \text{ (Cluster Size)} = 0x25000$

메타데이터 및 구조체 분석

번호	이름	설명
0	\$MFT	MFT에 대한 MFT Entry (MFT 영역의 크기, 위치, 할당 정보 등 저장)
1	\$MFTMirr	0~3번 Entry 파일 정보 복사
2	\$LogFile	메타데이터의 트랜잭션 저널 정보
3	\$Volume	볼륨의 레이블, 식별자, 버전 등 정보
4	\$AttrDef	속성의 식별자, 이름, 크기 등 정보
5	.(Root Directory)	디렉터리 구조의 파일을 빠르게 접근하기 위해 Index 구조로 저장
6	\$Bitmap	볼륨 클러스터 할당 정보
7	\$Boot	부팅 가능한 볼륨인 경우 부트섹터 정보
8	\$BadClus	배드 섹터를 가지는 클러스터 정보
9	\$Secure	파일의 보안, 접근 제어 관련 정보

번호	이름	설명
10	\$Uppcase	모든 유니코드 문자의 대문자
11	\$Extend	추가 메타데이터 파일을 저장하는 파일 (\$ObjID, \$Quota, \$Reparse, \$UsnJrnl)
12-15	사용 안함	미래를 위해 예약
16-23	사용 안함	포맷 후 생성되는 파일 정보
-	\$ObjID	파일의 고유 ID 정보 저장
-	\$Quota	사용량 정보 저장
-	\$Reparse	Reparse Point에 대한 정보 저장
-	\$UsnJrnl	파일이나 디렉터리 변경사항 정보 저장
24 이후	일반 파일	일반적인 파일이나 디렉터리 실제 저장 위치

메타데이터 및 구조체 분석

■ \$MFT 주요 정보

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C0000000	46	49	4C	45	30	00	03	00	95	52	00	02	00	00	00	00	FILE0...•R.....
00C0000010	01	00	01	00	38	00	01	00	A0	01	00	00	00	04	00	00	8...
00C0000020	00	00	00	00	00	00	00	00	07	00	00	00	00	00	00	00	
00C0000030	02	00	00	00	00	00	00	00	10	00	00	00	60	00	00	00	`...

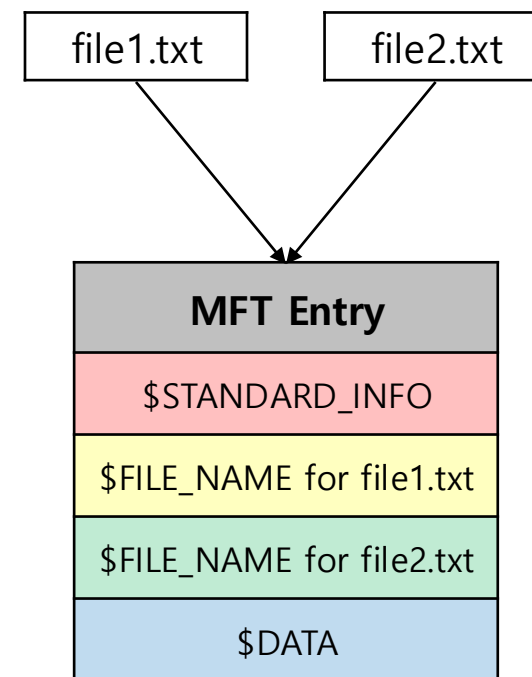
Offset	Field Name	Size	Description / Value
0x00	Signature	4 bytes	정상 시: FILE, 오류 시: BAAD / 0x46 49 4C 45 = FILE
0x08	\$LogFile 트랜잭션 위치	8 bytes	해당 Entry가 기록되는 트랜잭션 위치 / 0x20005295
0x14	첫 번째 속성 오프셋	2 bytes	첫 번째 속성 오프셋 / 0x38
0x2C	Entry 번호	4 bytes	MFT Entry 번호 / 0x00 00 00 00 = 0번

속성 타입	속성명	오프셋 / 범위	주요 관찰 필드 / 값
0x10	\$STANDARD_INFORMATION	0x38 / 0x38~0x97	생성, 수정, MFT 파일 수정, 마지막 접근 시간 / 2026-02-16 16:57:33 KST
0x30	\$FILE_NAME	0x98 / 0x98~0xFF	파일 이름 / MFT
0x80	\$DATA	0x100 / 0x100~0x147	Run List 값 / \$MFT의 주소, 64개 클러스터
0xB0	\$BITMAP	0x148 / 0x148~0x197	사용중인 Entry / 0~15번, 24~39번, 46번, 47번

전체 구조

■ \$FILE_NAME 개수[추가]

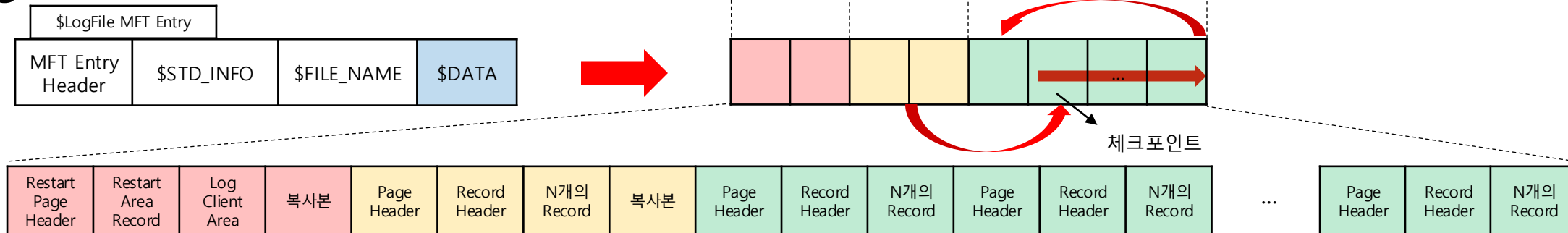
- 하드링크 수에 따라 \$FILE_NAME의 개수 변화
 - 하드링크: 여러 파일 이름이 동일한 MFT Entry 및 \$DATA를 가리키게 하는 기술
 - 하드링크 수만큼 MFT Entry 내부에 \$FILE_NAME 속성 존재
 - 하드링크 수는 MFT Entry 헤더의 Hard Link Count에서 관리
- 하드링크 파일이 삭제되는 경우
 - 파일 삭제는 없으며, 단순히 Hard Link Count 하나 감소
 - Hard Link Count가 0이 되면 \$DATA가 제거되며 파일 삭제



[하드 링크 예시 구조]

메타데이터 및 구조체 분석

■ \$LogFile 전체 구조 및 설명

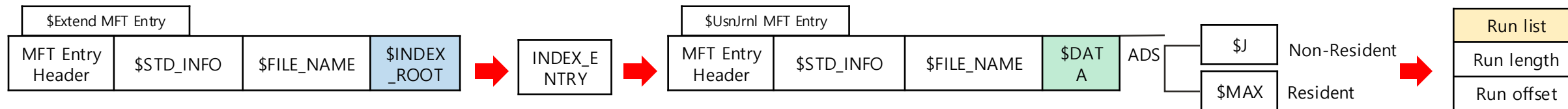


영역	구분	각 구분 설명	특징	오류 복구 과정
재시작 영역	Restart Page Header	\$LogFile의 페이지 크기, 재시작 오프셋 등 기록	- 오류 복구 시 체크포인트를 통해 복구 시작 위치 파악 - 복구 범위 축소 - 로그 파일 구조 정보 저장	재시작 영역 읽어 체크포인트 파악 → 체크포인트 이후 트랜잭션 체인 구성 → Redo / Undo 적용 → 체크포인트 갱신
	Restart Area Record	최근 작업 LSN, 클라이언트 정보 등 기록		
	Log Client Area	클라이언트 정보, 가장 오래된 / 최근 작성된 LSN 등 기록		
로깅 영역	Page Header	페이지 내 최근 레코드의 LSN, 트랜잭션 수행 시 필요 페이지 수 등 기록	- 작성할 레코드 위치 제공 - 실제 로그 데이터 기록 - 버퍼 페이지 → 일반 페이지 작성 - 링 버퍼 구조* 동작	
	Record Header	현재 작업 및 이전 작업 레코드 LSN, 트랜잭션 ID 등 기록		
	Record	실제 Redo, Undo Opcode 및 데이터 기록		

*링 버퍼 구조: 할당된 로그 영역에 순차적으로 로그 데이터가 기록되는데, 공간이 다 차게 되면 맨 앞 부터 다시 순차적으로 덮어 쓰는 방식

메타데이터 및 구조체 분석

■ \$UsnJrnl 전체 구조 및 설명



파일명	구성	관찰 가능 정보
\$UsnJrnl	\$J	USN 값, 이벤트 발생 시각, 이벤트 플래그, 이벤트 발생 주체 정보, 이벤트 적용 객체 등
	\$MAX	로그데이터 최대 크기, \$UsnJrnl 파일 생성시각, \$UsnJrnl 시작 레코드 등

■ 데이터 저장 방식

- 파일 / 디렉터리 변경 원인 정보 기록 → 데이터 저장은 X
- Journal Entry 구조체 단위로 정보 저장 → 속성 가장 뒤에 기록하는 방식 (Append 방식)
 - 최대 허용 크기를 넘는 경우 → 앞쪽 Old Journal Entry를 Sparse 처리

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ 실습 환경

- OS: Microsoft Windows 11 Home
- 논리 디스크: N (97.6 GB)
- 생성 폴더 / 파일: NTFS_Find_file_1 / ntfs_minhyuk_find_file_1.txt

■ 실습 결과

- VBR을 통해 MFT Entry를 찾고, \$. (Root Directory)에서 \$INDEX_ROOT, \$INDEX_ALLOCATION 분석을 통해 생성한 폴더 위치 파악
- 파악한 폴더 위치에서 생성한 파일 확인 가능 및 위치 파악
- 파악한 파일 위치에서 \$DATA 분석을 통해 콘텐츠 확인 가능

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ \$MFT 시작 위치 계산

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0000000000	EB	52	90	4E	54	46	53	20	20	20	20	00	02	08	00	00	ëR.NTFS
0000000010	00	00	00	00	00	F8	00	00	3F	00	FF	00	00	08	00	00	ø...?.ÿ.....
0000000020	00	00	00	00	80	00	80	00	FF	E7	34	0C	00	00	00	00	€.€.ÿç4.....
0000000030	00	00	0C	00	00	00	00	00	02	00	00	00	00	00	00	00	

Offset	Field Name	Size	Description / Value
0x0B	Byte Per Sector	2 bytes	섹터 당 바이트 / 0x200 = 512 bytes
0x0D	Sector Per Cluster	1 bytes	클러스터 당 섹터 / 0x08
0x30	Start Cluster for \$MFT	8 bytes	\$MFT 시작 클러스터 / 0x0C 00 00 *\$MFT 주소 = 0x0C0000 * 0x1000 = 0xC0000000

■ \$MFT 시작 위치 = (Start Cluster of \$MFT) * (Bytes Per Sector) * (Sector Per Cluster)

= 0x0C0000 * 0x200 * 0x8 = 0xC0000000

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ Root Directory MFT Entry - \$INDEX_ROOT 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C00015E0	90	00	00	00	58	00	00	00	04	18	00	00	00	06	00		X.....
00C00015F0	38	00	00	00	20	00	00	00	24	00	49	00	33	00	07	00	8... ..\$.I.3...
00C0001600	30	00	00	00	01	00	00	00	00	10	00	00	01	00	00	00	0.....
00C0001610	10	00	00	00	28	00	00	00	28	00	00	00	01	00	00	00	(...(.....

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Resident Header (8 bytes) / 0x90 (\$INDEX_ROOT), 0x00 (Resident 방식)
0x18	Attribute Name	8 bytes	속성명 / \$I30
0x20	Type of Attribute in Index	4 bytes	인덱스의 속성 유형 / 0x30: \$FILE_NAME
0x30	Offset to First Index Entry	4 bytes	첫 번째 인덱스 엔트리 위치 / 0xC0001610 + 0x10 = 0xC0001620

■ Root directory의 MFT Entry 위치

- Root Directory의 MFT Entry 번호: 5번 → $0xC0000000 + 0x5 * 0x400 = 0xC0001400$

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ Root Directory MFT Entry - \$INDEX_ALLOCATION 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C0001630	00	00	00	00	00	00	00	00	A0	00	00	00	50	00	00	00	P...
00C0001640	01	04	40	00	00	00	08	00	00	00	00	00	00	00	00	00	..@.....
00C0001650	00	00	00	00	00	00	00	00	48	00	00	00	00	00	00	00	H.....
00C0001660	00	10	00	00	00	00	00	00	00	10	00	00	00	00	00	00	
00C0001670	00	10	00	00	00	00	00	00	24	00	49	00	33	00	30	00	\$.I.3.0.
00C0001680	11	01	24	00	00	00	00	00	B0	00	00	00	28	00	00	00	..\$......°...(...

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	64 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0xA0 (\$INDEX_ALLOCATION), 0x01 (Non-Resident 방식)
0x18	Attribute Name	8 bytes	속성명 / \$I30
0x20	Run List	3 bytes	Run List / Run length: 0x01, Run offset: 0x24 *데이터가 존재하는 클러스터 위치 0x24 * 0x1000 = 0x24000

- \$INDEX_ALLOCATION Run List = 0x24 * 0x1000 = 0x24000

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ \$INDEX_ALLOCATION의 'NTFS_Find_file_1' 폴더 Index Entry 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0000024560	26	00	00	00	00	00	01	00	78	00	62	00	00	00	00	00	&.....x.b.....
0000024570	05	00	00	00	00	00	05	00	09	22	51	F3	25	AA	DC	01	"Qó%aÜ.
--- 중략 ---																	
00000245B0	10	00	4E	00	54	00	46	00	53	00	5F	00	46	00	69	00	..N.T.F.S._.F.i.
00000245C0	6E	00	64	00	5F	00	46	00	69	00	6C	00	65	00	5F	00	n.d._.F.i.l.e._.
00000245D0	31	00	00	00	00	00	01	00	32	00	00	00	00	00	01	00	1.....2.....

Offset	Field Name	Size	Description / Value
0x00	File Reference Address	8 bytes	Index Entry가 참조하는 파일 또는 디렉터리의 MFT 주소 값 / 0x0001000000000026 (MFT Entry 38번)
0x08	Length of This Index Entry	2 bytes	Index Entry의 총 크기 / 0x78
0x0A	Offset to End of Name	2 bytes	이름의 끝 위치 / 0x62
0x0C	Index Entry Flags	4 bytes	해당 Index Entry의 Sub Node 존재 및 마지막 Node인지 알려줌 / 0x00 (0x00: Sub Node 없음 / 0x01: Sub Node 존재 / 0x02: Node 마지막)
0x18	\$FILE_NAME Attribute	-	\$FILE_NAME 속성 Header + Contents

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ \$INDEX_ALLOCATION의 'ntfs_Minhyuk_find_file_1.txt' 파일 Index Entry 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C0009980	2A	00	00	00	00	00	01	00	90	00	7A	00	00	00	00	00	*.....z.....
00C0009990	26	00	00	00	00	00	01	00	98	CF	1E	FB	25	AA	DC	01	&.....~İ.û%äÜ.
--- 중략 ---																	
00C00099D0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C00099E0	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C00099F0	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	06	00	n.d._.f.i.l.e...
00C0009A00	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....

Offset	Field Name	Size	Description / Value
0x00	File Reference Address	8 bytes	Index Entry가 참조하는 파일 또는 디렉터리의 MFT 주소 값 / 0x1000000000002A (MFT Entry 42번)
0x08	Length of This Index Entry	2 bytes	Index Entry의 총 크기 / 0x90
0x0A	Offset to End of Name	2 bytes	이름의 끝 위치 / 0x7A
0x0C	Index Entry Flags	4 bytes	해당 Index Entry의 Sub Node 존재 및 마지막 Node인지 알려줌 / 0x00 (0x00: Sub Node 없음 / 0x01: Sub Node 존재 / 0x02: Node 마지막)
0x18	\$FILE_NAME Attribute	-	\$FILE_NAME 속성 Header + Contents

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

- 'ntfs_minhyuk_find_file_1.txt' 파일 MFT Entry 분석 (MFT Entry 42번)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A8F0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C000A900	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C000A910	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	5F	00	n.d._.f.i.l.e._.
00C000A920	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....
00C000A930	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@... (.....
00C000A940	10	00	00	00	18	00	00	00	9B	2F	1D	FC	78	14	F1	11	>/..üx.ñ.
00C000A950	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	.:~`erø“€...H...
00C000A960	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000A970	06	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00C000A980	00	70	00	00	00	00	00	00	00	66	00	00	00	00	00	00	.p.....f.....
00C000A990	00	66	00	00	00	00	00	00	21	07	78	18	00	00	00	00	.f.....!..x.....

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 bytes	Run List Offset / 0x40
0x40	Run List	4 bytes	Run List / Run length: 0x01, Run offset: 0x1878

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (1)

■ 'ntfs_minhyuk_find_file_1.txt' \$DATA Run List 이동

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001878000	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
0001878010	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN
0001878020	48	59	55	4B	5F	46	49	4E	44	5F	46	49	4C	45	5F	31	HYUK_FIND_FILE_1
0001878030	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
0001878040	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN
0001878050	48	59	55	4B	5F	46	49	4E	44	5F	46	49	4C	45	5F	31	HYUK_FIND_FILE_1
0001878060	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
0001878070	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN
0001878080	48	59	55	4B	5F	46	49	4E	44	5F	46	49	4C	45	5F	31	HYUK_FIND_FILE_1
0001878090	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
00018780A0	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN

■ Run List를 통해 계산된 Offset 이동 → txt 파일 실제 데이터 확인 가능

- $0x1878 * 0x1000 = 0x1878000$

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (2)

■ 실습 환경

- OS: Microsoft Windows 11 Home
- 논리 디스크: N: (97.6 GB)
- 생성 폴더 / 파일: NTFS_Find_file_1 / ntfs_minhyuk_find_file_1.txt
- 파일 데이터 크기 증가: 25.5 KB → 2.88 MB
- 파일 데이터 크기 감소: 2.88 MB → 48 bytes

■ 실습 결과

- 데이터 증가 / 감소 여부에 따라 기존 Non-Resident가 Resident로 전환되지는 않음
- 그러나 클러스터 할당 확장 / 축소는 이루어짐
- 실제 데이터는 파일을 열었을 때와 동일하게 기록되어 있으며, 축소의 경우 0이 많은 공간을 차지

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (2)

- 파일 데이터 크기 증가 전 / 후
 - 파일 데이터 크기 증가: 25.5 KB → 2.88 MB

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A8F0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C000A900	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C000A910	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	5F	00	n.d._.f.i.l.e._.
00C000A920	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....
00C000A930	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@...(...
00C000A940	10	00	00	00	18	00	00	00	9B	2F	1D	FC	78	14	F1	11	>/..üx.ñ.
00C000A950	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	..~"erø"€...H...
00C000A960	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000A970	06	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00C000A980	00	70	00	00	00	00	00	00	66	00	00	00	00	00	00	00	.p.....f.....
00C000A990	00	66	00	00	00	00	00	00	21	07	78	18	00	00	00	00	.f.....!.x.....



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A8F0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C000A900	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C000A910	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	5F	00	n.d._.f.i.l.e._.
00C000A920	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....
00C000A930	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@...(...
00C000A940	10	00	00	00	18	00	00	00	9B	2F	1D	FC	78	14	F1	11	>/..üx.ñ.
00C000A950	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	..~"erø"€...H...
00C000A960	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000A970	E3	02	00	00	00	00	00	00	40	00	00	00	00	00	00	00	ä.....@.....
00C000A980	00	40	2E	00	00	00	00	00	C0	3E	2E	00	00	00	00	00	.@.....Ä>.....
00C000A990	C0	3E	2E	00	00	00	00	00	22	E4	02	78	18	00	00	00	Ä>....."ä.x.....

Resident / Non - Resident 전환 여부	Non-Resident Flag		새 클러스터 할당 여부
	Run length	Run offset	
기존과 동일 하게 Non - Resident 유지	0x07 → 0x02E4	0x1878 (기존과 동일)	추가 클러스터 할당

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (2)

■ 파일 데이터 크기 증가 후

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001878000	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
0001878010	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN
0001878020	48	59	55	4B	5F	46	49	4E	44	5F	46	49	4C	45	5F	31	HYUK_FIND_FILE_1
-- 중략 --																	
0001EB8FD0	4E	54	46	53	5F	4D	49	4E	48	59	55	4B	5F	46	49	4E	NTFS_MINHYUK_FIN
0001EB8FE0	44	5F	46	49	4C	45	5F	31	4E	54	46	53	5F	4D	49	4E	D_FILE_1NTFS_MIN
0001EB8FF0	48	59	55	4B	5F	46	49	4E	44	5F	46	49	4C	45	5F	31	HYUK_FIND_FILE_1

- 데이터 영역 확인을 통해 데이터 증가 및 감소 전후 실제 저장 상태 비교
- 실제 파일을 열었을 때와 동일하게 데이터 저장
- 데이터 크기 증가 전보다 많은 데이터 공간 차지하는 것 확인 가능

메타데이터 및 구조체 분석 - [실습] 파일 찾기 (2)

- 파일 데이터 크기 감소 전 / 후
 - 파일 데이터 크기 감소: 2.88 MB → 48 bytes

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A8F0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C000A900	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C000A910	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	5F	00	n.d._.f.i.l.e._.
00C000A920	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....
00C000A930	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@...(...
00C000A940	10	00	00	00	18	00	00	00	9B	2F	1D	FC	78	14	F1	11	>/..üx.ñ.
00C000A950	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	..:~erø"€...H...
00C000A960	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000A970	E3	02	00	00	00	00	00	00	40	00	00	00	00	00	00	00	ä.....@.....
00C000A980	00	40	2E	00	00	00	00	00	C0	3E	2E	00	00	00	00	00	.@.....Ä>.....
00C000A990	C0	3E	2E	00	00	00	00	00	22	E4	02	78	18	00	00	00	À>....."ä.x....



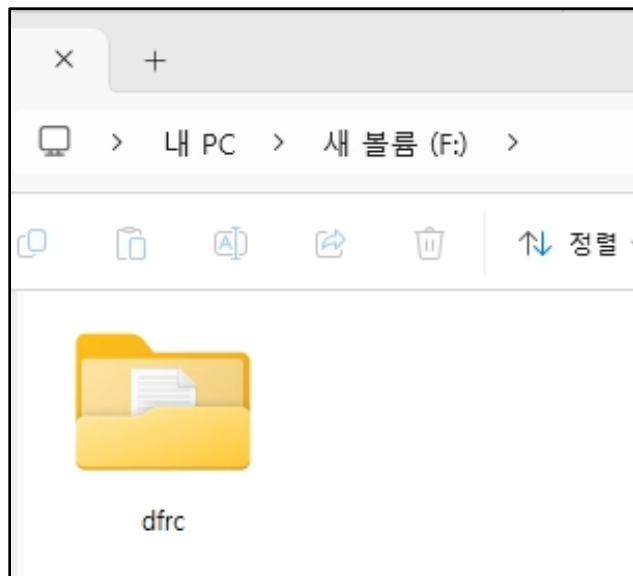
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A8F0	1C	00	6E	00	74	00	66	00	73	00	5F	00	6D	00	69	00	..n.t.f.s._.m.i.
00C000A900	6E	00	68	00	79	00	75	00	6B	00	5F	00	66	00	69	00	n.h.y.u.k._.f.i.
00C000A910	6E	00	64	00	5F	00	66	00	69	00	6C	00	65	00	5F	00	n.d._.f.i.l.e._.
00C000A920	31	00	2E	00	74	00	78	00	74	00	00	00	00	00	00	00	1...t.x.t.....
00C000A930	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@...(...
00C000A940	10	00	00	00	18	00	00	00	9B	2F	1D	FC	78	14	F1	11	>/..üx.ñ.
00C000A950	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	..:~erø"€...H...
00C000A960	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000A970	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00C000A980	00	10	00	00	00	00	00	00	30	00	00	00	00	00	00	00	0.....
00C000A990	30	00	00	00	00	00	00	00	21	01	78	18	00	00	00	00	0.....!..x....

Real Size 값	Non-Resident Flag 유지 여부	새 클러스터 할당 여부
0x2E3EC0 → 0x30	유지 ○ (0x01)	추가 할당 클러스터 축소

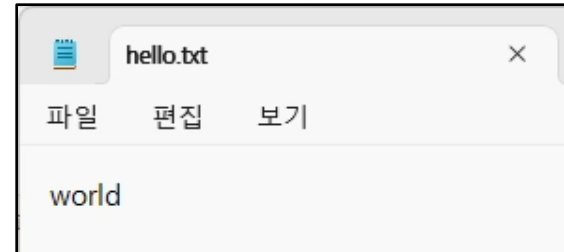
메타데이터 및 구조체 분석 - [실습] 파일 삭제 (1)

■ 실습 환경

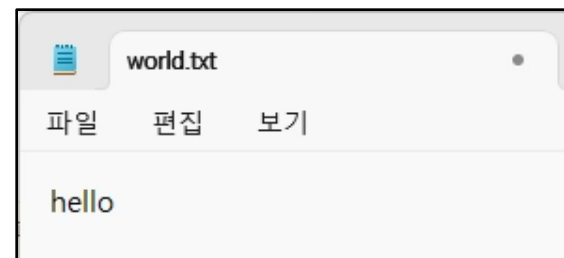
- 1 GB의 가상 하드디스크 생성 → 디스크 초기화 후 NTFS 파일 시스템 할당
- F:\dfrc 경로에 hello.txt, world.txt 생성



[폴더 생성]



[hello.txt 내용]



[world.txt 내용]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (1)

■ 파일 완전 삭제 전 'dfrc' MFT Entry 확인

- 'dfrc' MFT Entry의 \$INDEX_ALLOCATION → hello.txt, world.txt의 Index Entry 확인 가능

- hello.txt의 File Reference Address for Filename: MFT Entry 42번에 위치 (0x2A)

- world.txt의 File Reference Address for Filename: MFT Entry 43번에 위치 (0x2B)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545E960	F0	00	00	00	00	00	00	00	2A	00	00	00	00	00	01	00	ø.....*
1545E970	68	00	54	00	00	00	00	00	26	00	00	00	00	00	01	00	h.T.....&.....
--- 중략 ---																	
1545E9B0	20	00	00	00	00	00	00	00	09	00	68	00	65	00	6C	00	h.e.l.
1545E9C0	6C	00	6F	00	2E	00	74	00	78	00	74	00	74	00	78	00	l.o...t.x.t.t.x.
1545E9D0	2B	00	00	00	00	00	01	00	68	00	54	00	00	00	00	00	+.....h.T.....
--- 중략 ---																	
1545EA20	09	00	77	00	6F	00	72	00	6C	00	64	00	2E	00	74	00	..w.o.r.l.d...t.
1545EA30	78	00	74	00	74	00	78	00	00	00	00	00	00	00	00	00	x.t.t.x.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (1)

■ 파일 완전 삭제 후 'dfrc' MFT Entry 확인

- hello.txt Index Entry 삭제 → world.txt Index Entry가 삭제된 위치로 이동

- File Reference Address for Filename 은 기존 0x2B 동일

- 기존 world.txt Index Entry 위치 → File Reference Address for Filename 값 해제 확인

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545E960	88	00	00	00	00	00	00	00	2B	00	00	00	00	00	01	00	^.....+.....
1545E970	68	00	54	00	00	00	00	00	26	00	00	00	00	00	01	00	h.T.....&.....
--- 중략 ---																	
1545E9B0	20	00	00	00	00	00	00	00	09	00	77	00	6F	00	72	00	w.o.r.
1545E9C0	6C	00	64	00	2E	00	74	00	78	00	74	00	74	00	78	00	l.d...t.x.t.t.x.
1545E9D0	00	00	00	00	00	00	00	00	10	00	00	00	02	00	00	00	
--- 중략 ---																	
1545EA20	09	00	77	00	6F	00	72	00	6C	00	64	00	2E	00	74	00	..w.o.r.l.d...t.
1545EA30	78	00	74	00	74	00	78	00	00	00	00	00	00	00	00	00	x.t.t.x.....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (1)

■ 파일 완전 삭제 전 / 후 'hello.txt' MFT Entry 비교

■ 'hello.txt'의 MFT Entry Header 변경 확인

- LSN, Sequence Number, Flags, Fixup Array 변경

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545EC00	46	49	4C	45	30	00	03	00	38	70	20	00	00	00	00	00	FILE0...8p
1545EC10	01	00	01	00	38	00	01	00	58	01	00	00	00	04	00	00	8...X.....
1545EC20	00	00	00	00	00	00	00	00	05	00	00	00	27	00	00	00	'...
1545EC30	05	00	00	00	00	00	00	00	10	00	00	00	60	00	00	00	`...



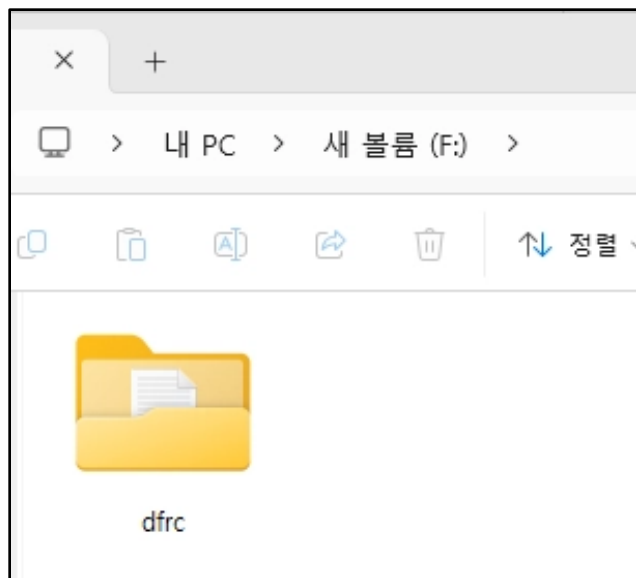
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545EC00	46	49	4C	45	30	00	03	00	60	7F	20	00	00	00	00	00	FILE0...`
1545EC10	02	00	01	00	38	00	00	00	58	01	00	00	00	04	00	00	8...X.....
1545EC20	00	00	00	00	00	00	00	00	05	00	00	00	27	00	00	00	'...
1545EC30	06	00	00	00	00	00	00	00	10	00	00	00	60	00	00	00	`...

- 이외 \$STANDARD_INFORMATION, \$FILE_NAME, \$DATA 값 변화 X

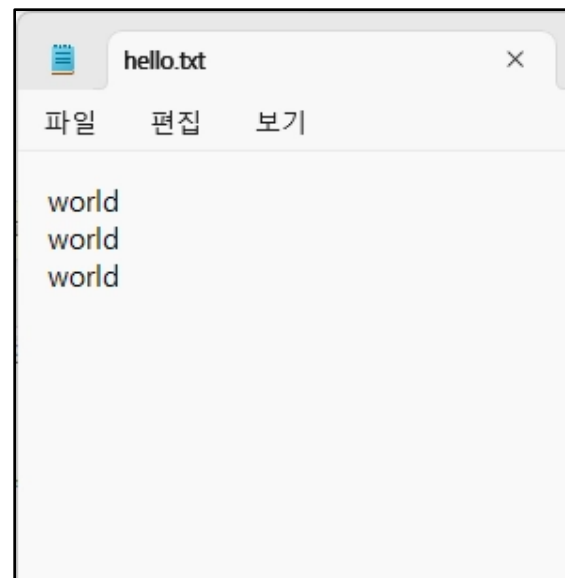
메타데이터 및 구조체 분석 - [실습] 파일 삭제 (2)

■ 실습 환경

- 1 GB의 가상 하드디스크 생성 → 디스크 초기화 후 NTFS 파일 시스템 할당
- F:\dfrc 경로에 hello.txt 생성



[폴더 생성]



[hello.txt 내용]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (2)

- 파일 완전 삭제 전 \$MFT의 \$BITMAP Attribute 확인
 - Attribute Type ID: 0xB0 (\$BITMAP)
 - Non - Resident: 0x1
 - Run lists: 1st (0x15454000) → 해당 위치로 이동

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
15455140	31	40	55	54	01	00	00	00	B0	00	00	00	50	00	00	00	1@UT....°...P...
15455150	01	00	40	00	00	00	05	00	00	00	00	00	00	00	00	00	..@.....
15455160	01	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
15455170	00	20	00	00	00	00	00	00	08	10	00	00	00	00	00	00	
15455180	08	10	00	00	00	00	00	00	31	01	54	54	01	31	01	D1	1.TT.1.Ñ
15455190	AB	FE	00	00	00	00	00	00	FF	FF	FF	FF	00	00	00	00	«p.....ÿÿÿÿ....

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (2)

■ 파일 완전 삭제 전 / 후 \$MFT의 Bit 변화 확인

■ MFT Entry 할당 확인

- 할당이 되어있으면 bit가 1, 비어 있는 경우 0으로 설정
- 16 ~ 23번 Entry를 제외하고 0 ~ 42번까지 할당 상태 → 완전 삭제 후 42번 엔트리 할당 해제

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
15454000	FF	FF	00	FF	FF	07	00	00	00	00	00	00	00	00	00	00	ÿÿ.ÿÿ.....
15454010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
15454000	FF	FF	00	FF	FF	03	00	00	00	00	00	00	00	00	00	00	ÿÿ.ÿÿ.....
15454010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

- 0x07 (00000111, 40 ~ 42번까지 할당) → 0x03 (0000011, 40 ~ 41번까지 할당)

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (2)

- 파일 완전 삭제 전 / 후 \$Bitmap 파일 확인
 - 파일 완전 삭제 전 / 후 디스크 이미징 → \$Bitmap 파일 추출 후 비교
 - 클러스터 2000 ~ 2003 제외, 클러스터 0 ~ 2007 할당
 - 파일 완전 삭제 전 / 후 변화 X (Resident 이기에 별도의 클러스터 할당 X)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyyyy
00000010	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyyyy
00000020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyyyy
--- 중략 ---																	
000000D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyyyy
000000E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyyyy
000000F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	0F	00	00	00	00	yyyyyyyyyy.....

- 결론: 파일 완전 삭제 시, MFT Entry 할당 상태만 변화 → 데이터 덧씌워지지 않는 경우 복원 가능성

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (2)

■ 파일 완전 삭제 후 \$MFT의 \$BITMAP Attribute 확인

- Attribute Type ID: 0xB0 (\$BITMAP)
- Non – Resident: 0x1
- Run lists: 1st (0x15454000)

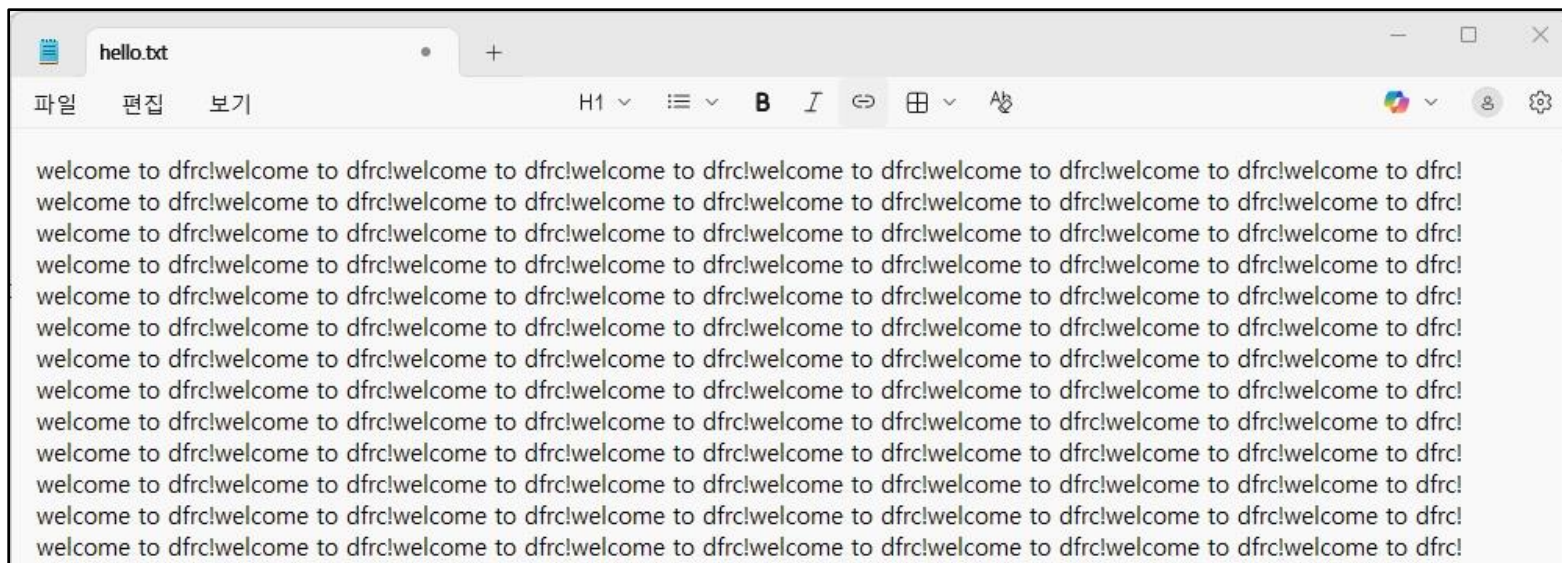
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
15455140	31	40	55	54	01	00	00	00	B0	00	00	00	50	00	00	00	1@UT....°...P...
15455150	01	00	40	00	00	00	05	00	00	00	00	00	00	00	00	00	..@.....
15455160	01	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
15455170	00	20	00	00	00	00	00	00	08	10	00	00	00	00	00	00	
15455180	08	10	00	00	00	00	00	00	31	01	54	54	01	31	01	D1	1.TT.1.Ñ
15455190	AB	FE	00	00	00	00	00	00	FF	FF	FF	FF	00	00	00	00	«p.....ÿÿÿÿ....

- 파일 완전 삭제 전 / 후 변화 X

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

■ 실습 환경

- 1 GB의 가상 하드디스크 생성 → 디스크 초기화 후 NTFS 파일 시스템 할당
- F:\dfrc 경로에 hello.txt 생성
 - 문자열 다수 채워 Non – resident 속성으로 변경



[hello.txt 내용]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

- 'hello.txt' 완전 삭제 후 MFT Entry 확인
 - \$LogFile의 트랜잭션 위치 값 (LSN): 0x0000000000020812A
 - Filename: hello.txt
 - Non-resident: 0x1 / 설정

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545F800	46	49	4C	45	30	00	03	00	2A	81	20	00	00	00	00	00	FILE0...*.
1545F810	02	00	01	00	38	00	00	00	80	01	00	00	00	04	00	00	8...€.....
	--- 중략 ---																
1545F8E0	00	00	00	00	00	00	00	00	20	00	00	00	00	00	00	00	
1545F8F0	09	00	68	00	65	00	6C	00	6C	00	6F	00	2E	00	74	00	..h.e.l.l.o...t.
1545F900	78	00	74	00	00	00	00	00	40	00	00	00	28	00	00	00	x.t.....@...(...
1545F910	00	00	00	00	00	00	04	00	10	00	00	00	18	00	00	00	
1545F920	23	4F	F9	89	7C	0C	F1	11	8A	BF	04	EA	56	BA	05	E1	#Où% .ñ.Š¿.êV°.á
1545F930	80	00	00	00	48	00	00	00	01	00	00	00	00	00	05	00	€...H.....

- \$LogFile에서 해당 LSN 검색

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

■ \$LogFile에서 LSN 검색

- 파일 완전 삭제 후 디스크 이미징 → \$LogFile 파일 추출
- HxD에서 LSN 검색
 - Redo OP: 0x03 (DeallocateFileRecordSegmentp) / Undo OP: 0x02 (InitializeFileRecordSegment)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
14FC4950	2A	81	20	00	00	00	00	00	14	81	20	00	00	00	00	00	*.
14FC4960	14	81	20	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
14FC4970	01	00	00	00	18	00	00	00	02	00	00	00	00	00	00	00	
14FC4980	03	00	02	00	28	00	00	00	28	00	18	00	18	00	01	00	 (... (...
14FC4990	00	00	00	00	04	00	02	00	0A	00	00	00	00	00	00	00	
14FC49A0	5F	54	01	00	00	00	00	00	46	49	4C	45	30	00	03	00	_T.....FILE0...
14FC49B0	9A	79	20	00	00	00	00	00	01	00	01	00	38	00	01	00	šy8...
14FC49C0	38	81	20	00	00	00	00	00	2A	81	20	00	00	00	00	00	8.*.
14FC49D0	2A	81	20	00	00	00	00	00	30	00	00	00	00	00	00	00	*.0.....

- LCN: 0x1545F / No. In Cluster: 0x4 / VCN: 0xA

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

■ 삭제 파일의 \$MFT Entry 주소 계산

- 방식 1: MFT Entry 시작 주소 + (VCN * 8 + No. In Cluster) * 200
= 0x15455000 + (0xA * 8 + 0x4) * 0x200 = 0x1545F800

- 방식 2: (LCN * 8 + No. In Cluster) * 0x200 = (0x1545F * 8 + 0x4) * 200 = 0x1545F800

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	This LSN								Previous LSN							
10	Undo Next LSN								Data Length				Sequence Num		Client Index	
20	Record Type				Transaction ID				Flag		Reserved					
30	Redo OP		Undo OP		Redo Offset		Redo Length		Undo Offset		Undo Length		Target Attribute		LSN to Follow	
40	Offset to Attribute		Offset in Attribute		No. In Cluster		Page Size		VCN – Cluster Number in \$MFT							
50	LCN – Cluster Number in Volume															

[\$LogFile의 Log Record 구조]

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

■ 삭제 파일의 \$MFT Entry 이동 후 남아 있는 파일 정보 확인

■ MFT Entry 주소

- MFT Entry 시작 섹터 + (VCN * 8) + No. In Cluster
- \$DATA Attribute의 1st Run List: $0x7D4 * 0x1000 = 0x7D4000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
1545F800	46	49	4C	45	30	00	03	00	2A	81	20	00	00	00	00	00	FILE0...*.
1545F810	02	00	01	00	38	00	00	00	80	01	00	00	00	04	00	00	8...€.....
	--- 중략 ---																
1545F8F0	09	00	68	00	65	00	6C	00	6C	00	6F	00	2E	00	74	00	..h.e.l.l.o...t.
1545F900	78	00	74	00	00	00	00	00	40	00	00	00	28	00	00	00	x.t.....@...(...
1545F910	00	00	00	00	00	00	04	00	10	00	00	00	18	00	00	00	
1545F920	23	4F	F9	89	7C	0C	F1	11	8A	BF	04	EA	56	BA	05	E1	#Où% .ñ.Šž.êVº.á
1545F930	80	00	00	00	48	00	00	00	01	00	00	00	00	00	05	00	€...H.....
1545F940	00	00	00	00	00	00	00	00	01	00	00	00	00	00	00	00	
1545F950	40	00	00	00	00	00	00	00	00	20	00	00	00	00	00	00	@.....
1545F960	10	17	00	00	00	00	00	00	10	17	00	00	00	00	00	00	
1545F970	21	02	D4	07	00	00	00	00	FF	FF	FF	FF	82	79	47	11	!.Ô.....ÿÿÿÿ,yG.

메타데이터 및 구조체 분석 - [실습] 파일 삭제 (3)

- 삭제 파일의 \$MFT Entry 이동 후 남아 있는 파일 정보 확인
 - 계산한 Run List Offset 이동
 - 데이터 잔존 확인

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
007D4000	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4010	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4020	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4030	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4040	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4050	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4060	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!
007D4070	77	65	6C	63	6F	6D	65	20	74	6F	20	64	66	72	63	21	welcome to dfrc!

[실습] 파일 복구 (1)

■ 실습 환경

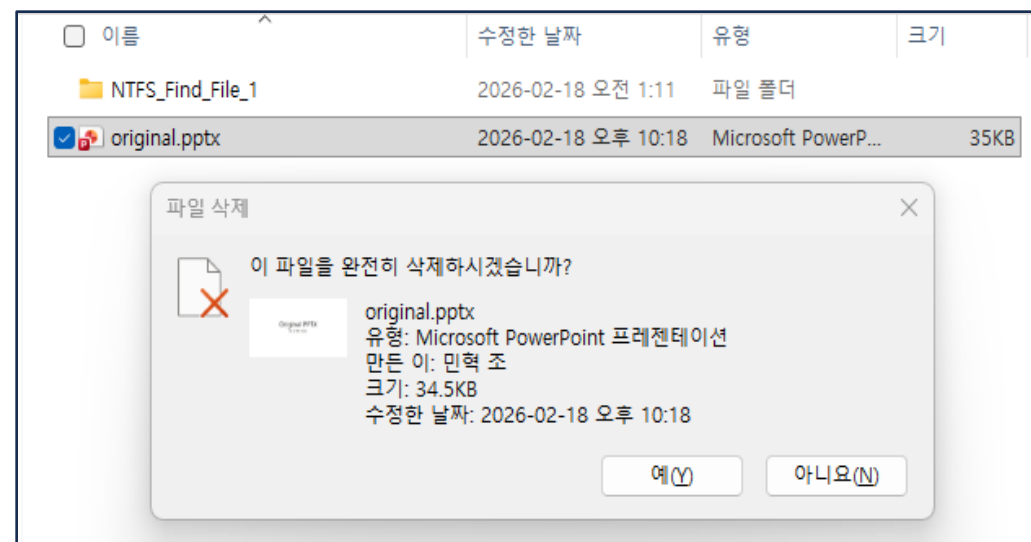
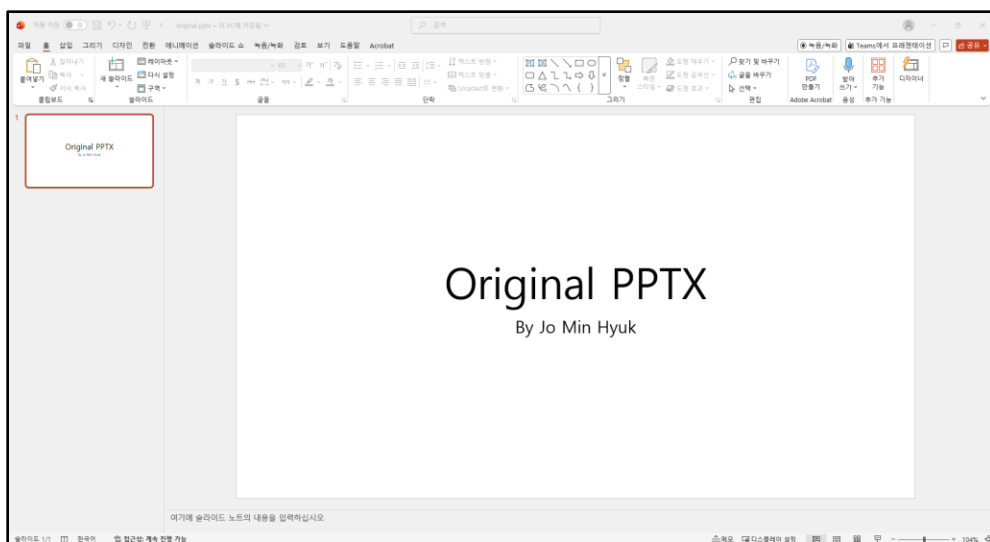
- OS: Microsoft Windows 11 Home
- 논리 디스크: N: (97.6 GB)
- 생성 파일: ./original.pptx

■ 실습 결과

- 완전 삭제된 파일 \$INDEX_ALLOCATION에서 MFT Entry 번호 0으로 변화
- MFT Entry 번호 확인 불가
- 수동 분석으로 완전 삭제된 파일의 MFT Entry 번호 확인 가능
- \$DATA 영역 데이터 보존 → 시그니처 기반 파일 카빙을 통해 복구

[실습] 파일 복구 (1)

- 파일 생성 및 삭제
 - original.pptx 생성
 - Shift + Delete로 완전 삭제 진행



[original.pptx 생성 및 삭제]

[실습] 파일 복구 (1)

- \$MFT 시작 위치 계산
 - \$MFT 시작 위치 = 0x0C0000 * 0x1000 = 0xC0000000

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0000000000	EB	52	90	4E	54	46	53	20	20	20	20	00	02	08	00	00	ëR.NTFS
0000000010	00	00	00	00	00	F8	00	00	3F	00	FF	00	00	08	00	00	ø...?.ÿ.....
0000000020	00	00	00	00	80	00	80	00	FF	E7	34	0C	00	00	00	00	€.€.ÿç4.....
0000000030	00	00	0C	00	00	00	00	00	02	00	00	00	00	00	00	00	

Offset	Field Name	Size	Description / Value
0x0B	Byte Per Sector	2 bytes	섹터 당 바이트 / 0x200 = 512 bytes
0x0D	Sector Per Cluster	1 bytes	클러스터 당 섹터 / 0x08
0x30	Start Cluster for \$MFT	8 bytes	\$MFT 시작 클러스터 / 0x0C0000 *\$MFT 주소 = 0x0C0000 * 0x1000 = 0xC0000000

[실습] 파일 복구 (1)

■ Root directory MFT Entry - \$INDEX_ROOT 분석

■ Root Directory의 MFT Entry 번호: 5번 → $0xC0000000 + 0x5 * 0x400 = 0xC0001400$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C00015E0	90	00	00	00	58	00	00	00	00	04	18	00	00	00	06	00	X.....
00C00015F0	38	00	00	00	20	00	00	00	24	00	49	00	33	00	0C	00	8... ..\$.I.3...
00C0001600	30	00	00	00	01	00	00	00	00	10	00	00	01	00	00	00	0.....
00C0001610	10	00	00	00	28	00	00	00	28	00	00	00	01	00	00	00	(...(.....

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Resident Header (8 bytes) / 0x90 (\$INDEX_ROOT), 0x00 (Resident 방식)
0x18	Attribute Name	8 bytes	속성명 / \$I30
0x20	Type of Attribute in Index	4 bytes	인덱스의 속성 유형 / 0x30: \$FILE_NAME
0x30	Offset to First Index Entry	4 bytes	첫 번째 인덱스 엔트리 위치 / $0xC0001638 + 0x10 = 0xC0001648$

[실습] 파일 복구 (1)

■ Root Directory MFT Entry - \$INDEX_ALLOCATION 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C0001630	00	00	00	00	00	00	00	00	A0	00	00	00	50	00	00	00	P...
00C0001640	01	04	40	00	00	00	08	00	00	00	00	00	00	00	00	00	..@.....
00C0001650	00	00	00	00	00	00	00	00	48	00	00	00	00	00	00	00	H.....
00C0001660	00	10	00	00	00	00	00	00	00	10	00	00	00	00	00	00	
00C0001670	00	10	00	00	00	00	00	00	24	00	49	00	33	00	30	00	\$.I.3.0.
00C0001680	11	01	24	00	00	00	00	00	B0	00	00	00	28	00	00	00	..\$......°....(...

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0xA0 (\$INDEX_ALLOCATION), 0x01 (Non-Resident 방식)
0x18	Attribute Name	8 bytes	속성명 / \$I30
0x20	Run List	5 bytes	Run List / Run length: 0x01, Run offset: 0x24 *데이터가 존재하는 클러스터 위치 0x24 * 0x1000 = 0x24000

[실습] 파일 복구 (1)

- 완전 삭제 후 'original pptx' Index Entry 확인
 - Index Entry의 File Reference Address 값이 0으로 변경 → MFT Entry 추적 불가

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C0009A10	00	00	00	00	00	00	00	00	10	00	00	00	02	00	00	00	
00C0009A20	FF	FF	FF	FF	82	79	47	11	43	11	71	5D	39	AA	DC	01	ÿÿÿÿ,yG.C.q]9aÜ.
00C0009A30	1C	04	52	89	39	AA	DC	01	E2	F0	5B	89	39	AA	DC	01	..R%9aÜ.âð[%9aÜ.
00C0009A40	1C	04	52	89	39	AA	DC	01	00	90	00	00	00	00	00	00	..R%9aÜ.....
00C0009A50	3B	8A	00	00	00	00	00	00	20	00	00	00	00	00	00	00	;Š.....
00C0009A60	0D	00	6F	00	72	00	69	00	67	00	69	00	6E	00	61	00	..o.r.i.g.i.n.a.
00C0009A70	6C	00	2E	00	70	00	70	00	74	00	78	00	74	00	78	00	l...p.p.t.x.t.x.

Offset	Field Name	Size	Description / Value
0x00	File Reference Address	8 bytes	Index Entry가 참조하는 파일 또는 디렉터리의 MFT 주소 값 / 0x0000000000000000
0x08	Length of This Index Entry	2 bytes	Index Entry의 총 크기 / 0x10, 비정상적인 값
0x0A	Offset to End of Name	2 bytes	이름의 끝 위치 / 0x00, 비정상적인 값
0x0C	Index Entry Flags	4 bytes	해당 Index Entry의 Sub Node 존재 및 마지막 Node인지 알려줌 / 0x02 (0x00: Sub Node 없음 / 0x01: Sub Node 존재 / 0x02: Node 마지막)
0x18	\$FILE_NAME Attribute	-	\$FILE_NAME 속성 Header + Contents

[실습] 파일 복구 (1)

■ 삭제 파일 데이터 확인

- 0x400씩 MFT Entry 추적 결과 43번 (0x2B) MFT Entry에서 삭제된 파일 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000B400	46	49	4C	45	30	00	03	00	77	D0	00	02	00	00	00	00	FILE0...wD.....
00C000B410	05	00	01	00	38	00	00	00	88	01	00	00	00	04	00	00	8...^.....
--- 중략 ---																	
00C000B490	00	00	00	00	00	00	00	00	30	00	00	00	78	00	00	00	0...x...
00C000B4A0	00	00	00	00	00	00	04	00	5C	00	00	00	18	00	01	00	\.....
00C000B4B0	26	00	00	00	00	00	01	00	01	0E	89	28	45	AA	DC	01	&.....%(E ^a Ü.
00C000B4C0	14	C2	95	3F	45	AA	DC	01	FE	2C	9E	3F	45	AA	DC	01	.Â•?E ^a Ü.p,ž?E ^a Ü.
00C000B4D0	14	C2	95	3F	45	AA	DC	01	00	90	00	00	00	00	00	00	.Â•?E ^a Ü.....
00C000B4E0	3B	8A	00	00	00	00	00	00	20	00	00	00	00	00	00	00	;Š.....
00C000B4F0	0D	00	6F	00	72	00	69	00	67	00	69	00	6E	00	61	00	..o.r.i.g.i.n.a.
00C000B500	6C	00	2E	00	70	00	70	00	74	00	78	00	00	00	00	00	l...p.p.t.x.....

[실습] 파일 복구 (1)

- ‘original.pptx’ MFT Entry 분석
 - \$DATA의 Run List: 0x1B5C * 0x1000 = 0x1B5C000

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000B530	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	.:~`erø“€...H...
00C000B540	01	00	00	00	00	00	03	00	00	00	00	00	00	00	00	00	
00C000B550	08	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00C000B560	00	90	00	00	00	00	00	00	3B	8A	00	00	00	00	00	00	;Š.....
00C000B570	3B	8A	00	00	00	00	00	00	21	09	5C	1B	00	00	00	00	;Š.....!.\\.....

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 Bytes) + Non-Resident Header (48 Bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 bytes	Run List Offset / 0x40
0x40	Run List	5 bytes	Run List / Run length: 0x01, Run offset: 0x1B 5C

[실습] 파일 복구 (1)

■ Run List 이동

- 0x1B5C000 이동: original.pptx의 데이터 발견 가능 → pptx의 경우 zip 구조로 표현

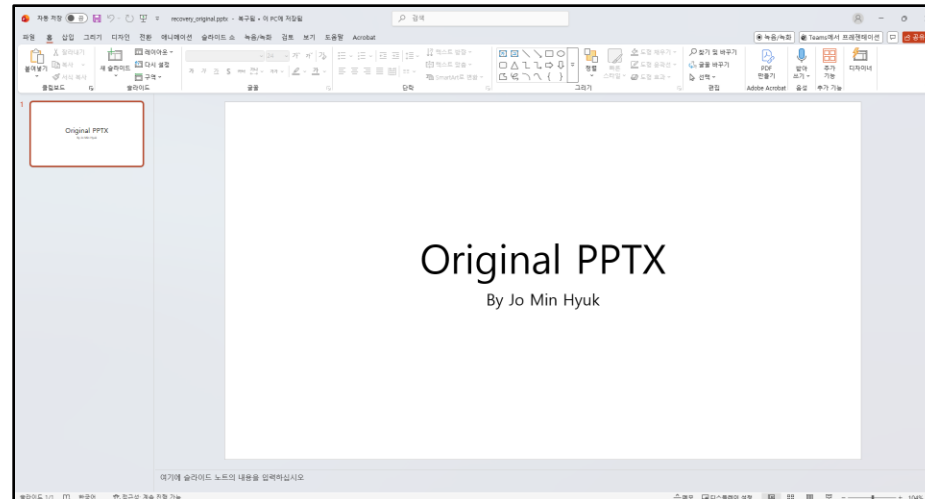
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001B5C000	50	4B	03	04	14	00	06	00	08	00	00	00	21	00	DF	CC	PK.....!.ßİ
0001B5C010	18	F5	AD	01	00	00	46	0C	00	00	13	00	08	02	5B	43	.õ....F.....[C
0001B5C020	6F	6E	74	65	6E	74	5F	54	79	70	65	73	5D	2E	78	6D	ontent_Types].xm
0001B5C030	6C	20	A2	04	02	28	A0	00	02	00	00	00	00	00	00	00	1 ¢..(.....
--- 중략 ---																	
0001B64A10	00	52	7B	00	00	64	6F	63	50	72	6F	70	73	2F	61	70	.R{..docProps/ap
0001B64A20	70	2E	78	6D	6C	50	4B	05	06	00	00	00	00	25	00	25	p.xmlPK.....%.
0001B64A30	00	4D	0B	00	00	D8	7E	00	00	00	00	00	00	00	00	00	.M...Ø~.....

[실습] 파일 복구 (1)

■ 삭제 파일 복구

- 블록 크기만큼 복사 후 저장 → original.pptx 복구 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	50	4B	03	04	14	00	06	00	08	00	00	00	21	00	DF	CC	PK.....!.βİ
00000010	18	F5	AD	01	00	00	46	0C	00	00	13	00	08	02	5B	43	.õ....F.....[C
00000020	6F	6E	74	65	6E	74	5F	54	79	70	65	73	5D	2E	78	6D	ontent_Types].xm
00000030	6C	20	A2	04	02	28	A0	00	02	00	00	00	00	00	00	00	l ¢..(.....



[original.pptx 복구]

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ 실습 환경

- OS: Microsoft Windows 11 Home
- 논리 디스크: N: (97.6 GB)
- 생성 파일: ./original_png.png, ./original_hwp.hwp

■ 실습 결과

- 완전 삭제된 파일의 Index 정보 완전 소실 또는 Entry 번호, 파일 크기 소실
- MFT Entry 번호 추적 후 \$DATA 영역에서 시그니처 기반 파일 카빙
- PNG의 경우 알려진 헤더와 푸터로 카빙하여 데이터 복구
- HWP의 경우 구조 분석을 통해 크기 계산 후 카빙하여 데이터 복구

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ 파일 생성 및 삭제

- PNG, HWP (5.X버전) 파일 생성 및 크기 확인
 - PNG: 9,355 bytes / HWP: 62,976 bytes
- 파일 완전 삭제 후 Index Entry 정보 확인 → PNG, HWP 인덱싱 정보 확인 불가 / 실제 데이터 크기 확인 불가



[png_original.png]

```
DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC D
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
```

[hwp_original.hwp]

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ PNG MFT Entry 번호 추적

- MFT Entry 42번 (0x2A)에서 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000AC00	46	49	4C	45	30	00	03	00	46	B0	00	02	00	00	00	00	FILE0...F°.....
00C000AC10	05	00	01	00	38	00	00	00	90	01	00	00	00	04	00	00	8.....
-- 중략 --																	
00C000A8F0	10	00	70	00	6E	00	67	00	5F	00	6F	00	72	00	69	00	..p.n.g._.o.r.i.
00C000A900	67	00	69	00	6E	00	61	00	6C	00	2E	00	70	00	6E	00	g.i.n.a.l...p.n.
00C000A910	67	00	00	00	00	00	00	00	40	00	00	00	28	00	00	00	g.....@...(...
00C000A920	00	00	00	00	00	00	05	00	10	00	00	00	18	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

- PNG MFT Entry \$DATA 영역 분석
 - Run List: 0x1881 * 0x1000 = 0x1881000

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000A940	80	00	00	00	48	00	00	00	01	00	00	00	00	00	04	00	€...H.....
00C000A950	00	00	00	00	00	00	00	00	02	00	00	00	00	00	00	00	
00C000A960	40	00	00	00	00	00	00	00	00	30	00	00	00	00	00	00	@.....0.....
00C000A970	8B	24	00	00	00	00	00	00	8B	24	00	00	00	00	00	00	<\$.....<\$.....
00C000A980	21	03	81	18	00	00	00	00	FF	FF	FF	FF	82	79	47	11	!.....ÿÿÿÿ,yG.

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 bytes	Run List Offset / 0x40
0x40	Run List	4 bytes	Run List / Run length: 0x01, Run offset: 0x1881

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

- PNG MFT Entry 실제 데이터 값 및 복구 범위 확인

- 0x1881000 이동 → PNG 헤더 및 푸터 확인

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001881000	89	50	4E	47	0D	0A	1A	0A	00	00	00	0D	49	48	44	52	%PNG.....IHDR
0001881010	00	00	04	80	00	00	02	88	08	06	00	00	00	08	2B	81	...€...^.....+.
-- 중략 --																	
0001883480	00	00	00	49	45	4E	44	AE	42	60	82	00	00	00	00	00	...IENDB` ,.....
0001883490	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00018834A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00018834B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ PNG 파일 복구 수행

- 블록 크기만큼 복사 후 저장 → PNG 복구 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	50	4B	03	04	14	00	06	00	08	00	00	00	21	00	DF	CC	PK.....!.Bİ
00000010	18	F5	AD	01	00	00	46	0C	00	00	13	00	08	02	5B	43	.õ....F.....[C
00000020	6F	6E	74	65	6E	74	5F	54	79	70	65	73	5D	2E	78	6D	ontent_Types].xm
00000030	6C	20	A2	04	02	28	A0	00	02	00	00	00	00	00	00	00	1 ¢..(.....



[png_original.png 복구]

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ HWP MFT Entry 번호 추적

- 0x400 단위로 수동 분석을 통해 MFT Entry 43번 (0x2B)에서 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000AC00	46	49	4C	45	30	00	03	00	46	B0	00	02	00	00	00	00	FILE0...F°.....
00C000AC10	05	00	01	00	38	00	00	00	90	01	00	00	00	04	00	00	8.....
-- 중략 --																	
00C000ACF0	10	00	68	00	77	00	70	00	5F	00	6F	00	72	00	69	00	..h.w.p._.o.r.i.
00C000AD00	67	00	69	00	6E	00	61	00	6C	00	2E	00	68	00	77	00	g.i.n.a.l...h.w.
00C000AD10	70	00	00	00	00	00	00	00	40	00	00	00	28	00	00	00	p.....@...(...
00C000AD20	00	00	00	00	00	00	04	00	10	00	00	00	18	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ HWP MFT Entry \$DATA 영역 분석

■ Run List: 0x1884 * 0x1000 = 0x1884000

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000AD40	80	00	00	00	48	00	00	00	01	00	00	00	00	00	03	00	€...H.....
00C000AD50	00	00	00	00	00	00	00	00	0F	00	00	00	00	00	00	00	
00C000AD60	40	00	00	00	00	00	00	00	00	00	01	00	00	00	00	00	@.....
00C000AD70	00	F6	00	00	00	00	00	00	00	F6	00	00	00	00	00	00	.ö.....ö.....
00C000AD80	21	10	84	18	00	00	00	00	FF	FF	FF	FF	82	79	47	11	!.,,.....ÿÿÿÿ,yG.

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 bytes	Run List Offset / 0x40
0x40	Run List	4 bytes	Run List / Run length: 0x01, Run offset: 0x1884

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

- HWP MFT Entry 실제 데이터 값 및 복구 범위 확인
 - HWP의 파일 구조 분석을 통해 삭제된 HWP 파일의 총 크기 파악 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001884000	D0	CF	11	E0	A1	B1	1A	E1	00	00	00	00	00	00	00	00	Đĩ.à¡±.á.....
0001884010	00	00	00	00	00	00	00	00	3E	00	03	00	FE	FF	09	00	>...þÿ..
0001884020	06	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00	

Offset	Field Name	Size	Description / Value
0x00	Signature	8 bytes	헤더 시그니처 / 0xD0 CF 11 E0 A1 B1 1A E1
0x18	Minor version	2 bytes	- / 0x3E = 62
0x1A	Major version	2 bytes	- / 0x03 = 3
0x1E	Sector shift	2 bytes	2^(Sector shift)로 표현 / 섹터 크기: 2^9 = 512 bytes
0x2C	Number of FAT sectors	4 bytes	FAT sector 개수 / 0x01

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ HWP 파일 구조 분석을 통한

- HWP 헤더 / 푸터: D0 CF 11 E0 A1 B1 1A E1 / 없음
- HWP의 파일 구조 OLE (Object Linking and Embedding)를 따름
 - OLE: 헤더 영역 (512 bytes)와 데이터 영역 (512 bytes ~)로 구성된 복합 문서 파일 형식
 - 섹터 단위 구성, FAT 체인 구조
- 즉, 섹터의 크기 및 FAT 체인 구성을 파악하면 HWP 파일 크기 확인 가능
 - HWP 총 크기 = (총 섹터 수 + 1) * 섹터 크기 = 129 * 512 bytes = 0x10200 (66,048 bytes)
 - 총 섹터 수 = FAT 개수 * (섹터 크기 / 4) = 1 * (512 / 4) = 1 * 128 = 128개

메타데이터 및 구조체 분석 - [실습] 파일 복구 (2)

■ HWP 파일 복구 수행

- 블록 크기만큼 복사 후 저장 → HWP 복구 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	D0	CF	11	E0	A1	B1	1A	E1	00	00	00	00	00	00	00	00	Đ Ĭ.à;±.á.....
00000010	00	00	00	00	00	00	00	00	3E	00	03	00	F3	FF	09	00	>...óÿ..



```
DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC D
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DF
51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DFRC DFIR 51DF
```

[hwp_original.hwp 복구]

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

■ 실습 환경

- OS: Microsoft Windows 11 Home
- 논리 디스크: N: (97.6 GB)
- 생성 파일: ./original_png.png, ./original_hwp.hwp

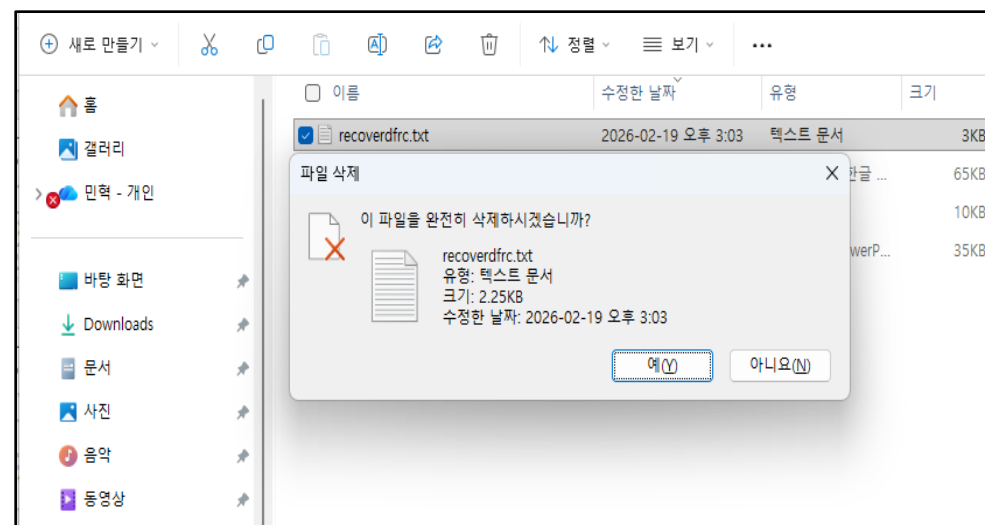
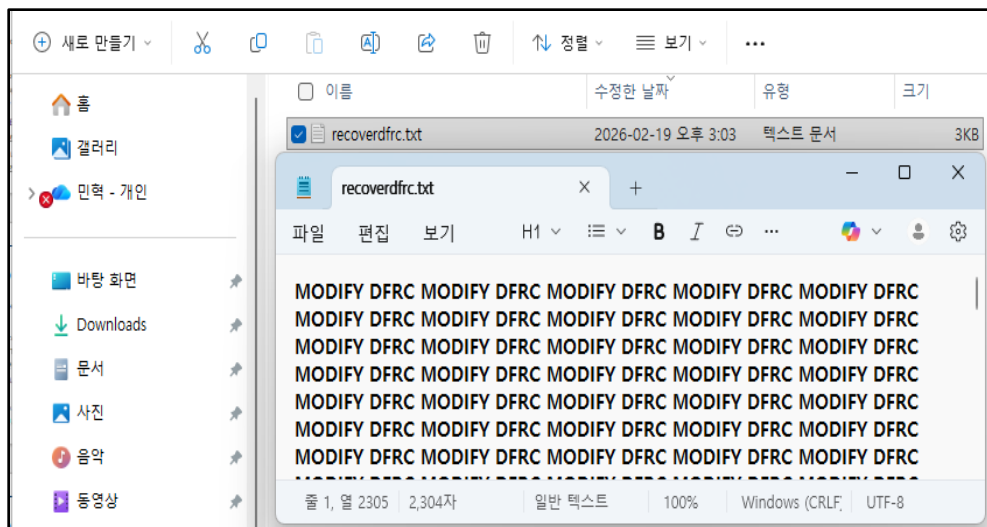
■ 실습 결과

- MFT Entry 번호에서 LSN 확인 가능
- 확인한 LSN을 통해 \$LogFile에서 검색 및 \$FILE_NAME 내용 확인
- 트랜잭션 체인 구성 후 \$DATA에 대한 Undo 적용 엔트리 파악 후 해당 엔트리 분석
- \$DATA 영역 분석 후 Run list를 통해 데이터 영역 파악하여 데이터 복구

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

■ 파일 생성, 수정, 완전 삭제 진행

- recoverdfrc.txt 파일 생성 → 초기 RECOVER DFRC 문자열 반복 저장
- recoverdfrc.txt 파일 수정 → MODIFY DFRC 문자열 반복 저장
- 완전 삭제 진행



[recoverdfrc.txt 생성 및 삭제]

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- 'recoverdfrc.txt' MFT Entry에서 LSN 확인
 - MFT Entry를 통한 \$LogFile의 LSN 확인
 - LSN: 0x0200EC39, Non- Resident 플래그: 0x01 (Non-Resident)

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000BC00	46	49	4C	45	30	00	03	00	39	EC	00	02	00	00	00	00	FILE0...9i.....
00C000BC10	02	00	01	00	38	00	00	00	88	01	00	00	00	04	00	00	8...^.....
--중략--																	
00C000BCF0	0F	00	72	00	65	00	63	00	6F	00	76	00	65	00	72	00	..r.e.c.o.v.e.r.
00C000BD00	64	00	66	00	72	00	63	00	2E	00	74	00	78	00	74	00	d.f.r.c...t.x.t.
00C000BD10	40	00	00	00	28	00	00	00	00	00	00	00	00	00	04	00	@... (.....
00C000BD20	10	00	00	00	18	00	00	00	95	1A	5A	B1	32	0D	F1	11	•.Z±2.ñ.
00C000BD30	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	.:~`erø“€...H...
00C000BD40	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 MFT Entry 분석
 - \$DATA 분석을 통한 재시작 영역 위치 파악
 - 0xBBCF1 * 0x1000 = 0xBBCF1000

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C00008F0	08	03	24	00	4C	00	6F	00	67	00	46	00	69	00	6C	00	..\$.L.o.g.F.i.l.
00C0000900	65	00	00	00	00	00	00	00	80	00	00	00	48	00	00	00	e.....€...H...
00C0000910	01	00	40	00	00	00	01	00	00	00	00	00	00	00	00	00	..@.....
00C0000920	FF	3F	00	00	00	00	00	00	40	00	00	00	00	00	00	00	ÿ?.....@.....
00C0000930	00	00	00	04	00	00	00	00	00	00	00	04	00	00	00	00	
00C0000940	00	00	00	04	00	00	00	00	32	00	40	F1	BC	0B	00	00	2.@ñ%...

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 Bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 Bytes	Run List Offset / 0x40
0x40	Run List	6 Bytes	Run List / Run length: 0x4000, Run offset: 0xBBCF1

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 로깅 영역에서의 \$FILE_NAME 분석을 통한 레코드 파악
 - 로깅 영역 중 페이지 헤더 근처에서 타겟 파일명 확인 가능 → 찾고자하는 로깅 페이지 파악 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00BBD67000	52	43	52	44	28	00	09	00	E9	ED	00	02	00	00	00	00	RCRD(...éí.....
00BBD67010	03	00	00	00	02	00	02	00	E8	0F	00	00	00	00	00	00	è.....
00BBD67020	E9	ED	00	02	00	00	00	00	6D	60	00	00	DC	01	00	00	éí.....m`..Ü...
-- 중략 --																	
00BBD670A0	24	00	00	00	00	00	00	00	2F	00	00	00	00	00	01	00	\$....../.....
00BBD670B0	70	00	60	00	00	00	00	00	05	00	00	00	00	00	05	00	p.`.....
00BBD670C0	74	7A	71	17	65	A1	DC	01	9B	3C	D0	D5	65	A1	DC	01	tzq.ejÜ.><ÐÕejÜ.
00BBD670D0	9B	3C	D0	D5	65	A1	DC	01	9B	3C	D0	D5	65	A1	DC	01	><ÐÕejÜ.><ÐÕejÜ.
00BBD670E0	00	10	00	00	00	00	00	00	00	09	00	00	00	00	00	00	
00BBD670F0	20	00	00	00	00	00	00	00	0F	00	72	00	65	00	63	00	r.e.c.
00BBD67100	6F	00	76	00	65	00	72	00	64	00	66	00	72	00	63	00	o.v.e.r.d.f.r.c.
00BBD67110	2E	00	74	00	78	00	74	00	23	EC	00	02	00	00	00	00	..t.x.t.#i.....

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 LSN을 통한 트랜잭션 체인 구성-1

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00BBD671C0	00	00	00	00	00	00	00	00	39	EC	00	02	00	00	00	00	9i.....
00BBD671D0	23	EC	00	02	00	00	00	00	23	EC	00	02	00	00	00	00	#i.....#i.....
00BBD671E0	40	00	00	00	00	00	00	00	01	00	00	00	18	00	00	00	@.....
00BBD671F0	02	00	00	00	00	00	00	00	03	00	02	00	28	00	6D	60	(.m`

Offset	Field Name	Size	Description / Value
0x00	This LSN	8 bytes	현재 레코드의 LSN 값 / 0x200EC39
0x08	Previous LSN	8 bytes	이전 레코드의 LSN 값 / 0x200EC23
0x20	Redo OP	2 bytes	Redo 작업 코드 / 0x03: DeallocateFileRecordSegment
0x22	Undo OP	2 bytes	Undo 작업 코드 / 0x02: InitializeFileRecordSegment

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 LSN을 통한 트랜잭션 체인 구성-2

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00BBD67230	01	00	01	00	38	00	01	00	47	EC	00	02	00	00	00	00	8...Gi.....
00BBD67240	39	EC	00	02	00	00	00	00	39	EC	00	02	00	00	00	00	9i.....9i.....
00BBD67250	30	00	00	00	00	00	00	00	01	00	00	00	18	00	00	00	0.....
00BBD67260	00	00	00	00	00	00	00	00	16	00	15	00	28	00	08	00	(...

Offset	Field Name	Size	Description / Value
0x00	This LSN	8 bytes	현재 레코드의 LSN 값 / 0x200EC47
0x08	Previous LSN	8 bytes	이전 레코드의 LSN 값 / 0x200EC39
0x20	Redo OP	2 bytes	Redo 작업 코드 / 0x16: ClearBitsInNonResidentBitMap
0x22	Undo OP	2 bytes	Undo 작업 코드 / 0x15: SetBitsInNonResidentBitMap

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

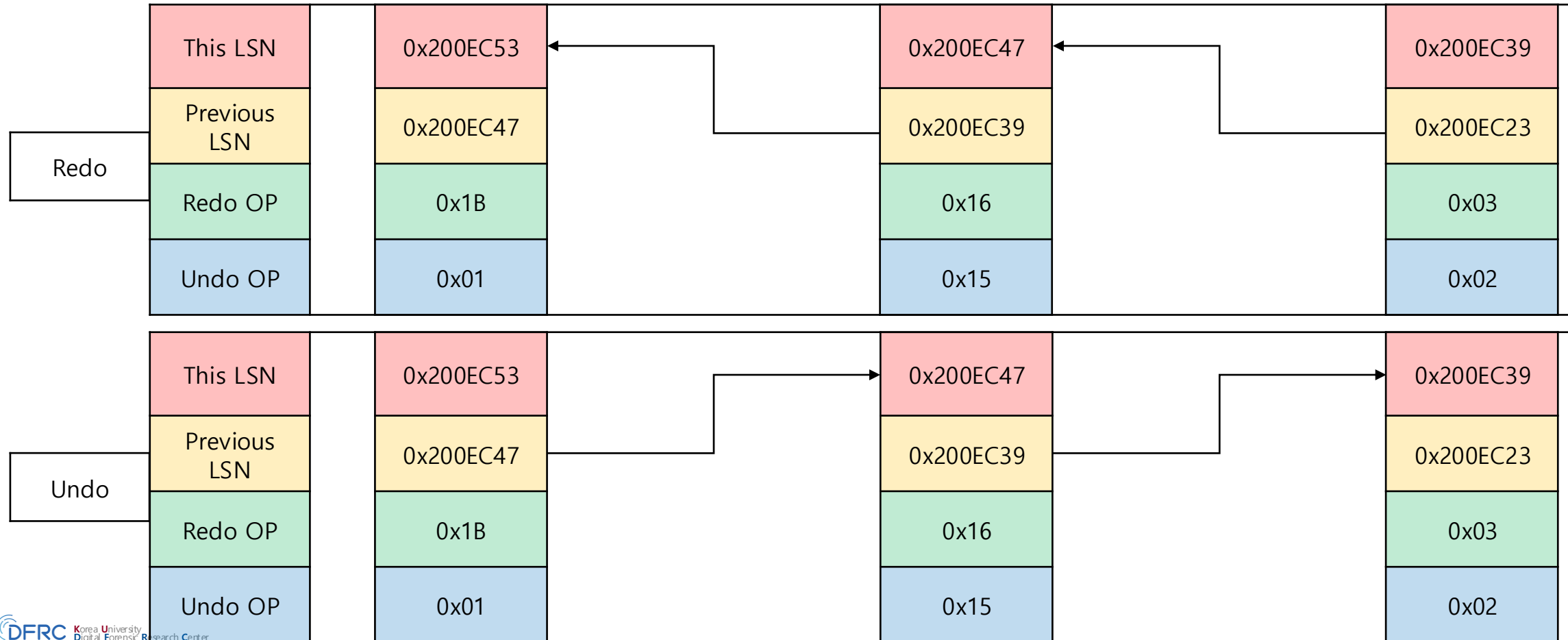
■ \$LogFile의 LSN을 통한 트랜잭션 체인 구성-3

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00BBD67290	2F	00	00	00	01	00	00	00	53	EC	00	02	00	00	00	00	/.....Si.....
00BBD672A0	47	EC	00	02	00	00	00	00	00	00	00	00	00	00	00	00	Gi.....
00BBD672B0	28	00	00	00	00	00	00	00	01	00	00	00	18	00	00	00	(.....
00BBD672C0	02	00	00	00	00	00	00	00	1B	00	01	00	28	00	00	00	(...

Offset	Field Name	Size	Description / Value
0x00	This LSN	8 bytes	현재 레코드의 LSN 값 / 0x200EC53
0x08	Previous LSN	8 bytes	이전 레코드의 LSN 값 / 0x200EC47
0x20	Redo OP	2 bytes	Redo 작업 코드 / 0x1B: ForgetTransaction
0x22	Undo OP	2 bytes	Undo 작업 코드 / 0x01: CompensationLogRecord

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 LSN을 통한 트랜잭션 체인 구성 종합 및 동작 과정 파악
 - Redo와 Undo 관점에서의 트랜잭션 체인 구성 가능



메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- \$LogFile의 LSN을 통한 트랜잭션 체인 구성 종합 및 동작 과정 파악
 - Redo 관점에서의 동작 과정: 0x03 → 0x16 → 0x1B
 - MFT Entry 해제 → 비트맵 해제 → 커밋 완료 처리
 - Undo 관점에서의 동작 과정: 0x01 → 0x15 → 0x02
 - Undo 보상 레코드 기록 → 비트맵 할당 → MFT Entry 생성

OP 코드	OP 종류	설명
0x01	CompensationLogRecord	로그 레코드 보상 작업
0x02	InitializeFileRecordSegment	새로운 MFT Entry 생성 및 초기화
0x03	DeallocateFileRecordSegment	MFT Entry 해제
0x15	SetBitsInNonResidentBitMap	특정 클러스터 영역 사용 중으로 표시
0x16	ClearBitsInNonResidentBitMap	특정 클러스터 영역 빈 공간으로 표시
0x1B	ForgetTransaction	커밋 되어 완료 처리

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- 0x2F번 MFT Entry의 \$DATA 속성 분석
 - MFT Entry Offset 계산식을 이용해 MFT Entry 이동 및 \$DATA 분석
 - 타겟 MFT Entry Offset = (0xC000B * 8 + 0x6) * 0x200 = 0xC000BC00

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00C000BD30	8D	3A	98	AF	65	72	F8	93	80	00	00	00	48	00	00	00	.:~`erø“€...H...
00C000BD40	01	00	00	00	00	00	05	00	00	00	00	00	00	00	00	00	
00C000BD50	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00C000BD60	00	10	00	00	00	00	00	00	00	09	00	00	00	00	00	00	
00C000BD70	00	09	00	00	00	00	00	00	21	01	95	18	00	00	00	00	!..•.....

Offset	Field Name	Size	Description / Value
0x00	Attribute Header	24 bytes	Common Header (16 bytes) + Non-Resident Header (48 bytes) / 0x80 (\$DATA), 0x01 (Non-Resident 방식)
0x20	Offset of Run List	2 bytes	Run List Offset / 0x40
0x40	Run List	4 bytes	Run List / Run length: 0x01, Run offset: 0x1895

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

- 0x2F번 MFT Entry의 데이터 영역 확인
 - Run list 계산: $0x1895 * 0x1000 = 0x1895000$

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001895000	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
0001895010	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D
-- 중략 --																	
0001895870	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
0001895880	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D
0001895890	46	52	43	20	4D	4F	44	49	46	59	20	44	46	52	43	20	FRC MODIFY DFRC
00018958A0	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
00018958B0	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D
00018958C0	46	52	43	20	4D	4F	44	49	46	59	20	44	46	52	43	20	FRC MODIFY DFRC
00018958D0	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
00018958E0	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D
00018958F0	46	52	43	20	4D	4F	44	49	46	59	20	44	46	52	43	20	FRC MODIFY DFRC

메타데이터 및 구조체 분석 - [실습] 파일 복구 (3)

■ recoverdfrc.txt 데이터 복구

- 데이터 영역만큼의 범위 복사 및 새로운 파일로 저장하여 결과 확인
 - 기존 작성한 파일 데이터 복구 확인 가능

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
00000010	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D
00000020	46	52	43	20	4D	4F	44	49	46	59	20	44	46	52	43	20	FRC MODIFY DFRC
00000030	4D	4F	44	49	46	59	20	44	46	52	43	20	4D	4F	44	49	MODIFY DFRC MODI
00000040	46	59	20	44	46	52	43	20	4D	4F	44	49	46	59	20	44	FY DFRC MODIFY D



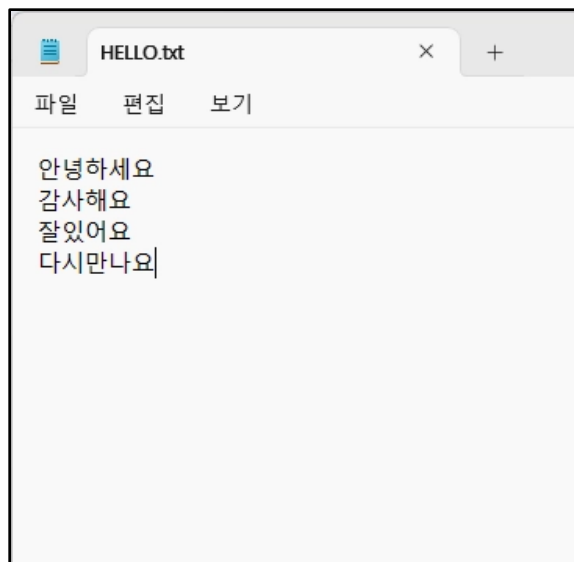
```
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC MODIFY DFRC
```

[recoverdfrc.txt 복원]

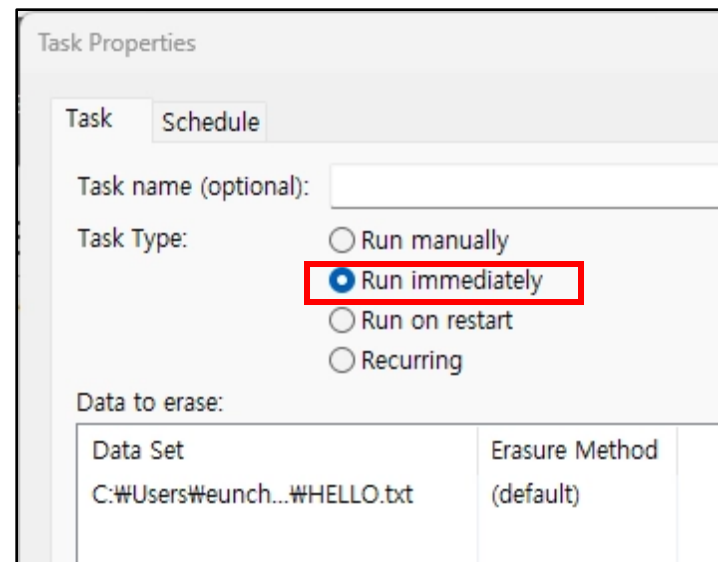
메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ 파일 생성 및 삭제

- C:\Users\WeunchaeKIM\Desktop\파일시스템세미나\file 경로 → HELLO.txt 파일 생성
- 와이핑 도구로 HELLO.txt 삭제
 - Eraser 이용 → Run immediately 옵션 (즉시 삭제)



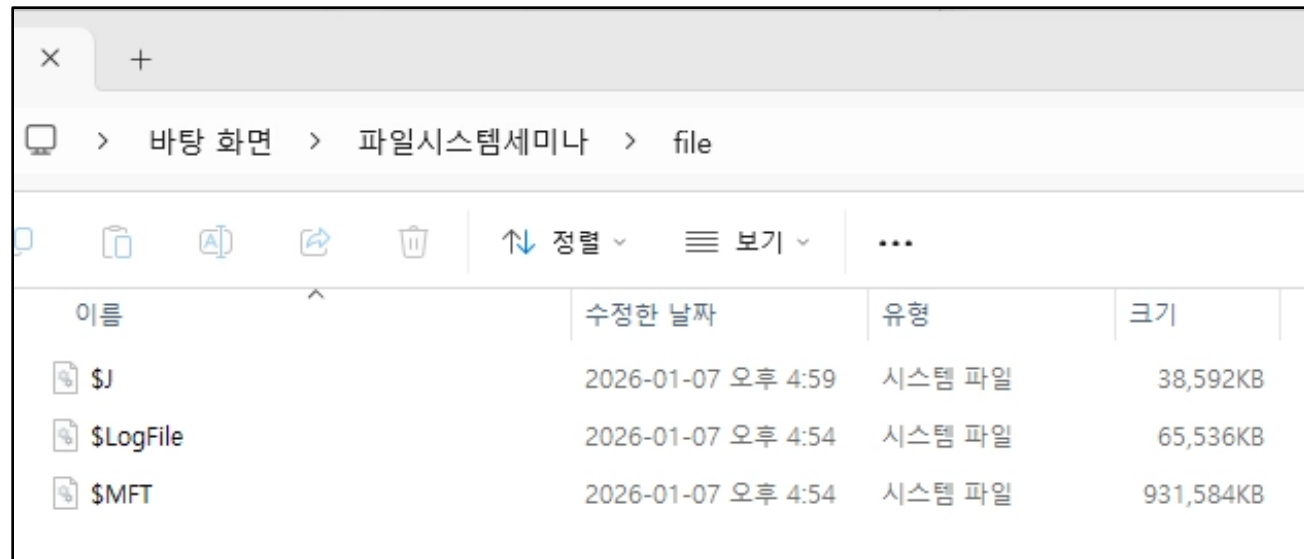
[hello.txt 내용]



[Eraser 옵션 설정]

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

- 분석 대상 파일 추출
 - 삭제 완료 직후 이미징
 - \$LogFile, \$UsnJrnl:\$J, \$MFT 추출



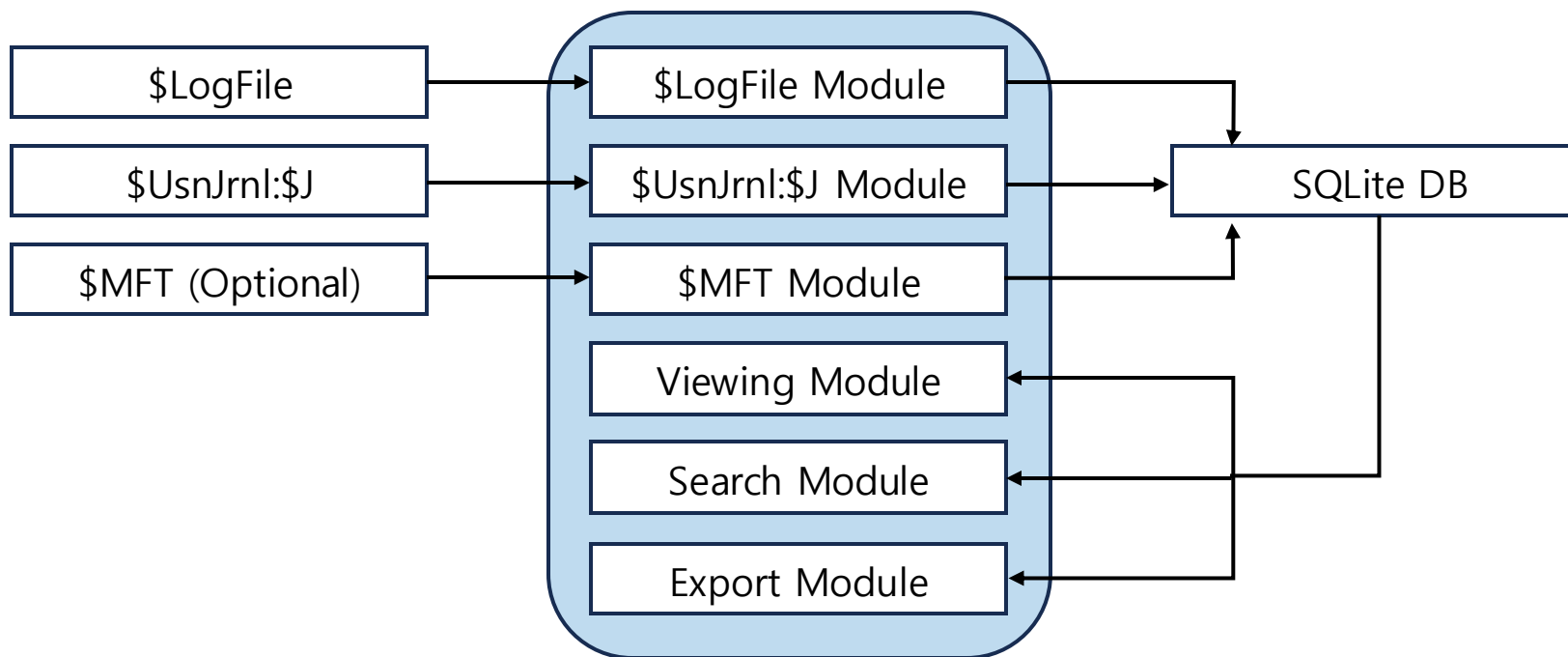
이름	수정한 날짜	유형	크기
\$J	2026-01-07 오후 4:59	시스템 파일	38,592KB
\$LogFile	2026-01-07 오후 4:54	시스템 파일	65,536KB
\$MFT	2026-01-07 오후 4:54	시스템 파일	931,584KB

[파일 추출]

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 소개

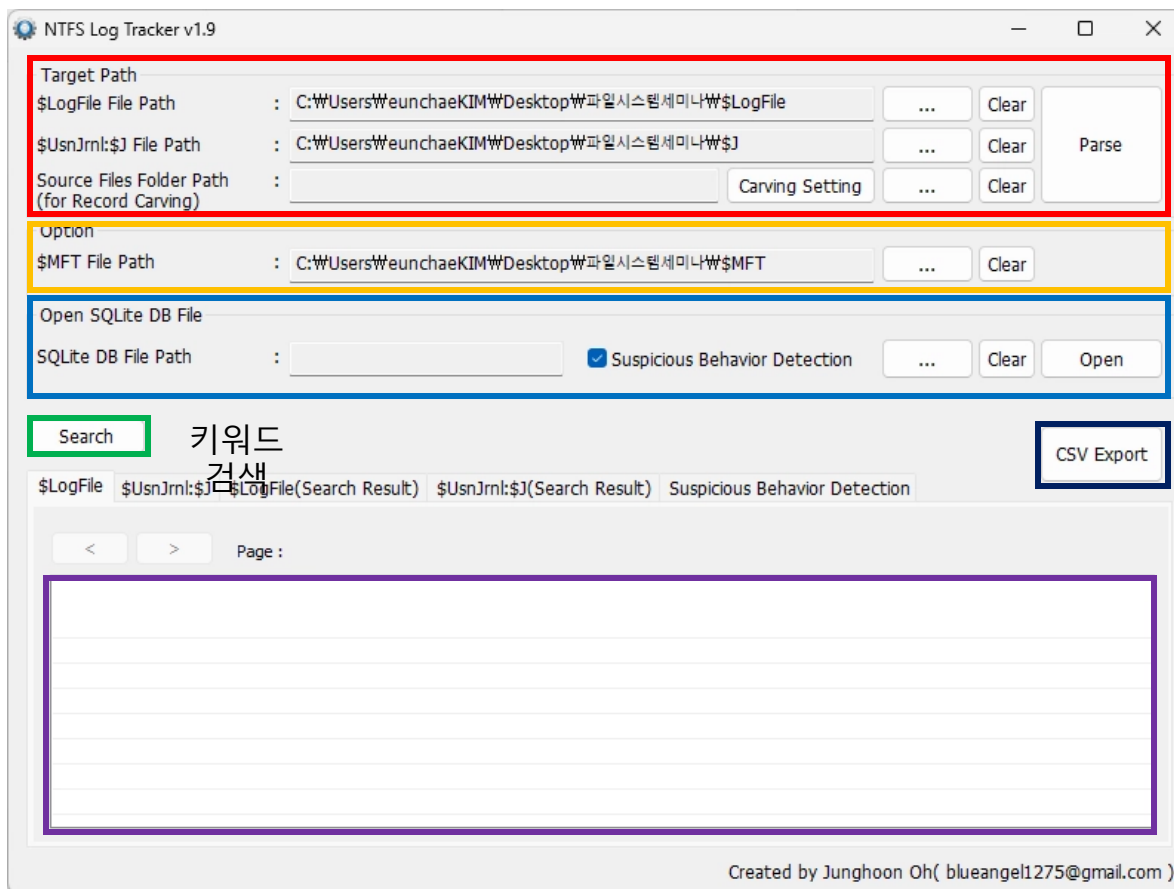
- NTFS 트랜잭션 로그 분석 도구 → 파일 시스템의 변경 이력 추적
- 분석 대상: \$LogFile, \$UsnJrnl:\$J, \$MFT (Optional)
 - \$MFT 파일이 있을 경우 정확한 파일명, 전체 경로 복원 가능



메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 사용자 인터페이스

- 추출한 파일 입력 → Parse 진행



The screenshot shows the NTFS Log Tracker v1.9 application window. It features several input fields and buttons for configuring file analysis. Annotations with colored boxes and text are present:

- Red box:** Encloses the 'Target Path' section, including '\$LogFile File Path', '\$UsrJm1:\$J File Path', and 'Source Files Folder Path'. A note to the right says '\$LogFile / \$UsrJm1:\$J 경로 입력'.
- Yellow box:** Encloses the 'Option' section, specifically the '\$MFT File Path' field. A note to the right says '전체 경로 생성을 위한 \$MFT 경로 입력'.
- Blue box:** Encloses the 'Open SQLite DB File' section, including the 'SQLite DB File Path' field and the 'Open' button. A note to the right says '이전에 생성된 SQLite DB 읽기'.
- Green box:** Encloses the 'Search' button. A note to the right says '키워드 검색'.
- Dark blue box:** Encloses the 'CSV Export' button. A note to the right says 'CSV 출력'.
- Purple box:** Encloses the large table area at the bottom, which displays search results. A note to the right says '파싱 결과 출력'.

At the bottom of the window, it says 'Created by Junghoon Oh(blueangel1275@gmail.com)'.

\$LogFile / \$UsrJm1:\$J 경로 입력

전체 경로 생성을 위한 \$MFT 경로
입력

이전에 생성된 SQLite DB
읽기

CSV
출력

파싱 결과
출력

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 분석 결과 - \$LogFile

- \$LogFile 탭에서 파싱 결과 확인 → LSN 기준 오름차순 정렬
- 주요 속성 정리

속성	설명
LSN (Log Sequence Number)	각 로그 레코드에 부여되는 고유 번호 / 트랜잭션의 순서를 식별
EventTime	해당 트랜잭션이 발생한 시간
Event	수행된 작업 유형 설명 (Create, Delete, Rename 등)
Detail	변경된 속성 또는 내부 동작 상세 정보
File / Directory Name	대상 파일 또는 디렉터리 이름
Full Path	복원된 전체 경로 (\$MFT 필요)
CreationTime	파일 생성 시간
ModifiedTime	파일 내용 수정 시간
MFTModifiedTime	MFT Entry 메타데이터가 수정된 시간

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

- NTFS Log Tracker 분석 결과 - \$LogFile
 - 결과 화면 캡처

LSN	EventTime(UTC+9)	Event	Detail	File/Directory Name	Full Path(from \$MFT)							
LSN	EventTime(UTC+9)	Event	Detail	File/Directory Name	Full Path(from \$MFT)	CreationTime	ModifiedTime	MFTModifiedTime	AccessedTime	Redo	Target VCN	Cluster Index
19915113776	2026-02-18 22:51:29	Updating Modified Time	ModifiedTime : 2026-02-18 22:51...	adtemp <Guessed>	\\Users\\WeunchaeKIM\\AppData\\Local\\Temp\\Wa...					Update Resident Value	0xA053	0
19915113921	2026-02-18 22:51:29	File Creation		ftk_bc920053-f04b-40f3-92e4-8...	\\Users\\WeunchaeKIM\\AppData\\Local\\Temp\\Wa...	2026-02-18 22:51:29	2026-02-18 22:51:29	2026-02-18 22:51:29	2026-02-18 22:51:29	Initialize File Record Segment	0x725	6
19915114159		Writing Content of Resident File	Writing Size : 176	ftk_bc920053-f04b-40f3-92e4-8...	\\Users\\WeunchaeKIM\\AppData\\Local\\Temp\\Wa...					Update Resident Value	0x725	6
19915114479		Writing Content of Non-Resident File	Data Runs(in Volume) : 3666240(...							Update Mapping Pairs	0x6883	2
19915114543		Updating Modified Time	ModifiedTime : 2026-02-18 22:50...	Reporting and NEL-journal <Gues...	\\Users\\WeunchaeKIM\\AppData\\Local\\GoogleW...					Update Resident Value	0x6883	2
19915114719		Updating Modified Time	ModifiedTime : 2026-02-18 22:50...	Reporting and NEL <Guessed>	\\Users\\WeunchaeKIM\\AppData\\Local\\GoogleW...					Update Resident Value	0x6883	0
19915115062		Updating Modified Time	ModifiedTime : 2026-02-18 22:51...	XboxGamingOverlayTraces_FT_...	\\Users\\WeunchaeKIM\\AppData\\Local\\Package...					Update Resident Value	0xED5	2
19915115264	2026-02-18 22:51:31	File Creation		f_008dfa	\\Users\\WeunchaeKIM\\AppData\\Local\\GoogleW...	2026-02-18 22:51:31	2026-02-18 22:51:31	2026-02-18 22:51:31	2026-02-18 22:51:31	Initialize File Record Segment	0x7E23	4
19915115617	2026-02-18 22:51:31	Writing Content of Non-Resident File	Data Runs(in Volume) : 35038149...	f_008dfa	\\Users\\WeunchaeKIM\\AppData\\Local\\GoogleW...					Update Mapping Pairs	0x7E23	4
19915116062	2026-02-18 22:51:31	Writing Content of Non-Resident File	Data Runs(in Volume) : 3669888(...							Update Mapping Pairs	0x6875	0
						CreationTime	ModifiedTime	MFTModifiedTime	AccessedTime	Redo	Target VCN	Cluster Index

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 분석 결과 - \$UsnJrnl:\$J

- \$UsnJrnl:\$J 탭에서 파싱 결과 확인 → USN 기준 오름차순 정렬
- 주요 속성 정리

속성	설명
TimeStamp	파일 변경 이벤트 발생 시간
USN (Update Sequence Number)	변경 이벤트에 부여된 고유 증가 번호
File / Directory Name	변경 대상 파일 또는 디렉터리 이름
Event	변경 사유 (Data Extend, File Create, Rename 등)
FileReferenceNumber	대상 파일의 MFT 참조 번호
ParentFileReferenceNumber	상위 디렉터리의 MFT 참조 번호

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

- NTFS Log Tracker 분석 결과 - \$UsnJrnl:\$J
 - 결과 화면 캡처

TimeStamp(UTC+9)		USN	File/Directory Name	Full Path(from \$MFT)					
TimeStamp(UTC+9)	USN	File/Directory Name	Full Path(from \$MFT)	Event	Source Info	File Attribute	Carving Flag	FileReferenceNumber	ParentFileReferenc
2026-02-18 16:48:10	4269801472	1f26b0397e745be1_0		File_Created , Data_Added	Normal	Archive		0x0006000000003D196	0x000800000000004
2026-02-18 16:48:10	4269801568	1f26b0397e745be1_0		File_Created , Data_Added , Data_Overwritten	Normal	Archive		0x0006000000003D196	0x000800000000004
2026-02-18 16:48:10	4269801664	1f26b0397e745be1_0		File_Created , Data_Added , Data_Overwritten , File_Closed	Normal	Archive		0x0006000000003D196	0x000800000000004
2026-02-18 16:48:10	4269801760	cd72dd99a6199e05_0		File_Created	Normal	Archive		0x00090000000042DBE	0x000800000000004
2026-02-18 16:48:10	4269801856	cd72dd99a6199e05_0		File_Created , Data_Added	Normal	Archive		0x00090000000042DBE	0x000800000000004
2026-02-18 16:48:10	4269801952	cd72dd99a6199e05_1		File_Created	Normal	Archive		0x00070000000042DBF	0x000800000000004
2026-02-18 16:48:10	4269802048	cd72dd99a6199e05_1		File_Created , Data_Added	Normal	Archive		0x00070000000042DBF	0x000800000000004
2026-02-18 16:48:10	4269802144	cd72dd99a6199e05_0		File_Created , Data_Added , Data_Overwritten	Normal	Archive		0x00090000000042DBE	0x000800000000004
2026-02-18 16:48:10	4269802240	cd72dd99a6199e05_0		File_Created , Data_Added , Data_Overwritten , File_Closed	Normal	Archive		0x00090000000042DBE	0x000800000000004
2026-02-18 16:48:10	4269802336	cd72dd99a6199e05_1		File_Created , Data_Added , File_Closed	Normal	Archive		0x00070000000042DBF	0x000800000000004
2026-02-18 16:48:10	4269802432	d6a749bf0db46e4d_0		File_Created	Normal	Archive		0x00070000000042DC0	0x000800000000004
2026-02-18 16:48:10	4269802528	d6a749bf0db46e4d_0		File_Created , Data_Added	Normal	Archive		0x00070000000042DC0	0x000800000000004
2026-02-18 16:48:10	4269802624	03d607c6778d5e63_0	\\Users\\WeunchaeKIM\\AppData\\Roaming\\Notion\\Partitions\\notion\\Shared Dictionary\\cache\\03d607c67...	File_Closed , File_Deleted	Normal	Archive		0x000800000000415F1	0x000300000000063
Event				Source Info	File Attribute	Carving Flag	FileReferenceNumber	ParentFileReferenc	

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 분석 결과 - Search

- 사용자 인터페이스 'Search' 버튼으로 검색 수행 가능
 - 여러 속성 검색 시 AND 처리
 - 검색 결과는 별도의 탭에서 확인 가능
- \$LogFile 검색 가능 속성
 - LSN, EventTime, Event, File / Directory Name, Full Path 등
- \$UsnJrnl 검색 가능 속성
 - TimeStamp, USN, File / Directory Name, Full Path, Event 등

\$LogFile(Search Result) \$UsnJrnl:\$J(Search Result)



Search	
\$LogFile	
LSN	:
Event Time	:
Event	:
File/Directory Name	:
Full Path	:
Target VCN	:
MFT Cluster Index	:
\$UsnJrnl	
TimeStamp	:
USN	:
File/Directory Name	:
Full Path	:
Event	:
FileReferenceNumber	:
ParentFileReferenceNumber	:

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 분석 결과

■ Suspicious Behavior Detection

- 단시간 내 다량 삭제·대규모 파일명 변경 등 비정상적 파일 시스템 조작 행위 추적 기능
- 오탐 가능성 존재 → 파싱 기록 교차 분석 필요

탐지 이벤트	설명
시스템 시간 변조	시스템 시간을 과거 혹은 미래로 변경하는 행위
의심스러운 도구 설치	알려진 완전 삭제 도구, 암호화 도구, 스테가노그래피 도구, 파일 타임스탬프 변조 도구 설치 흔적 확인
의심스러운 도구 실행	알려진 완전 삭제 도구, 암호화 도구, 스테가노그래피 도구, 파일 타임스탬프 변조 도구 실행 흔적 확인
의심스러운 도구 흔적	완전 삭제 도구의 전형적인 패턴 추적 및 암호화 도구 임시 파일 등 확인
사용자 문서 파일 삭제	Word, Excel, PPT, HWP, PDF 문서에 대한 삭제
파일 타임스탬프 변조	변조 도구 혹은 악성 코드에 의한 파일 스탬프 변조

메타데이터 및 구조체 분석 - [실습] NTFS Log Tracker 활용

■ NTFS Log Tracker 분석 결과

- Suspicious Behavior Detection 탭에서 'Trace of Suspicious Programs' 감지
 - 해당 로그 디테일 확인 → 전형적인 와이핑 도구 흔적 발견 (파일명이 2회 이상 변경 후 삭제)
 - HELLO.txt → fHzWkrl/] → jvdd2r1=K 등으로 파일명 다회 변경 후 삭제된 기록 확인

\$LogFile \$UsnJrnl:\$J \$LogFile(Search Result) \$UsnJrnl:\$J(Search Result) Suspicious Behavior Detection				
Detection Overview				
No	Category			
1	Timestamp Manipulation	14	\$LogFile	
2	Execution of Suspicious Programs	1	\$LogFile	
3	Traces of Suspicious Programs	1	\$LogFile	
4	Deletion of Document File	17	\$UsnJrnl:\$J	
5	Installation of Suspicious Programs	1	\$UsnJrnl:\$J	
6	Execution of Suspicious Programs	1	\$UsnJrnl:\$J	
7	Traces of Suspicious Programs	1	\$UsnJrnl:\$J	
8	Timestamp Manipulation	2	\$UsnJrnl:\$J	

Detail				
The Trace of Typical Data Wiping : Deletion after renaming more than 2 times				
2026-02-18 22:50:02	4311321728	fHzWkrl/]	\\Users\\eunchaeKIM\\Desktop\\파일시스템세미나\\file\\fHzWkrl/]	
2026-02-18 22:50:02	4311321808	jvdd2r1=K	\\Users\\eunchaeKIM\\Desktop\\파일시스템세미나\\file\\jvdd2r1=K	
2026-02-18 22:50:02	4311321888	jvdd2r1=K	\\Users\\eunchaeKIM\\Desktop\\파일시스템세미나\\file\\jvdd2r1=K	
2026-02-18 22:50:02	4311321968	jvdd2r1=K	\\Users\\eunchaeKIM\\Desktop\\파일시스템세미나\\file\\jvdd2r1=K	

The Sleuth Kit (TSK)

소개

■ The Slueth Kit (TSK)란?

- 디스크 이미지를 분석하고 파일을 복구할 수 있는 명령줄 도구
- Linux, macOS, Windows 등 다양한 운영체제에서 동작 가능
- 오픈 소스 → 환경, 목적에 맞게 커스터마이징 가능
- 지원 파일 시스템
 - EXT2, EXT3, EXT4
 - FAT, exFAT
 - HFS
 - NTFS 등



기능

■ 주요 명령어

- mmls: 디스크 이미지 내의 파티션 테이블 구조 확인
- fsstat: 파일 시스템 상세 정보 확인
- fls: 특정 파티션 내의 파일 및 디렉터리 목록 표시
 - r / r: 현재 존재하는 파일
 - r / r *: 삭제된 파일
 - d / d: 현재 존재하는 디렉터리
 - d / d *: 삭제된 디렉터리
- istat: 특정 파일의 메타데이터 분석
- icat: 특정 파일 내용 추출

Autopsy 소개

■ Autopsy란?

- TSK 기반 GUI 포렌식 분석 소프트웨어
 - 명령줄 도구인 TSK를 그래픽 환경에서 사용 가능

■ 주요 기능

- 케이스 생성 및 관리 → 사건 단위 분석 가능
- 디스크 및 파일 시스템 분석
- 자동 분석 모듈 지원
- 보고서 생성



Autopsy 소개

■ 주요 모듈 정리

- 디스크 이미지 분석 시, Ingest Module 창에서 선택 가능
- 대용량 이미지 자동 분석 가능
- 다른 수사관이더라도 이미지 분석 일관성 / 재현성 확보 가능

이름	역할
Recent Activity	사용자 최근 활동 아티팩트 추출
Hash Lookup	Hash 값 산출 → Notable / Known / Unknown 등 판단
File Type Identification	파일 헤더, 시그니처 기반 파일 유형 식별
Extension Mismatch Detector	확장자와 실제 파일 타입 불일치 탐지
Keyword Search	문자열 추출 및 키워드 검색
Encryption Detection	암호화 / 암호 가능성 파일 탐지
Picture Analyzer	이미지 파일 속 EXIF* 정보 분석

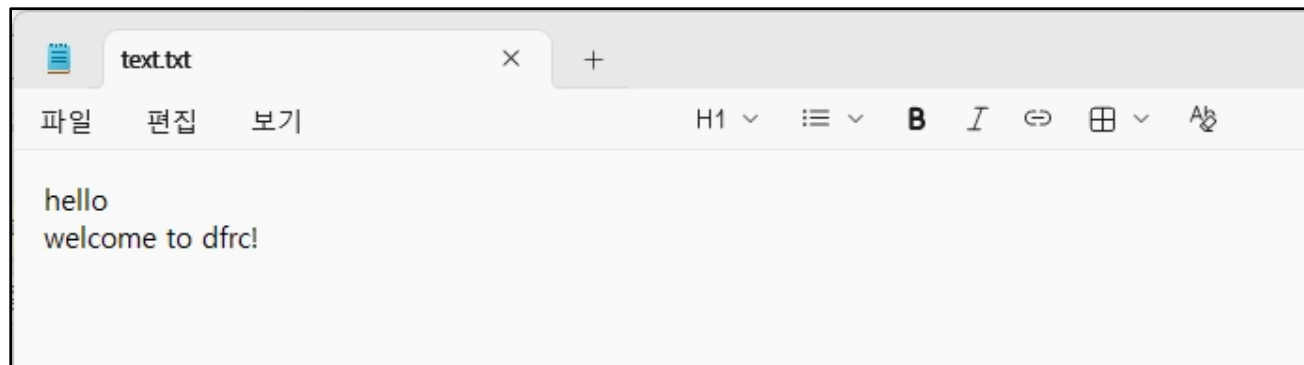
* EXIF (Exchangeable Image File Format): 사진, 오디오 파일 속에 촬영 정보를 같이 넣어두는 메타데이터 포맷 (카메라 모델, 촬영 일자, 셔터 스피드 등 저장)

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ 실습 환경

- FAT32
 - 1 GB 크기 VHD 생성 후 text.txt 파일 생성 → 이미지 생성 (fat32_exp1.001)
- NTFS
 - 1 GB 크기 VHD 생성 후 text.txt 파일 생성 → 이미지 생성 (ntfs_exp1.001)



[text.txt 내용]

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 파일 시스템 메타데이터 출력

- 명령: `fsstat fat32_exp1.001` → 파일 시스템의 전체 구조 요약 확인 가능

```
FILE SYSTEM INFORMATION
-----
File System Type: FAT32

OEM Name: MSDOS5.0
Volume ID: 0x748bbf46
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): File System Type Label: FAT32
Next Free Sector (FS Info): 8240
Free Sector Count (FS Info): 2082768

Sectors before file system: 128

File System Layout (in sectors)
Total Range: 0 - 2091007
* Reserved: 0 - 4121
** Boot Sector: 0
** FS Info Sector: 1
** Backup Boot Sector: 6
* FAT 0: 4122 - 6156
* FAT 1: 6157 - 8191
* Data Area: 8192 - 2091007
```

구분	값	설명
File System Type	FAT32	파일 시스템 타입
Next Free Sector	Sector 8240	빈 공간을 찾기 시작하는 기준 위치
Free Sector Count	2082768 Sectors	현재 남아있는 빈 섹터 수
Reserved	Sector 0 - 4121	부트 정보, FAT 관리 등 메타데이터가 저장된 예약 영역
Boot Sector	Sector 0	BPB 및 구조 정보 포함 섹터 위치
FS Info Sector	Sector 1	남은 공간, 다음 할당 위치 저장 섹터 위치
Backup Boot Sector	Sector 6	Boot Sector 손상 대비 백업 섹터 위치
FAT 0	Sector 4122 - 6156	클러스터 할당 정보 저장 섹터
FAT 1	Sector 6157 - 8191	FAT 0의 복사본 저장 섹터
Data Area	Sector 8192 - 2091007	실제 파일 및 디렉터리 저장 섹터 ¹⁶¹

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 파일 시스템 메타데이터 출력

- 명령: `fsstat fat32_exp1.001`

METADATA INFORMATION

Range: 2 - 33325062
Root Directory: 2
CONTENT INFORMATION

Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 2 - 260353
FAT CONTENTS (in sectors)

8192-8199 (8) -> EOF
8200-8207 (8) -> EOF
8208-8215 (8) -> EOF
8216-8223 (8) -> EOF
8224-8231 (8) -> EOF
8232-8239 (8) -> EOF

구분	값	설명
Range	2 - 33325062	TSK에서 자체적으로 할당 가능한 번호의 범위
Root Directory	2	루트 디렉터리 클러스터 번호
Sector Size	512	1 섹터 크기 (bytes)
Cluster Size	4096	1 클러스터 크기 (bytes)
Total Cluster Range	2 - 260353	실제 사용 가능한 클러스터 번호 범위
FAT CONTENTS	8192 - 8199 (8) -> EOF	클러스터들의 체인 구조 파악 EOF: 단일 클러스터, 체인 X (해당 예제는 체인 발견 X)
	8200 - 8207 (8) -> EOF	
	8208 - 8215 (8) -> EOF	

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 이미지 파일 및 디렉터리 목록 출력

- 명령: `fls -r fat32_exp1.001` (`-r` 옵션: 모든 하위 디렉터리를 재귀적으로 내려가며 목록 출력)

```
r/r 3:
d/d 6: System Volume Information
+ r/r 135: WPSettings.dat
r/r * 8: 새 텍스트 문서.txt
d/d 9: $RECYCLE.BIN
+ r/r 389: desktop.ini
r/r 10: text.txt
v/v 33325059: $MBR
v/v 33325060: $FAT1
v/v 33325061: $FAT2
V/V 33325062: $OrphanFiles
```

구분	설명
r / r	현재 존재하는 파일 의미
10	TSK 내 파일 식별을 위한 가상 번호
text.txt	파일 이름

- d / d: 현존 디렉터리 의미
- v / v: \$MBR, \$FAT1 등 볼륨 구조를 나타내는 메타데이터 파일
- V / V: \$OrphanFiles 등 특수 파일 모음 디렉터리
- +: 디렉터리 하위에 위치한 파일 의미 / *: 삭제된 파일, 덮어쓰워지지 않아 TSK가 찾아낸 상태

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 이미지에서 text.txt 메타데이터 확인

- 명령: `istat fat32_exp.001 10` (10: fls 명령으로 찾은 text.txt 파일 식별 번호 입력)

```
Directory Entry: 10
Allocated
File Attributes: File, Archive
Size: 24
Name: text.txt

Directory Entry Times:
Written:      2026-02-18 15:48:36 (Accessed:  2026-02-18 00:00:00 (Created:  2026-02-18 15:48:20 (
Sectors:
8232 0 0 0 0 0 0 0
```

- 파일 크기: 24 bytes
- Directory Entry Times
 - Written: 2026-02-18 15:48:36 / Accessed: 2026-02-18 00:00:00 / Created: 2026-02-18 15:48:20
- 할당 섹터: 1개 클러스터 중 8232 섹터에만 파일 데이터 할당

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS 파일 시스템 메타데이터 출력

- 명령: `fsstat ntfs_exp1.001` → 파일 시스템의 전체 구조 요약 확인 가능

```
FILE SYSTEM INFORMATION
-----
File System Type: NTFS
Volume Serial Number: 300E00E20E00A348
OEM Name: NTFS
Volume Name: 새 볼륨
Version: Windows XP

METADATA INFORMATION
-----
First Cluster of MFT: 87125
First Cluster of MFT Mirror: 2
Size of MFT Entries: 1024 bytes
Size of Index Records: 4096 bytes
Range: 0 - 256
Root Directory: 5

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 0 - 261374
Total Sector Range: 0 - 2091006
```

구분	값	설명
File System Type	NTFS	파일 시스템 타입
First Cluster of MFT	87125	\$MFT가 시작하는 클러스터 번호
Size of MFT Entries	1024	각 MFT Entry 크기 (bytes)
Size of Index Records	4096	디렉터리 인덱스 레코드 크기 (bytes)
Range	0 - 256	MFT Entry 번호 범위
Root Directory	5	루트 디렉터리 MFT Entry 번호
Sector Size	512	1 섹터 크기 (bytes)
Cluster Size	4096	1 클러스터 크기 (bytes)
Total Cluster Range	0 - 261374	디스크에 존재하는 전체 클러스터 범위
Total Sector Range	0 - 2091006	디스크에 존재하는 전체 섹터 범위

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS 파일 시스템 메타데이터 출력

- 명령: `fsstat ntfs_exp1.001` → \$AttrDef를 통해 속성 크기 및 저장 방식 요약

```
$AttrDef Attribute Values:
$STANDARD_INFORMATION (16)  Size: 48-72  Flags: Resident
$ATTRIBUTE_LIST (32)       Size: No Limit  Flags: Non-resident
$FILE_NAME (48)           Size: 68-578  Flags: Resident,Index
$OBJECT_ID (64)           Size: 0-256  Flags: Resident
$SECURITY_DESCRIPTOR (80)  Size: No Limit  Flags: Non-resident
$VOLUME_NAME (96)         Size: 2-256  Flags: Resident
$VOLUME_INFORMATION (112)  Size: 12-12  Flags: Resident
$DATA (128)               Size: No Limit  Flags:
$INDEX_ROOT (144)         Size: No Limit  Flags: Resident
$INDEX_ALLOCATION (160)    Size: No Limit  Flags: Non-resident
$BITMAP (176)             Size: No Limit  Flags: Non-resident
$REPARSE_POINT (192)      Size: 0-16384  Flags: Non-resident
$EA_INFORMATION (208)     Size: 8-8  Flags: Resident
$EA (224)                 Size: 0-65536  Flags:
$LOGGED_UTILITY_STREAM (256) Size: 0-65536  Flags: Non-resident
```

Flag	속성
Resident	\$STANDARD_INFORMATION \$FILE_NAME \$OBJECT_ID \$VOLUME_NAME \$VOLUME_INFORMATION \$INDEX_ROOT \$EA_INFORMATION
Non - Resident	\$ATTRIBUTE_LIST \$SECURITY_DESCRIPTOR \$INDEX_ALLOCATION \$BITMAP \$REPARSE_POINT \$LOGGED_UTILITY_STREAM
Resident / Non – Resident	\$DATA \$EA

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS 이미지 파일 및 디렉터리 목록 출력

- 명령: `fls -r ntfs_exp1.001` (`-r` 옵션: 모든 하위 디렉터리를 재귀적으로 내려가며 목록 출력)

```
++ r/r 41-128-1:      desktop.ini
r/r 9-128-8:      $Secure:$SDS
r/r 9-144-11:     $Secure:$SDH
r/r 9-144-14:     $Secure:$SII
r/r 10-128-1:     $UpCase
r/r 10-128-4:     $UpCase:$Info
r/r 3-128-3:      $Volume
d/d 36-144-1:     System Volume Information
+ r/r 37-128-1:    WPSettings.dat
r/r 38-128-1:     text.txt
V/V 256:          $OrphanFiles
```

구분	설명
r / r	현재 존재하는 파일 의미
38-128-1	38: MFT Entry 번호 128: Attribute 타입 1: Attribute ID
text.txt	파일 이름

- 38-128-1: MFT 38번 엔트리에 있는 \$DATA (0x80, 128) 속성의 1번째 인스턴스가 text.txt 파일 실제 데이터

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS 이미지에서 text.txt 메타데이터 확인

- 명령: `istat ntfs_exp.001 38-128-1` (38-128-1: fls 명령으로 찾은 text.txt 파일 식별 번호 입력)

```
MFT Entry Header Values:  
Entry: 38          Sequence: 1  
$LogFile Sequence Number: 2126928  
Allocated File  
Links: 1
```

- Entry: 38 / Sequence: 1
 - 38번 Entry가 1번 사용 → 여러 번 재할당 시, 값 증가
- \$LogFile Sequence Number (LSN): 2126928, 해당 엔트리 갱신 시 사용된 트랜잭션 로그 시퀀스 번호
- Allocated File: 현재 사용중인 파일
- Links: 1, 해당 파일을 가리키는 디렉터리 엔트리 개수 (2 이상이면 Hard Link)

[실습] FAT32 / NTFS 파일 시스템 분석

- TSK

- NTFS 이미지에서 text.txt 메타데이터 확인 (\$STANDARD_INFORMATION)
 - 명령: istat ntfs_exp.001 38-128-1 (38-128-1: fls 명령으로 찾은 text.txt 파일 식별 번호 입력)

```
$STANDARD_INFORMATION Attribute Values:  
Flags: Archive  
Owner ID: 0  
Security ID: 264 (S-1-5-21-3365887723-186230766-137636848-1002)  
Created: 2026-02-18 15:51:44.062786900 (File Modified: 2026-02-18 15:52:00.019400500 (MFT Modified: 2026-02-18 15:52:00.019400500 (Accessed: 2026-02-18 15:52:00.019400500 (
```

구분	값	설명
Flags	Archive	일반 파일이며 백업 대상 표시
Created	2026-02-18 15:51:44	파일 생성 시간
File Modified	2026-02-18 15:52:00	파일 내용 수정 시간
MFT Modified	2026-02-18 15:52:00	MFT 메타데이터 수정 시간
Accessed	2026-02-18 15:52:00	마지막 접근 시간

[실습] FAT32 / NTFS 파일 시스템 분석

- TSK

- NTFS 이미지에서 text.txt 메타데이터 확인 (\$FILE_NAME, \$OBJECT_ID)
 - 명령: istat ntfs_exp.001 38-128-1 (38-128-1: fls 명령으로 찾은 text.txt 파일 식별 번호 입력)

```
$FILE_NAME Attribute Values:  
Flags: Archive  
Name: text.txt  
Parent MFT Entry: 5      Sequence: 5  
Allocated Size: 0      Actual Size: 0  
Created:      2026-02-18 15:51:44.062786900 (File Modified: 2026-02-18 15:51:44.062786900 (MFT Modified: 2026-02-18 15:51:44.062786900 (Accessed: 2026-02-18 15:51:44.062786900 (  
$OBJECT_ID Attribute Values:  
Object Id: 89f9438c-0c7c-11f1-8abf-04ea56ba05e1
```

구분	값	설명
Flags	Archive	일반 파일이며 백업 대상 표시
Name	text.txt	파일 이름
Parent MFT Entry	5	상위 디렉터리의 MFT Entry (5: Root Directory)
Created / File Modified / MFT Modified / Accessed	2026-02-18 15:51:44	파일 생성 / 수정 / MFT 수정 / 접근 시간
Object ID	89f9438c-0c7c-11f1-...	파일의 고유 GUID

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS 이미지에서 text.txt 메타데이터 확인

- 명령: `istat ntfs_exp.001 38-128-1` (38-128-1: fls 명령으로 찾은 text.txt 파일 식별 번호 입력)

```
Attributes:
Type: $STANDARD_INFORMATION (16-0)   Name: N/A   Resident   size: 72
Type: $FILE_NAME (48-3)              Name: N/A   Resident   size: 82
Type: $OBJECT_ID (64-4)              Name: N/A   Resident   size: 16
Type: $DATA (128-1)                  Name: N/A   Resident   size: 23
```

- text.txt에 총 4개의 Attribute 존재 확인
 - 각 Attribute 별 Name, Resident 여부, Size 요약 제시

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ 삭제 파일 복구 실습

- FAT32: text.txt 완전 삭제 후 이미징 → fat32_exp2.001 생성
- NTFS: text.txt 완전 삭제 후 이미징 → nfts_exp2.001 생성
- 각 이미지의 파일시스템 및 파일 메타데이터를 삭제 전 이미지와 비교 및 분석 수행

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 text.txt 완전 삭제 후 파일 시스템 메타데이터 분석

- 명령: fsstat fat32_exp2.001
- Next Free Sector: 8240 → 8232 (1개 클러스터 할당 해제)
- Free Sector Count: 20822768 → 2082776 (이용 가능한 클러스터 1개 증가)

```
OEM Name: MSDOS5.0
Volume ID: 0x748bbf46
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): File System Type Label: FAT32
Next Free Sector (FS Info): 8240
Free Sector Count (FS Info): 20822768
```



```
OEM Name: MSDOS5.0
Volume ID: 0x748bbf46
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): File System Type Label: FAT32
Next Free Sector (FS Info): 8232
Free Sector Count (FS Info): 2082776
```

- FAT CONTENTS: 8232 - 8239 섹터 할당 해제 확인

FAT CONTENTS (in sectors)

```
-----
8192-8199 (8) -> EOF
8200-8207 (8) -> EOF
8208-8215 (8) -> EOF
8216-8223 (8) -> EOF
8224-8231 (8) -> EOF
8232-8239 (8) -> EOF
```



FAT CONTENTS (in sectors)

```
-----
8192-8199 (8) -> EOF
8200-8207 (8) -> EOF
8208-8215 (8) -> EOF
8216-8223 (8) -> EOF
8224-8231 (8) -> EOF
```

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

- FAT32 text.txt 완전 삭제 후 파일 및 디렉터리 목록 분석
 - 명령: fls -r fat32_exp2.001
 - r / r 10: text.txt → r / r * 10: _ext.txt 변경
 - * 표시 추가: 해당 파일이 삭제된 상태 → 삭제 전 데이터 잔존 가능성
 - 파일 삭제 시 실제 이름을 전부 삭제 X → 파일명 첫 글자 _ (0xE5)로 변경

```
r/r 3:
d/d 6: System Volume Information
+ r/r 135: WPSecrets.dat
r/r * 8: 새 텍스트 문서.txt
d/d 9: $RECYCLE.BIN
+ r/r 389: desktop.ini
r/r 10: text.txt
v/v 33325059: $MBR
v/v 33325060: $FAT1
v/v 33325061: $FAT2
V/V 33325062: $OrphanFiles
```



```
r/r 3:
d/d 6: System Volume Information
+ r/r 135: WPSecrets.dat
r/r * 8: 새 텍스트 문서.txt
d/d 9: $RECYCLE.BIN
+ r/r 389: desktop.ini
r/r * 10: _ext.txt
v/v 33325059: $MBR
v/v 33325060: $FAT1
v/v 33325061: $FAT2
V/V 33325062: $OrphanFiles
```

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ FAT32 text.txt 파일 복구

- 명령: `icat fat32_exp2.001 10 > recovered_fat32_text.txt`
- 삭제 이후에도 text.txt 파일 식별 번호 동일하게 유지
- icat 명령을 통해 복원 가능



[text.txt 복원]

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

- NTFS text.txt 완전 삭제 후 파일 시스템 메타데이터 분석
 - 명령: `fsstat ntfs_exp2.001`
 - 삭제 전 이미지와 동일 출력 결과
 - FAT32: FS INFO에 가용 클러스터 정보 저장 (Next Free Sector, Free Sector Count)
 - NTFS: 클러스터 사용 여부를 \$Bitmap으로 관리 → `fsstat` 명령에서 차이점 관찰 불가

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

- NTFS text.txt 완전 삭제 후 파일 및 디렉터리 목록 분석
 - 명령: fls -r ntfs_exp2.001
 - r / r 38-128-1: text.txt → - / r * 38-128-1: text.txt 변경
 - * 표시 추가: 해당 파일이 삭제된 상태 → 삭제 전 데이터 잔존 가능성
 - r → - 변경: 파일 할당 해제 의미 (MFT Entry Header Flags 변경)

```
++ r/r 41-128-1:      desktop.ini
r/r 9-128-8:      $Secure:$SDS
r/r 9-144-11:     $Secure:$SDH
r/r 9-144-14:     $Secure:$SII
r/r 10-128-1:     $UpCase
r/r 10-128-4:     $UpCase:$Info
r/r 3-128-3:      $Volume
d/d 36-144-1:     System Volume Information
+ r/r 37-128-1:    WPSecrets.dat
r/r 38-128-1:     text.txt
V/V 256:          $OrphanFiles
```



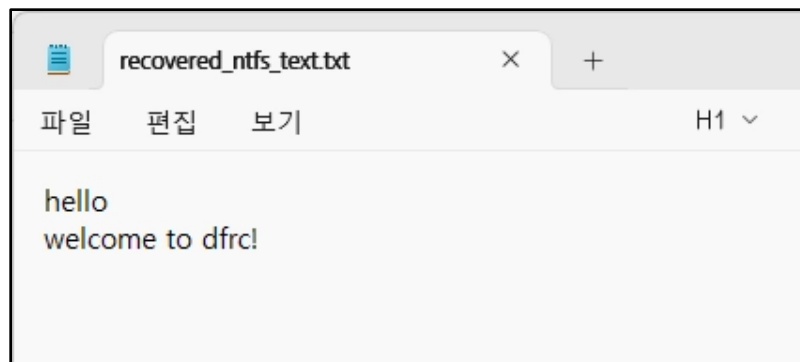
```
++ r/r 41-128-1:      desktop.ini
r/r 9-128-8:      $Secure:$SDS
r/r 9-144-11:     $Secure:$SDH
r/r 9-144-14:     $Secure:$SII
r/r 10-128-1:     $UpCase
r/r 10-128-4:     $UpCase:$Info
r/r 3-128-3:      $Volume
d/d 36-144-1:     System Volume Information
+ r/r 37-128-1:    WPSecrets.dat
-/r * 38-128-1:    text.txt
V/V 256:          $OrphanFiles
```

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK

■ NTFS text.txt 파일 복구

- 명령: `icat ntfs_exp2.001 38-128-1 > recovered_ntfs_text.txt`
- 삭제 이후에도 text.txt 파일 식별 번호 동일하게 유지
- icat 명령을 통해 복원 가능

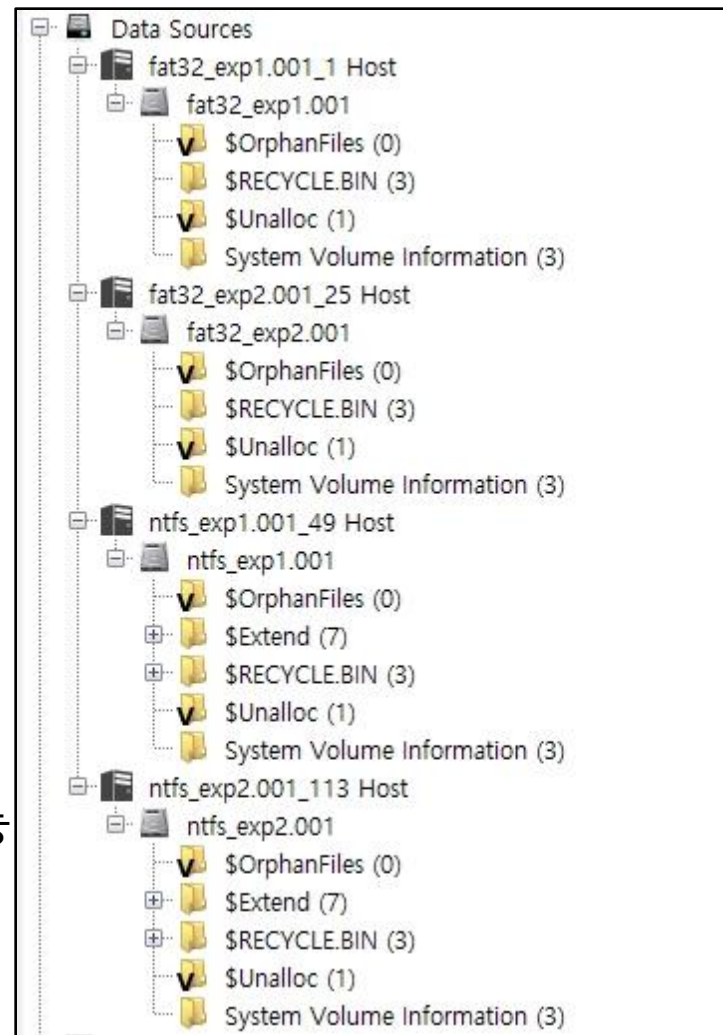


[text.txt 복원]

[실습] FAT32 / NTFS 파일 시스템 분석

■ Autopsy

- Data Sources: 이미지 내 파일 및 디렉터리 목록 확인 가능
 - Data Source 탭과 fls 명령 비교
 - 공통점
 - 파일 시스템 디렉터리의 계층적 구조 제공
 - 차이점
 - fls: 텍스트 형태, 디렉터리 구조 안에 존재하는 엔트리만 나열
 - Autopsy: GUI 제공, \$Unalloc 폴더 제공
 - \$Unalloc: 디스크 미할당 영역을 폴더의 형태로 제공하는 기능
→ 실제 디스크에 존재 X (가상 폴더)



[실습] FAT32 / NTFS 파일 시스템 분석

■ Autopsy

■ File Metadata: TSK의 istat 명령어 결과 제공

- 기존 istat 결과를 가독성 있게 재구성
- 일부 파일 및 폴더 istat 기능 제공 X
 - \$OrphanFiles 및 하위 파일
 - \$Unalloc 및 하위 파일 등
 - TSK에서 istat 명령 자체는 가능
 - 의미 없는 결과: Autopsy에서 제공

From The Sleuth Kit istat Tool:

```
Directory Entry: 10
Allocated
File Attributes: File, Archive
Size: 24
Name: text.txt

Directory Entry Times:
Written: 2026-02-18 15:48:36 (KST)
Accessed: 2026-02-18 00:00:00 (KST)
Created: 2026-02-18 15:48:20 (KST)

Sectors:
Starting address: 8232, length: 1
```

Name	S	C	O	Modified Time	Change Time
\$Unalloc				0000-00-00 00:00:00	0000-00-00 00:00:00
System Volume Information				2026-02-18 15:48:12 KST	0000-00-00 00:00:00
text.txt			2	2026-02-18 15:48:36 KST	0000-00-00 00:00:00

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Context	Annotations
Metadata								
Name:		/img_fat32_exp1.001/text.txt						
Type:		File System						
MIME Type:		text/plain						
Size:		24						
File Name Allocation:		Allocated						
Metadata Allocation:		Allocated						
Modified:		2026-02-18 15:48:36 KST						
Accessed:		2026-02-18 00:00:00 KST						
Created:		2026-02-18 15:48:20 KST						
Changed:		0000-00-00 00:00:00						
MD5:		1ac82bf959e322c112c25ca23023b599						
SHA-256:		dc948681364bb26c1e98868c6747f4846d3a32c9361f61af0b19246b2329dff4						
Hash Lookup Results:		UNKNOWN						
Internal ID:		18						
From The Sleuth Kit istat Tool:								
Directory Entry:		10						
Allocated								
File Attributes:		File, Archive						
Size:		24						
Name:		text.txt						
Directory Entry Times:								
Written:		2026-02-18 15:48:36 (KST)						
Accessed:		2026-02-18 00:00:00 (KST)						
Created:		2026-02-18 15:48:20 (KST)						
Sectors:								
Starting address:		8232, length: 1						

[실습] FAT32 / NTFS 파일 시스템 분석

■ Autopsy

■ File Views - Deleted Files: 삭제된 파일 복원 기능

• FAT32 복구 원리

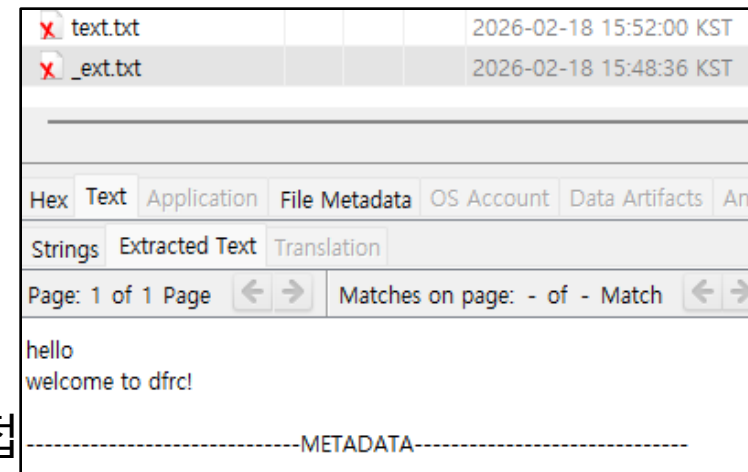
- 파일명 앞 글자 _ (0xE5) 변경 확인 → 파일명 전체 복원 불가
- Directory Entry에 남아있는 클러스터 위치 정보 → Data Area 접근

• NTFS 복구 원리

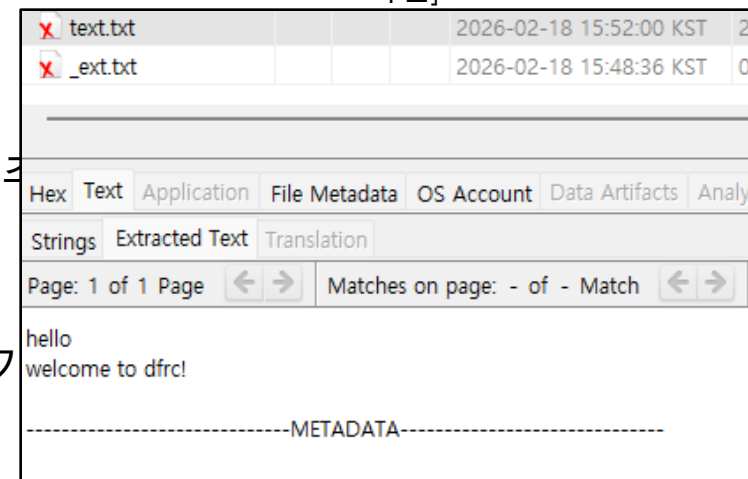
- MFT Entry의 Allocated 플래그 해제 확인
- MFT Entry 내부 \$DATA Run List 정보 → 데이터 클러스터 위치 추적

• Text 탭에서 텍스트 파일 내용 확인 가능

- TSK의 icat 기능과 유사, 'Extract File(s)' 기능으로 추출 및 저장 가능



[fat32_exp2.001 복구 파일]



[ntfs_exp2.001 복구 파일]

[실습] FAT32 / NTFS 파일 시스템 분석

■ TSK vs Autopsy

■ 비교

- 파일 시스템 메타 데이터 분석

- TSK fsstat 명령을 통해 분석 가능 / Autopsy는 구체적 기능 존재 X → Sector Size 등 일부 내용만 파악 가능

- 파일 및 디렉터리 목록 확인

- TSK fls 명령을 통해 분석 가능 / Autopsy Data Source 탭에서 GUI 형태 제공

- 삭제된 파일 복구

- TSK fls 명령으로 파일 식별 번호 확인 후 icat 복원 / Autopsy Deleted Files 탭에서 자동 복원

기능	TSK	Autopsy
파일 시스템 메타 데이터 분석	fsstat	-
파일 및 디렉터리 목록 확인	fls	Data Source
삭제된 파일 복구	fls + icat	Deleted Files

X-Ways

소개

■ 개요

- X-Ways Software Technology AG에서 개발한 전문 디지털 포렌식 분석 도구
- WinHex 기반으로 설계된 소프트웨어 → Low Level 데이터 접근 가능

- WinHex: 동일 기업에서 개발한 Hex Editor

■ 주요 기능

- 디스크 및 이미지 분석: RAW, E01 등 다양한 포렌식 이미지 포맷 지원
- 파일 시스템 분석: 메타데이터 및 아티팩트 분석 지원
- 검색 및 필터링: 키워드, 파일 유형, 해시 등 조건별 데이터 검색 및 필터링 가능
- 보고서 생성: 분석 결과를 기반으로 HTML 보고서 생성 가능

파일 시스템 분석 기능 비교

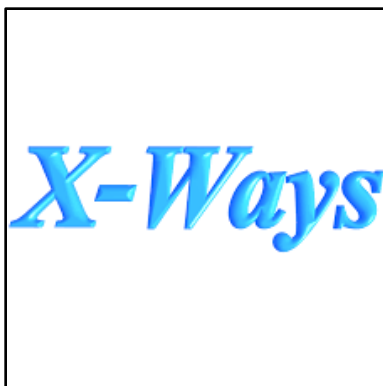
■ X-Ways vs. Axiom vs. Autopsy

■ SW 별 파일 시스템 분석 기능 비교

- X-Ways: WinHex 기반 설계 → 파일 시스템 로우 레벨 분석 용이
- Axiom: 자동화에 초점을 두어 다양한 형태의 아티팩트 분석 및 보고서 제공
- Autopsy: The Sleuth Kit (TSK) 기반 및 모듈 기반 분석으로 아티팩트 분석 용이

■ 파일 시스템 로우 레벨 구조 분석 측면 → X-Ways가 가장 용이

■ 아티팩트 및 사용자 행위 분석 측면 → Axiom, Autopsy는 아티팩트 분석에 초점



파일 시스템 분석 기능 비교

■ 파일 카빙 기능

- X-Ways: Hex 단위 접근 설계 → 수동 / 정밀 카빙 유리
- Axiom: 알려진 시그니처 / 아티팩트 중심 자동 카빙 기능 제공
- Autopsy: 대표적인 파일 유형 중심 카빙 → 고급 및 세밀 옵션 설정 제한적

■ 자동 아티팩트 분석

- X-Ways: 수동 분석 중심 설계 → 분석가 해석 능력 의존도 높음
- Axiom: 전용 아티팩트 파서 다수 내장 → 알려진 아티팩트 기반 자동 분석 강점
- Autopsy: 기본 아티팩트 모듈 제공 → 확장성 좋음 / 다수 모듈 동시 활성화 시 처리 시간 증가

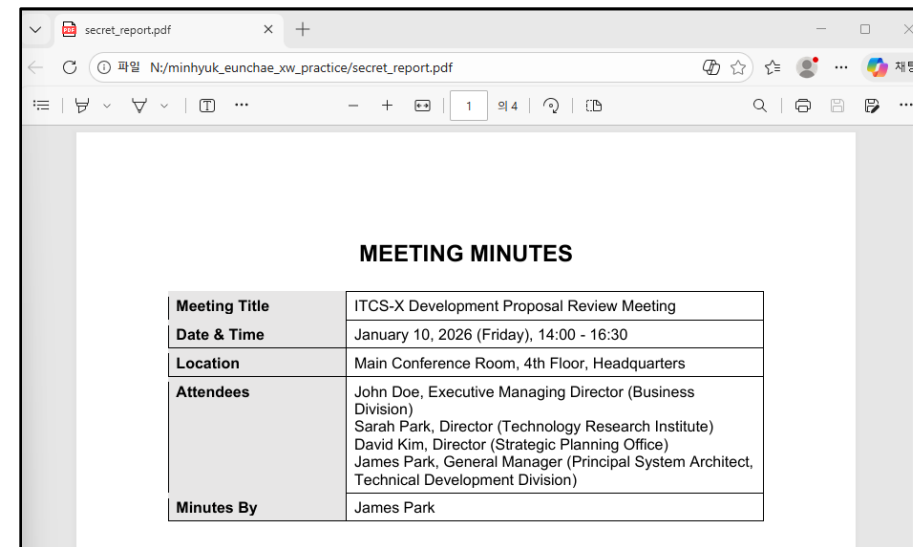
[실습] X-ways file carving 기능 사용

■ 시나리오

- 회사 내부에서 한 직원 PC에서 기밀 문서 유출 사건 발생
- X-Ways로 조사 도중 다음 경로에서 시그니처가 손상된 기밀 문서 완전 삭제 흔적 발견
 - N:/Minhyuk_eunchae_xw_practice/secret_report.pdf
 - 확장자, 파일 크기, 타임스탬프 정상
- X-Ways file carving 기능 이용해 secret_report.pdf 복구

[실습] X-ways file carving 기능 사용

- 실습용 파일 생성 및 조작 후 삭제
 - secret_report.pdf 생성
 - 파일 시그니처 조작
 - PDF 헤더 00 00 00 00으로 훼손
 - Shift + Delete로 완전 삭제 진행



[secret_report.pdf]

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	25	50	44	46	2D	31	2E	37	0D	0A	25	B5	B5	B5	B5	0D	%PDF-1.7..%µµµµ.
00000010	0A	31	20	30	20	6F	62	6A	0D	0A	3C	3C	2F	54	79	70	.1 0 obj..<</Typ
00000020	65	2F	43	61	74	61	6C	6F	67	2F	50	61	67	65	73	20	e/Catalog/Pages

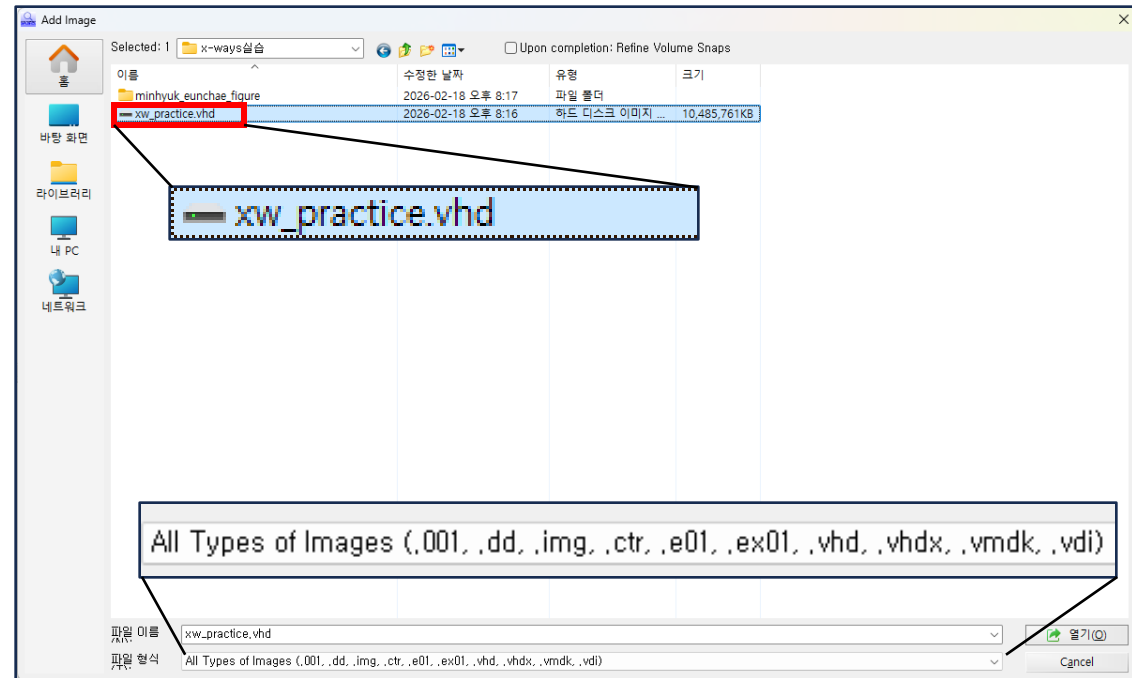
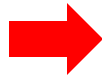
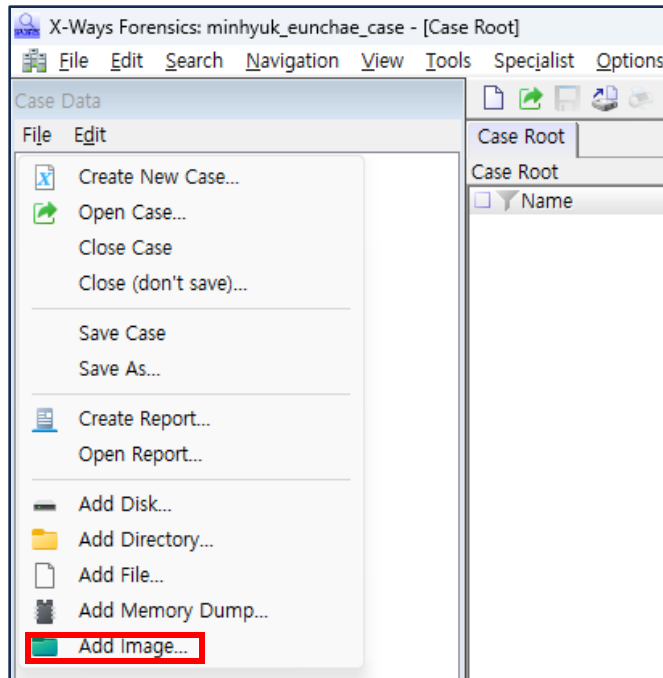


Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	00	00	00	00	2D	31	2E	37	0D	0A	25	B5	B5	B5	B5	0D	-1.7..%µµµµ.
00000010	0A	31	20	30	20	6F	62	6A	0D	0A	3C	3C	2F	54	79	70	.1 0 obj..<</Typ
00000020	65	2F	43	61	74	61	6C	6F	67	2F	50	61	67	65	73	20	e/Catalog/Pages

[실습] X-ways file carving 기능 사용

■ X-Ways 디스크 이미지 업로드

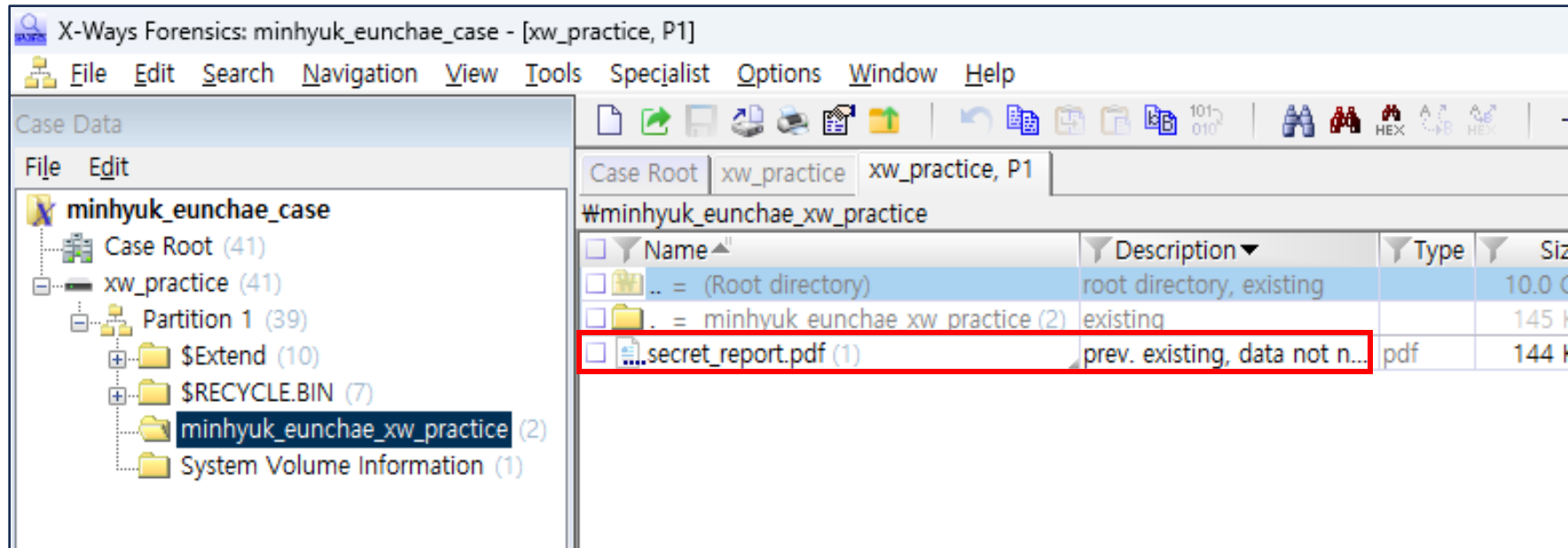
- 파일 완전 삭제 진행 후 가상 디스크 (vhd) X-Ways 업로드
 - .001, .dd, .img, .e01, .ex01, .vhd 등 다양한 이미지 지원 확인



[실습] X-ways file carving 기능 사용

■ X-Ways 디스크 이미지 분석

- 삭제 이후 원래 경로에서 secret_report.pdf 확인 가능
 - Description: Prev. existing, data not necessarily intact
 - 삭제 흔적 확인 / 실제 파일 내용은 확인 불가 상태 → 카빙 필요

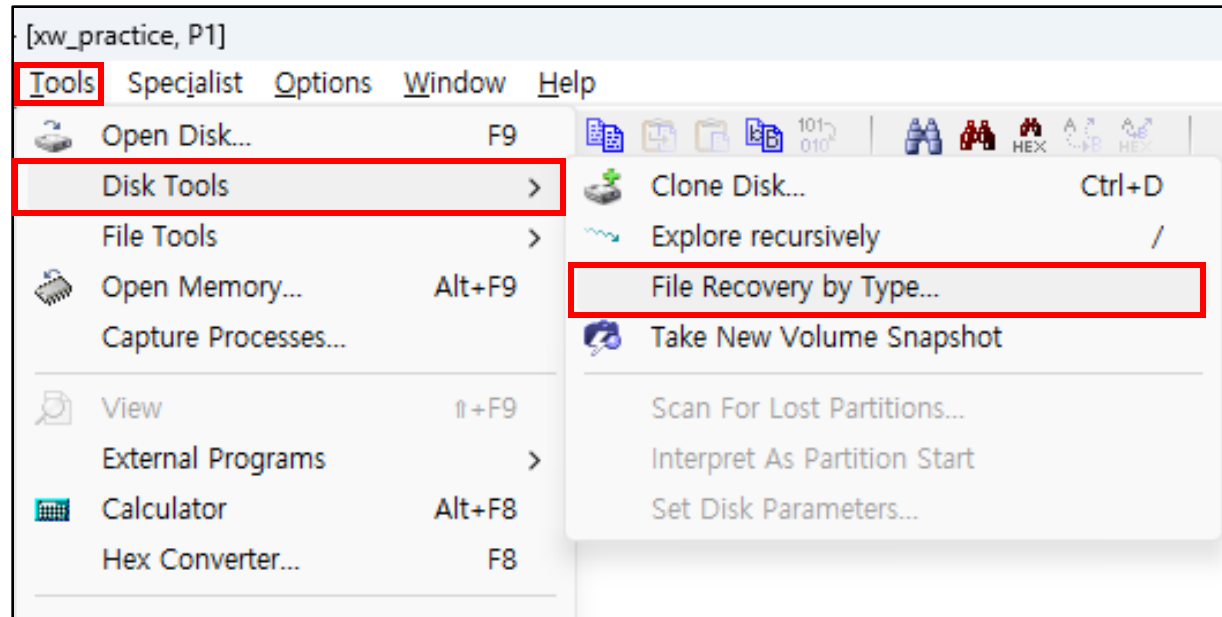


[실습] X-ways file carving 기능 사용

■ X-Ways 디스크 이미지 분석

■ Tools → Disk Tools → File Recovery by Type...

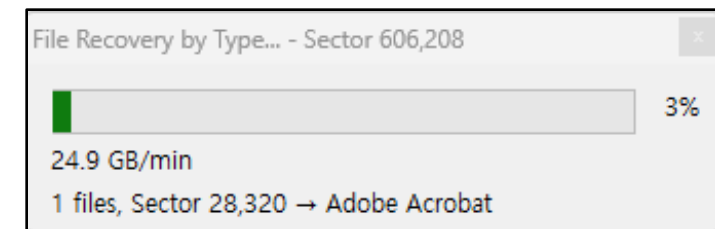
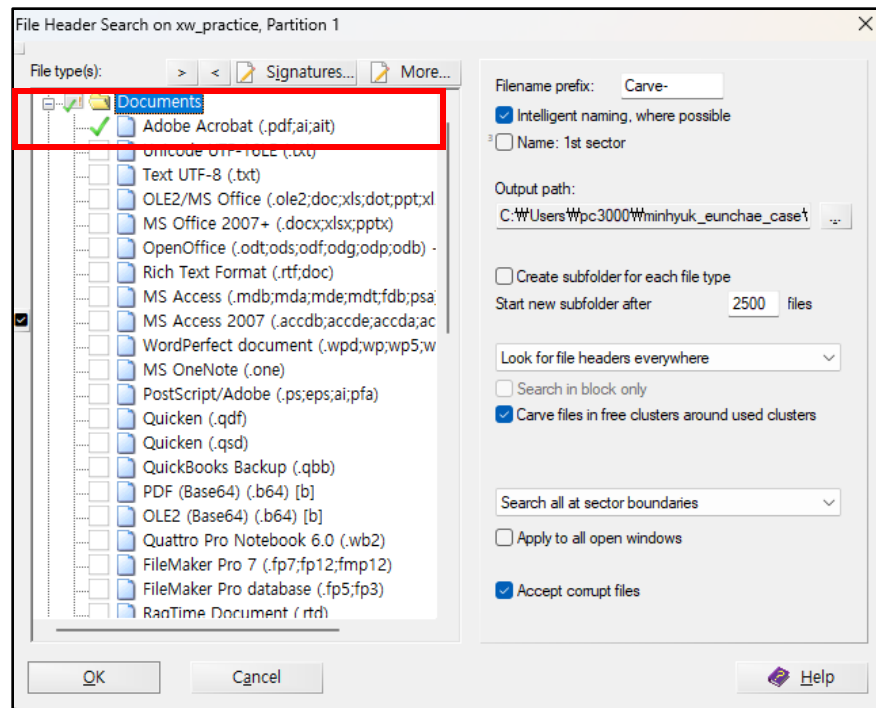
- 파일 헤더 및 시그니처 기반 카빙 기능
- 실습 목표: 헤더 훼손 파일 → 'File Recovery by Type...' 기능을 통해 복구 가능 여부 확인



[실습] X-ways file carving 기능 사용

■ File carving 기능 세팅 및 사용

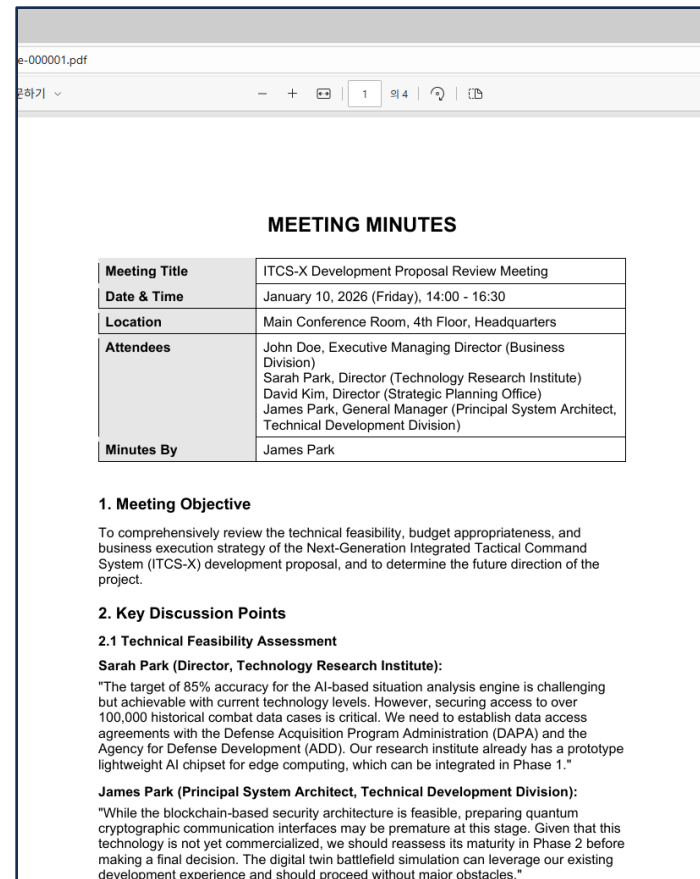
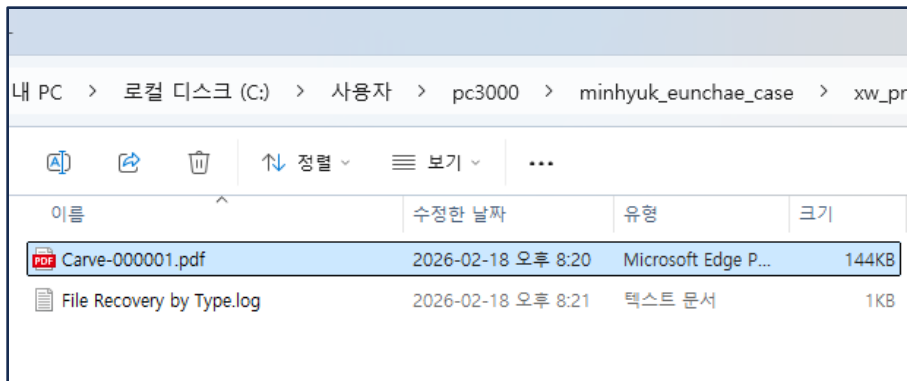
- 파일명을 통해 pdf 파일임을 인지 → File Type: Document – Adobe Acrobat 설정
- 카빙 수행



[실습] X-ways file carving 기능 사용

■ File carving 결과

- 설정한 경로에 카빙된 파일 저장
 - 카빙 파일: Carve-000001.pdf
 - 완전 삭제 전 파일 내용과 동일
- 파일 시그니처 훼손에도 카빙 기능 정상 작동 확인



[secret_report.pdf 복원]

Hashset 세팅 및 검색방법

■ HashSet 개념

- 파일의 해시 값 (MD5 / SHA-1 / SHA-256 등)을 모아둔 데이터베이스
- 알려진 정상 파일 / 알려진 악성 파일 식별 및 필터링 용도
 - 화이트리스트 (Known Good) HashSet: 분석 대상에서 제외 → 수사 시간 / 기간 단축 용도
 - 블랙리스트 (Known Bad) HashSet: 발견 즉시 위협으로 간주 해야 하는 악성 코드

■ HashSet 사용 예시

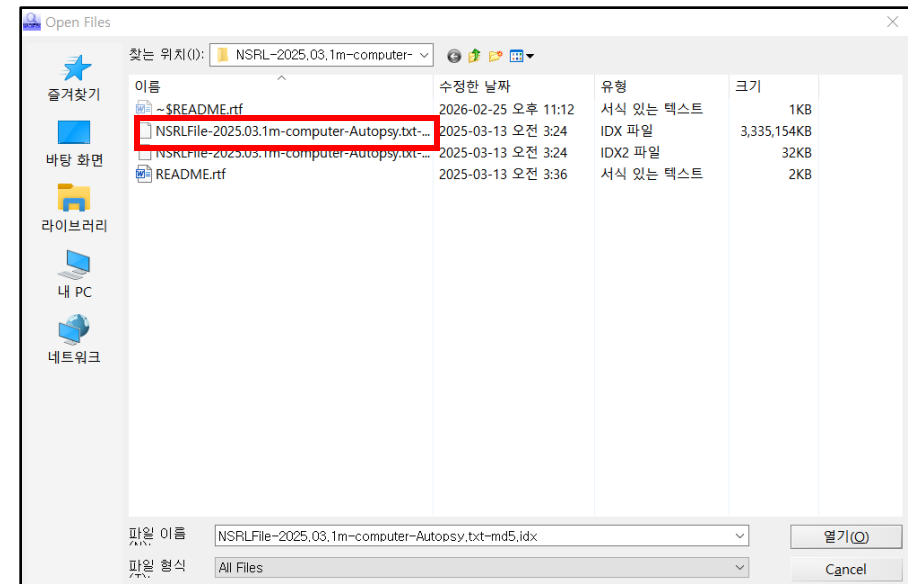
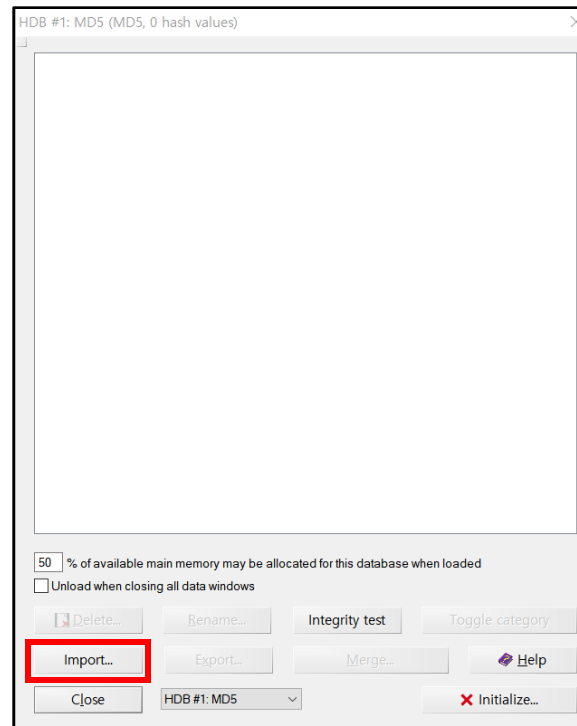
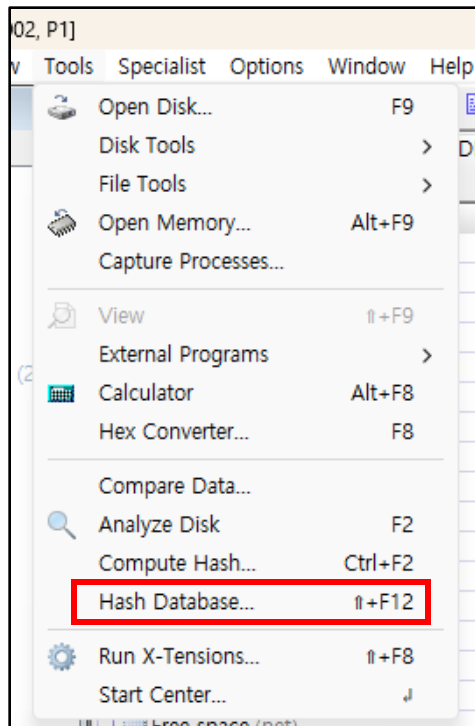
- NIST NSRL: 미국 NIST에서 관리하는 화이트리스트 HashSet
 - X-Ways, Autopsy 등 포렌식 도구에 세팅하여 사용 가능
- Virus Total: 대규모 해시 데이터베이스 기반 파일 악성 여부 판별 온라인 플랫폼

Hashset 세팅 및 검색방법

■ 도구별 방법소개 – X-ways

■ 해시 데이터베이스 추가 방식

- Tools → Hash Database → Import → Hashset 파일 선택



Hashset 세팅 및 검색방법

■ 도구별 방법소개 – X-ways

■ 해시 데이터베이스 추가 방식

• Category 선택

- Irrelevant / known good / harmless / ignorable

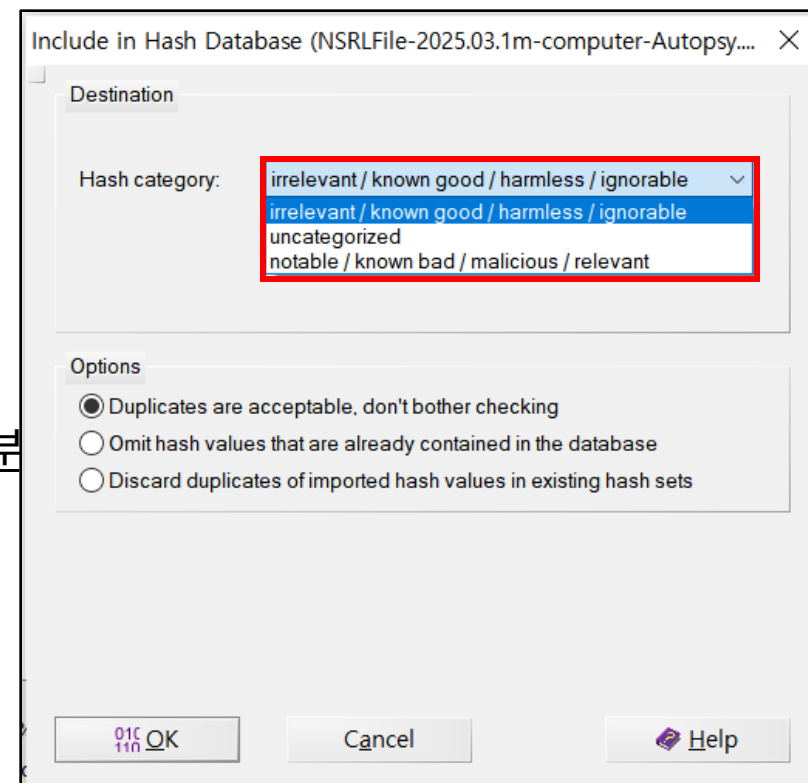
- 정상 파일, 무시해도 되는 항목

- uncategorized

- 아직 분류되지 않은 해시 → 조사 중 적절한 카테고리로 분

- Notable / known bad / malicious / relevant

- 악성 파일로 분류된 항목

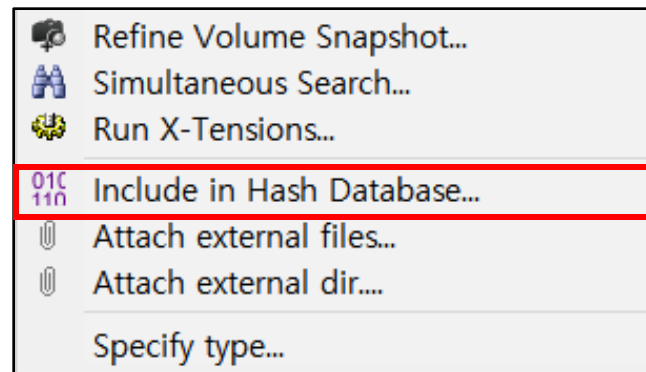


Hashset 세팅 및 검색방법

■ 도구별 방법소개 – X-ways

■ 개별 해시 추가 방식

- Specialist 탭 → Include in Hash Database 기능
- 개별 파일의 해시 값을 현재 로드된 해시 데이터베이스에 추가 가능

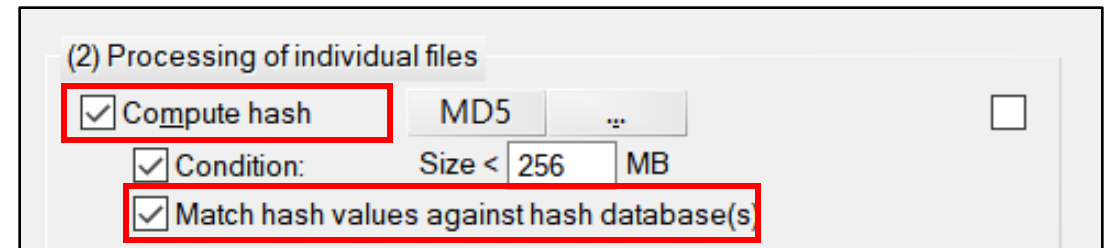
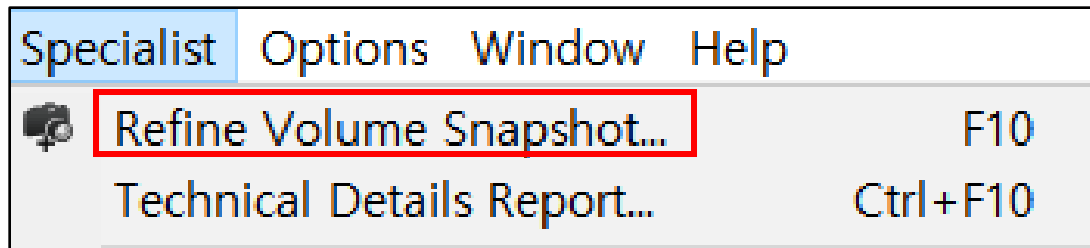


Hashset 세팅 및 검색방법

■ 도구별 방법소개 – X-ways

■ Specialist 탭 → Refine Volume Snapshot

- 새로 추가된 해시를 볼륨에 적용
- Compute hash → Match hash values against hash database
 - 각 파일에 대해 해시 값 계산 후 해시 데이터베이스와 비교 → Categorize 작업



Hashset 세팅 및 검색방법

■ 도구별 방법소개 – X-ways

- Refine Volume Snapshot 과정 → Hashset에 해당되는 파일 확인 가능
 - Hash set 열: 적용된 Hashset 이름이 출력
 - Hash category열: Hashset의 카테고리가 출력

Name	Description	Type	Size	Created	Modified	Record changed	Attr.	1st sector	Hash set	Hash category
.. = (Root directory)	existing, tagged (partially)		1.0 GB	26-02-16d12:53:02	26-02-16d13:11:06	26-02-16d13:11:06	SH	288		
Folder1 (2)	existing, tagged		25 B	26-02-16d12:53:27	26-02-16d20:35:50	26-02-16d20:35:50	I	697,076		
test.txt	existing, tagged	txt	12 B	26-02-16d13:11:11	26-02-16d13:11:17	26-02-16d13:11:17	IA	697,084		
test2.txt	existing, tagged	txt	13 B	26-02-16d20:35:45	26-02-16d20:35:54	26-02-16d20:35:54	IA	697,108	Hashset test	irrelevant

'Hashset test' HashSet에서
'test2.txt'가 irrelevant 파일로
분류

Hash set	Hash category
Hashset test	irrelevant

Hashset 세팅 및 검색방법

■ 도구별 방법소개 – Axiom

- 처리 세부 정보 → 해시 및 일치 항목 찾기 '켜기' → '모든 파일의 해시 값 계산' 체크
 - Axiom Process는 기본적으로 선택적 해시 계산 수행 → HashSet 이용을 위해 모든 파일 해시 계산 필요

처리 세부 정보	
아카이브 및 모바일 백업 검색	켜기
검색 키워드 추가	
파일에서 텍스트 추출 (OCR)	
해시 및 일치 항목 찾기	켜기
Magnet.AI로 채팅 분석	
Magnet.AI로 사진 분석	
검색에 CPS 데이터 추가	
추가 아티팩트 찾기	

해시 및 일치 항목 찾기

모든 파일의 해시 값 계산

AXIOM Process는 케이스의 미디어 파일에 대한 해시 값을 자동으로 계산할 수 있습니다. 미디어 파일 외에도 다른 모든 파일에 대한 해시 값을 계산하도록 선택할 수도 있습니다. 이 설정이 활성화되면 AXIOM Examine는 파일 시스템 탐색기 세부 정보 카드의 각 파일에 대한 해시 값을 표시합니다. 또한 AXIOM Process를 활성화하여 해시셋과 일치하는 파일을 판별할 수 있습니다.

해시가 계산되는 형식이나 가져온 해시 값이 저장되는 위치를 변경하려면 해시 설정을 구성합니다. AXIOM Process가 해시를 생성할 파일의 크기를 제한하여 처리 시간을 줄일 수도 있습니다.

☐ 미디어 파일의 해시 값만 계산

☒ 모든 파일의 해시 값 계산

해시 설정 구성

Hashset 세팅 및 검색방법

■ 도구별 방법소개 – Axiom

- 처리 세부 정보 → 해시 및 일치 항목 찾기 → 일치하는 해시 값이 있는 태그가 알려진 파일
 - HashSet과 일치하는 항목에 태깅
 - 해당 기능에서 이용하는 HashSet은 주로 블랙리스트

처리 세부 정보

아카이브 및 모바일 백업 검색

켜기

검색 키워드 추가

파일에서 텍스트 추출 (OCR)

해시 및 일치 항목 찾기

켜기

Magnet.AI로 채팅 분석

Magnet.AI로 사진 분석

검색에 CPS 데이터 추가

추가 아티팩트 찾기

일치하는 해시 값이 있는 태그가 알려진 파일

AXIOM Process는 MD5 또는 SHA1 해시 값을 기반 관심 파일에 태그를 지정할 수 있으므로 이러한 파일이 증거에 존재하는지 신속하게 식별할 수 있습니다. 예를 들어, 관심을 가진 문서나 기타 파일 형식을 식별할 수 있습니다. 파일에 태그를 지정하는 데 사용할 해시셋을 선택하십시오.

모두 지우기

해시 목록 추가

활성화 C	파일 경로	로드된 날짜	레코드 수	태그	소스	상태
☐	known_hash_txt.jpg.txt	2026-02-19 오후 8:22:12	2	해시 매칭	로컬 해시 집합	준비
☒	Blacklist.txt	2026-02-26 오후 5:07:40	1	DFRC_Blacklist	로컬 해시 집합	로드됨

Hashset 세팅 및 검색방법

■ 도구별 방법소개 – Axiom

- 처리 세부 정보 → 해시 및 일치 항목 찾기 → 관련 없는 파일 무시
 - 특정 카테고리에 해당하는 항목을 제외하는 방식
 - 해당 기능에서 이용하는 HashSet은 주로 화이트리스트

처리 세부 정보

아카이브 및 모바일 백업 검색

검색 키워드 추가

파일에서 텍스트 추출 (OCR)

해시 및 일치 항목 찾기

Magnet.AI로 채팅 분석

Magnet.AI로 사진 분석

검색에 CPS 데이터 추가

추가 아티팩트 찾기

켜기

켜기

관련 없는 파일 무시

사례와 관련 없는 파일의 MD5, SHA1, NSRL 해시를 가져와 AXIOM이 이에 대한 아티팩트를 생성하지 않도록 합니다. 예를 들어, 일반적인 OS 아이콘 및 화면 보호기에 대한 해시 값을 제공하여 증거가 복잡해지지 않도록 합니다. 각 MD5, SHA1, SHA1 해시 값은 해당 행에 표시되어야 합니다.

무시된 파일도 파일 시스템 탐색기에서 계속 볼 수 있습니다.

이 기능이 활성화되어 있으면 AXIOM은 모든 파일의 해시를 계산합니다.

파일 추가

모든 파일 제거

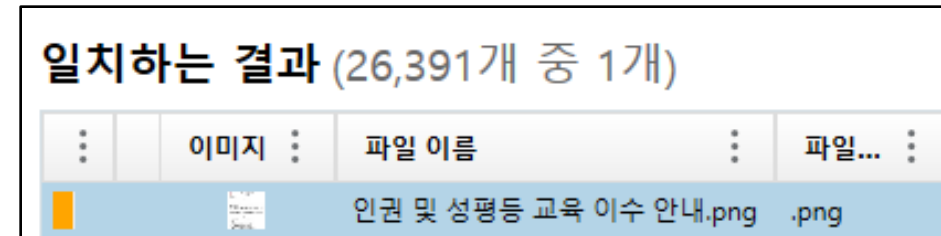
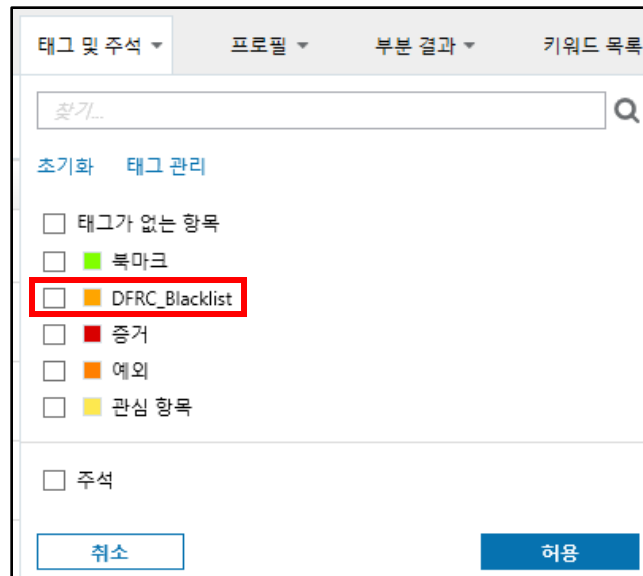
로딩된 레코드: 1

활성화됨	파일 소스	날짜 로드됨	레코드 수
☒	whitelist.txt	2026-02-26 오전 1	1

Hashset 세팅 및 검색방법

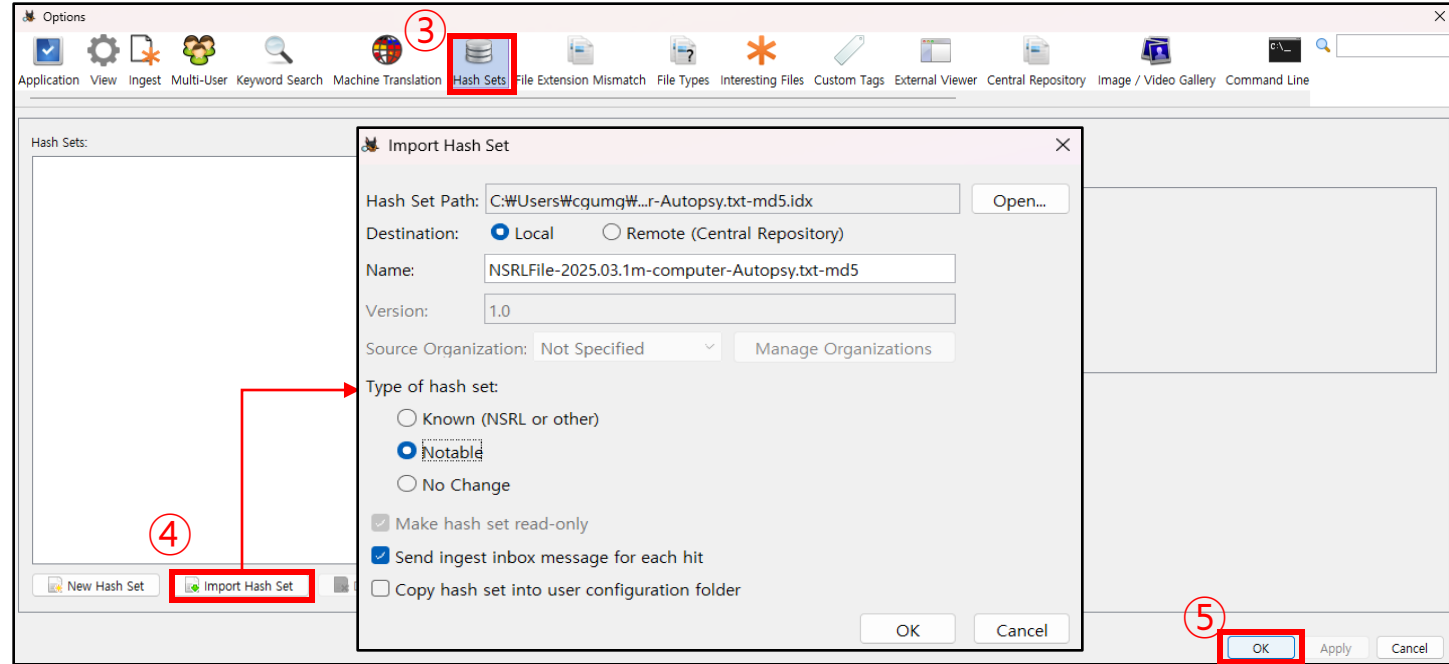
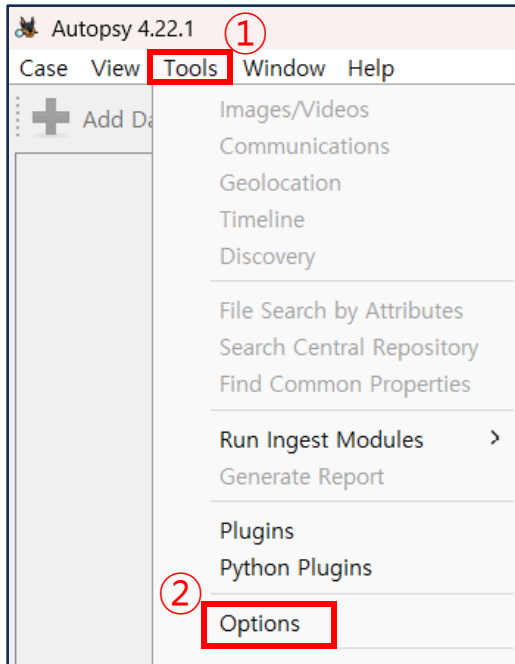
■ 도구별 방법소개 – Axiom

- 태그 및 주석 탭 → HashSet 관련 태그 선택
- 일치하는 결과 탭에서 HashSet과 일치하는 태그 확인 가능



Hashset 세팅 및 검색방법

- 도구별 방법소개 – Autopsy
 - Tools → Options → Hash Sets → Import Hash Set
 - HashSet 경로 설정 후 idx 파일 업로드



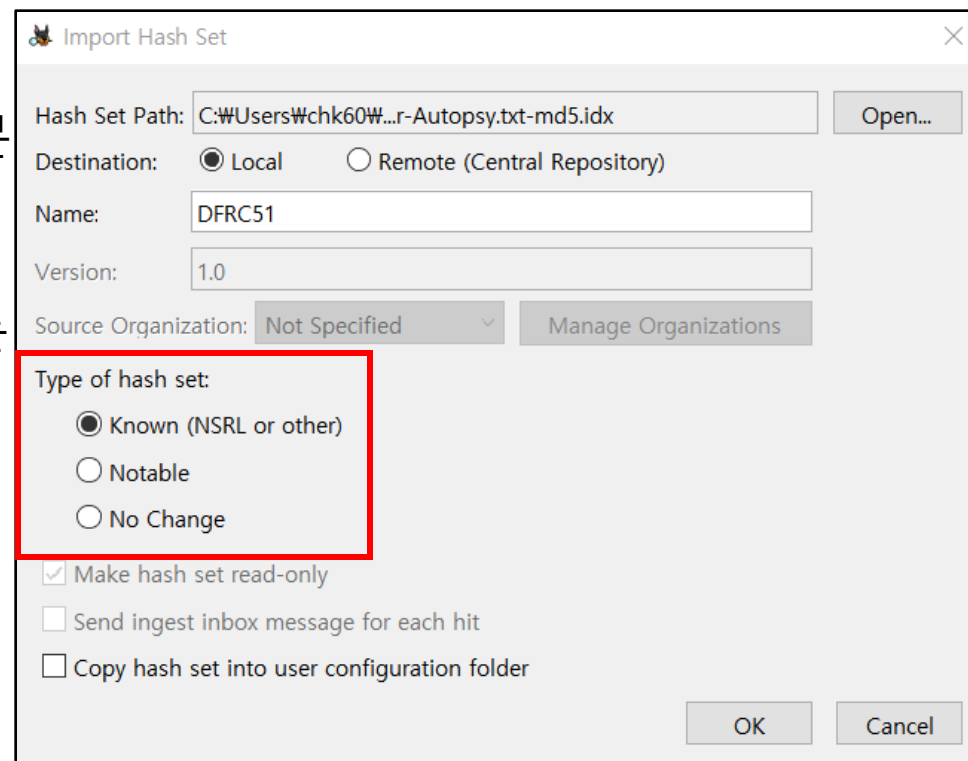
```
00000000000000000000000000000000|nsrl  
00000000000000000000000000000000|NSRLFile-2025.03.1m-computer-Autopsy.txt  
0000004875E627C250DFCD32DCEA2077|0000038127157105  
00000076C8A20F84C9095CB74E15CEC4|0000019164256690  
0000015367EEDD3FAF7EE378DD1992CC|0000003052437779  
000001D847E693AB82A15E6925D281ED|0000045849780821  
000002012397DB2E45A4B605C6422948|0000016289215801  
0000023066EC356F0BC38199260242EA|0000055487541045
```

Hashset 세팅 및 검색방법

■ 도구별 방법소개 – Autopsy

■ Hashset 종류 설정

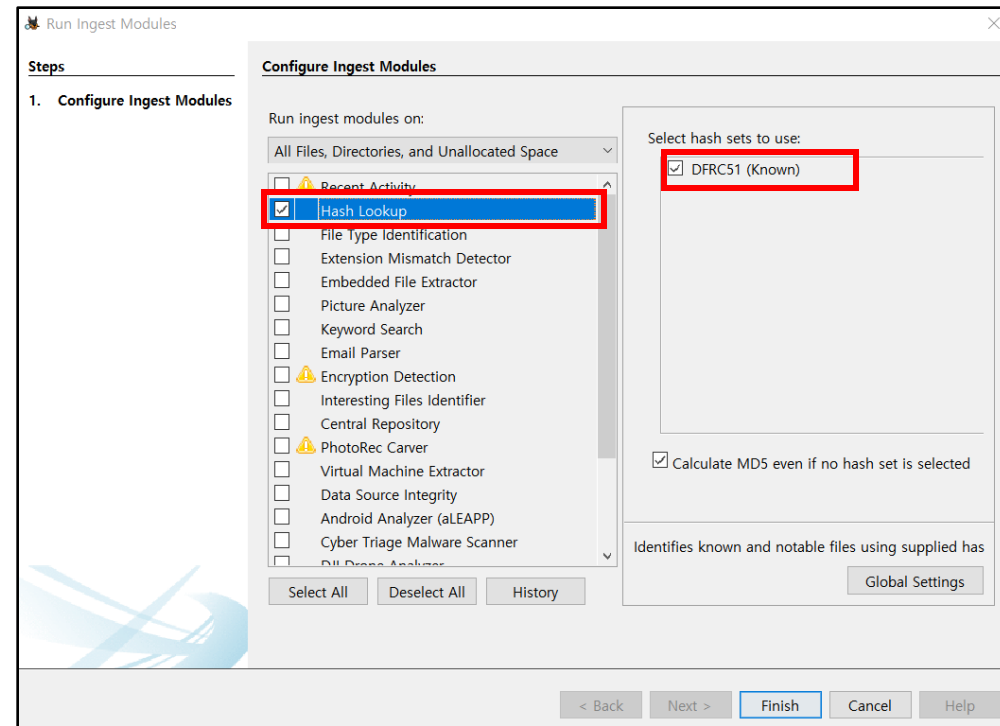
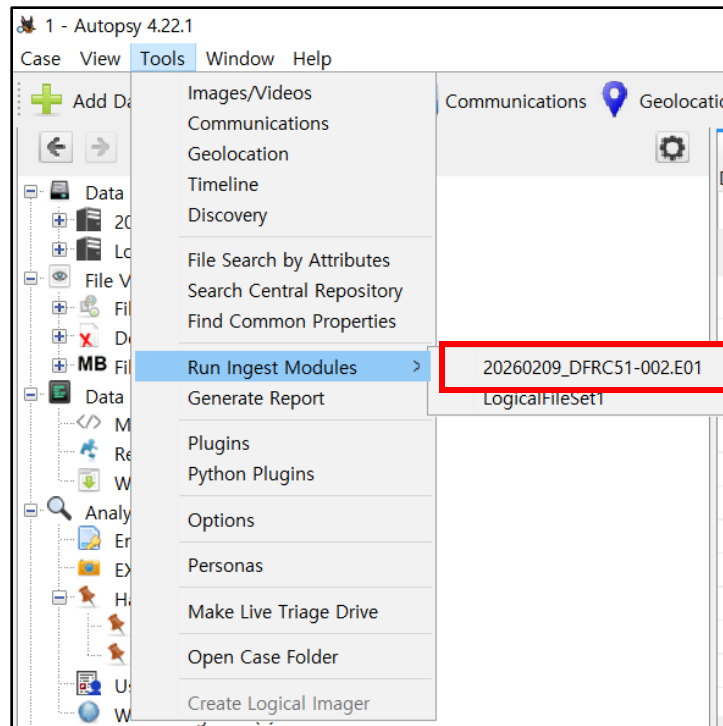
- Known (NSRL or other): 알려진 파일 Hashset
 - OS / 일반 프로그램 파일처럼 무시해도 되는 파일로 분
- Notable: 주의 / 의심 파일 Hashset
 - 악성코드 · 불법 이미지 등 발견 시 경고 태그 부여하는
- No change: 상태를 그대로 유지하는 옵션
 - 해시가 일치해도 Known / Notable로 상태 변경 X



The image shows a screenshot of the 'Import Hash Set' dialog box in Autopsy. The dialog has a title bar with a close button. It contains several input fields and radio buttons. The 'Hash Set Path' field is set to 'C:\Users\Wchk60W...r-Autopsy.txt-md5.idx' with an 'Open...' button next to it. The 'Destination' section has two radio buttons: 'Local' (selected) and 'Remote (Central Repository)'. The 'Name' field is 'DFRC51' and the 'Version' field is '1.0'. The 'Source Organization' dropdown is set to 'Not Specified' with a 'Manage Organizations' button next to it. A red rectangle highlights the 'Type of hash set' section, which contains three radio buttons: 'Known (NSRL or other)' (selected), 'Notable', and 'No Change'. Below this section are three checkboxes: 'Make hash set read-only' (checked), 'Send ingest inbox message for each hit' (unchecked), and 'Copy hash set into user configuration folder' (unchecked). At the bottom right are 'OK' and 'Cancel' buttons.

Hashset 세팅 및 검색방법

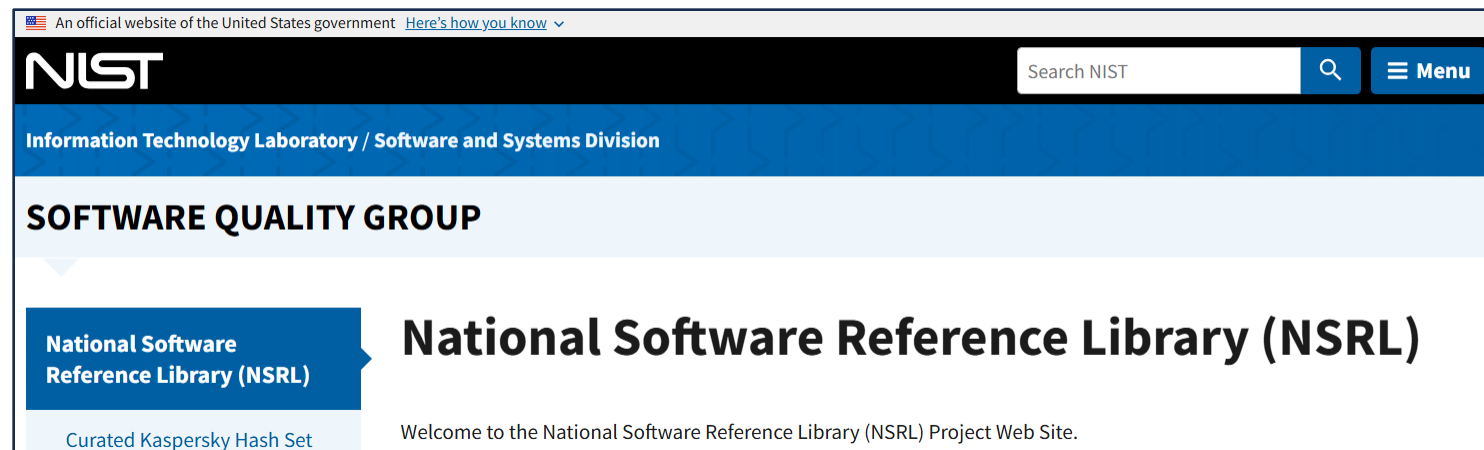
- 도구별 방법소개 – Autopsy
 - Run Ingest Module – 대상 이미지 설정
 - Hash Lookup 선택 – 원하는 Hashset 선택



[실습] NIST NSRL 활용

■ NIST NSRL 개념

- 미국 국립표준기술연구소에서 운영하는 National Software Reference Library의 약어
- 전 세계적으로 널리 사용되는 정상 소프트웨어 파일들의 해시 값 데이터베이스
- 디스크 분석 시 신속한 정상 소프트웨어 필터링 목적
 - 필터링을 통해 빠른 수사 및 악성 파일 검출 가능



[실습] NIST NSRL 활용

■ NIST NSRL 파일 다운로드

- Autopsy User Document → NIST NSRL 다운로드 링크 확인
 - <https://sourceforge.net/projects/autopsy/files/NSRL/>

NIST NSRL

<http://sourceforge.net/projects/autopsy/files/NSRL/>

Autopsy can use the [NIST NSRL](#) to detect 'known files'. The NSRL contains hashes of 'known files' that may be good or bad depending on your perspective and investigation type. For example, the existence of a piece of financial software may be interesting to your investigation and that software could be in the NSRL. Therefore, Autopsy treats files that are found in the NSRL as simply 'known' and does not specify good or bad. Ingest modules have the option of ignoring files that were found in the NSRL.

To use the NSRL, you may download a pre-made index from <http://sourceforge.net/projects/autopsy/files/NSRL/>. Download the **NSRL-XYZm-autopsy.zip** (where 'XYZ' is the version number. As of this writing, it is 247) and unzip the file. Use the **Tools** → **Options** menu and select the "Hash Database" tab. Click "Import Database" and browse to the location of the unzipped NSRL file. You can change the Hash Set Name if desired. Select the type of database desired, choosing "Send ingest inbox message for each hit" if desired, and then click "OK".



Home / Open Source Software / Security / Autopsy / Files



Autopsy Files

Brought to you by: [carrier](#)

Summary

Files

**Download Latest Version**
NSRL-2025.03.1m-computer-Autopsy.zip (1.7 GB)

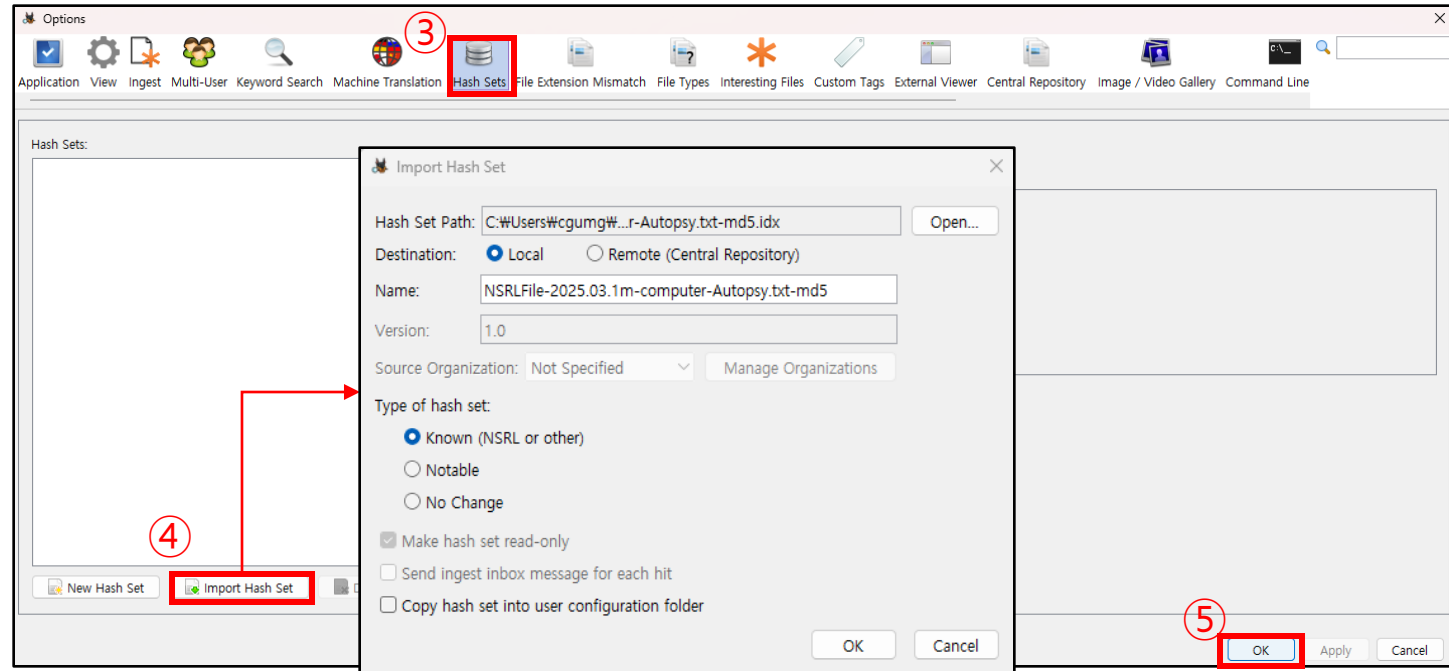
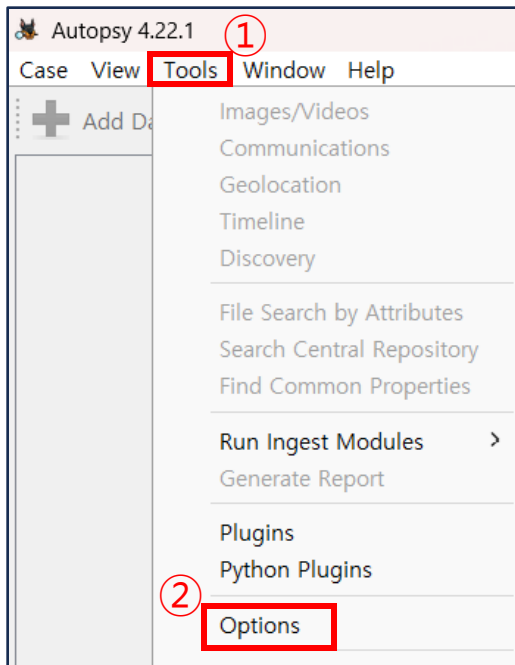


[실습] NIST NSRL 활용

■ Hashset 세팅

■ Autopsy 실행 – 1 (Known 설정)

- Tools → Options → Hash Sets → Import Hash Set → OK
- NSRL은 정상 소프트웨어 필터링 목적의 Hash set → Type of hash set 'Known' 설정

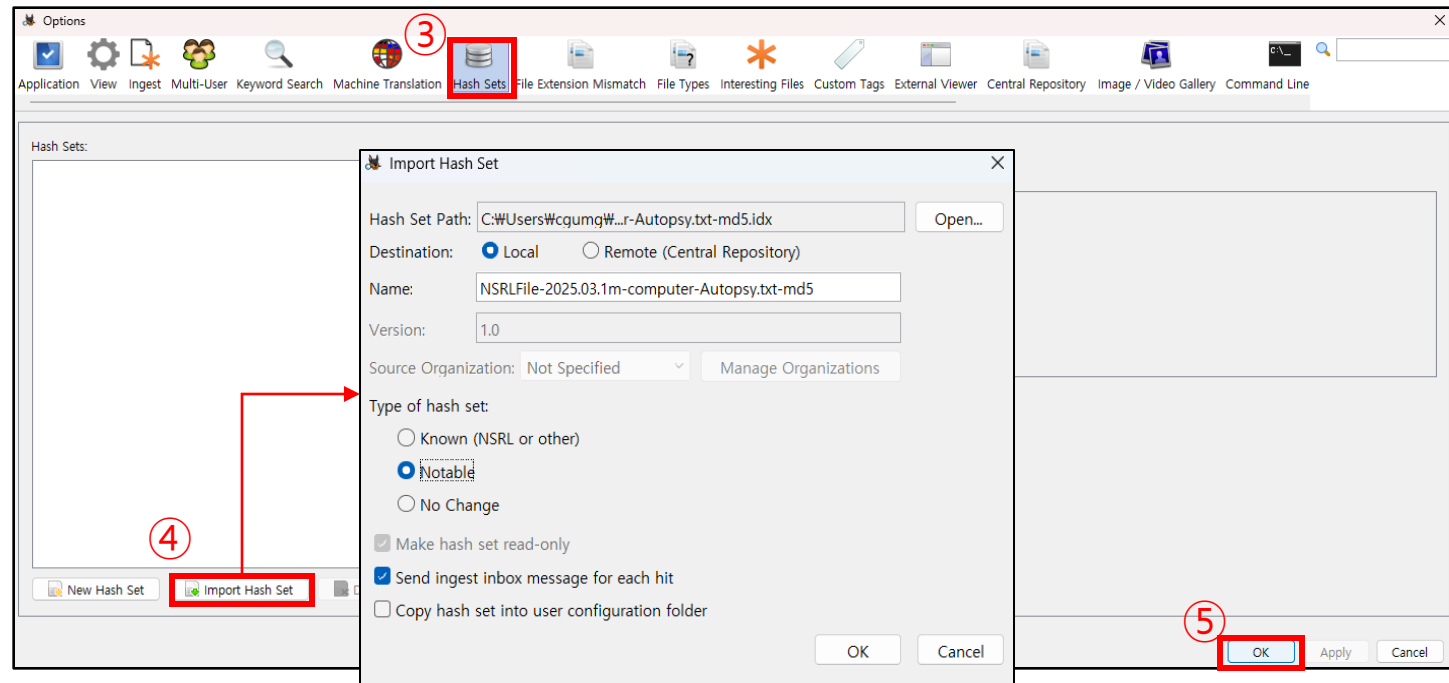
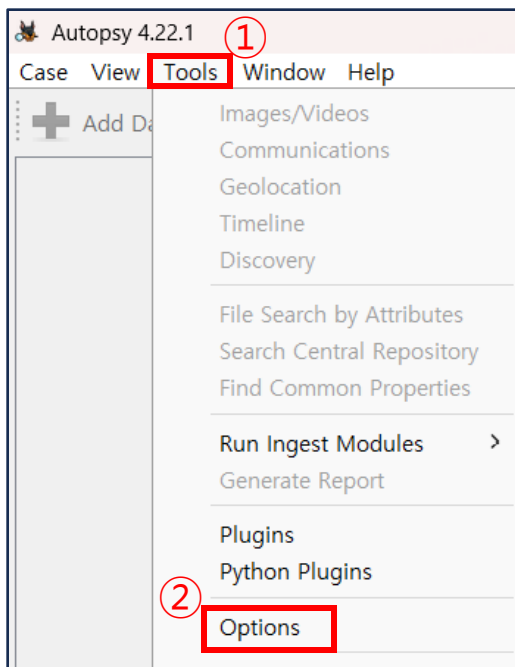


[실습] NIST NSRL 활용

■ Hashset 세팅

■ Autopsy 실행 – 2 (Notable 설정)

- Tools → Options → Hash Sets → Import Hash Set → OK
- 정상 소프트웨어 필터링 테스트를 목적으로 'Notable'로 설정하여 실습



[실습] NIST NSRL 활용

■ Hashset

- Autopsy의 메시지 확인 → 분석창 내 'Known' 열에서 결과 확인 가능
 - Known 설정 → unknown / known 표기
 - Notable 설정 → unknown / known / notable 표기
 - S (Score): 모듈이 해당 항목을 Interesting / Notable 하다고 판단 했을 경우 표시
 - C (Comment): 파일 / 아티팩트에 Comment가 있는지 표시
 - O (Occurences): 같은 항목이 과거 다른 케이스에서 몇 번 나왔는지 표시

Name	S	C	O	Known	Modified Time
📁 \$Deleted				unknown	2025-08-11 21:59:47 KST
📁 \$RmMetadata				unknown	2025-08-11 21:59:47 KST
📁 [current folder]				unknown	2025-08-10 21:24:05 KST
📁 [parent folder]				unknown	2026-02-09 13:30:12 KST
📄 \$ObjId				known	2025-08-10 21:24:05 KST
📄 \$Quota				known	2025-08-10 21:24:05 KST
📄 \$Reparse				known	2025-08-10 21:24:05 KST

Name	S	C	O	Known	Modified Time
📁 \$Txf				unknown	2025-08-11 21:59:47 KST
📁 \$TxfLog				unknown	2025-08-11 21:59:47 KST
📁 [current folder]				unknown	2025-08-11 21:59:47 KST
📁 [parent folder]				unknown	2025-08-10 21:24:05 KST
📄 \$Repair	!			notable	2025-08-11 21:59:47 KST
📄 \$Repair:\$Config	!		1	notable	2025-08-11 21:59:47 KST
📄 \$Repair:\$Corrupt			1	unknown	2025-08-11 21:59:47 KST
📄 \$Repair:\$Verify			1	unknown	2025-08-11 21:59:47 KST

[실습] NIST NSRL 활용

■ Hashset

■ Autopsy의 메시지 확인

- Notable → Known과는 달리 매치되면 Notable로 매치된 파일 / 프로그램 알림 발생

Hash Lookup	1	No known hash set.	2026/03/02 04:52:30
Hash Lookup	1	Notable: 새 텍스트 문서.txt	2026/03/02 04:52:30
Hash Lookup	1	Notable: recovery_fat32_pptx_perfect.txt	2026/03/02 04:52:30
Hash Lookup	1	Notable: recovery_fat32_txt_perfect.txt	2026/03/02 04:52:30
Hash Lookup	1	Notable: _48320EC.tmp	2026/03/02 04:52:30
Hash Lookup	1	Notable: ppt6DE9.tmp	2026/03/02 04:52:30
Hash Lookup	1	Notable: fat32_ppt_perfect.pptx	2026/03/02 04:52:30
Hash Lookup	1	Notable: 새 Microsoft PowerPoint 프레젠테이션.pptx	2026/03/02 04:52:30
Hash Lookup	1	Notable: 새 텍스트 문서.txt	2026/03/02 04:52:30
Hash Lookup	1	Notable: _05FD514.tmp	2026/03/02 04:52:30
Hash Lookup	1	Notable: pptC3EA.tmp	2026/03/02 04:52:30
Hash Lookup	1	Notable: fat32_ppt_original.pptx	2026/03/02 04:52:30
Hash Lookup	1	Notable: 새 Microsoft PowerPoint 프레젠테이션.pptx	2026/03/02 04:52:30
Hash Lookup	1	Notable: 새 텍스트 문서.txt	2026/03/02 04:52:30
Hash Lookup	1	Notable: desktop.ini	2026/03/02 04:52:30
Hash Lookup	1	Notable: »õ ¼·ý (Volume Label En	2026/03/02 04:52:31
Hash Lookup	1	Hash Lookup Results	2026/03/02 04:52:31

[실습] NIST NSRL 활용

■ Hashset

■ Analysis Results의 Hashset Hits 확인

- Notable → Known과는 달리 매치되면 Hashset Hits 발생 시키는 것 확인 가능

The screenshot shows the X-Ways Forensic interface. On the left, the 'Data Sources' pane lists various categories, with 'Minhyuk_NSRL (15)' highlighted under 'Data Sources'. A callout box points to the 'Analysis Results' pane, which shows 'Hashset Hits (15)' highlighted. The main window displays a table of file hashes and their corresponding file paths.

Source Name	S	C	O	MDS Hash	Comment	File Path
_05FDS14.tmp				d41d8cd98f00b204e9800998ecf8427e		/img_F/_05FDS14.tmp
pptC3EA.tmp				d41d8cd98f00b204e9800998ecf8427e		/img_F/pptC3EA.tmp
fat32_ppt_original.pptx				d41d8cd98f00b204e9800998ecf8427e		/img_F/fat32_ppt_original.pptx
새 Microsoft PowerPoint 프레젠테이션.pptx				d41d8cd98f00b204e9800998ecf8427e		/img_F/새 Microsoft PowerPoint 프레젠테이션.pptx
desktop.ini			S	a526b9e7c716b3489d8cc062fbce4005		/img_F/\$RECYCLE.BIN/desktop.ini
새 텍스트 문서.txt				d41d8cd98f00b204e9800998ecf8427e		/img_F/새 텍스트 문서.txt
=0 %%%y (Volume Label En				d41d8cd98f00b204e9800998ecf8427e		/img_F/=0 %%%y (Volume Label En
recovery_fat32_pptx_perfect.txt				d41d8cd98f00b204e9800998ecf8427e		/img_F/recovery_fat32_pptx_perfect.txt
새 텍스트 문서.txt				d41d8cd98f00b204e9800998ecf8427e		/img_F/새 텍스트 문서.txt
recovery_fat32_txt_perfect.txt				d41d8cd98f00b204e9800998ecf8427e		/img_F/recovery_fat32_txt_perfect.txt
_48320C.tmp				d41d8cd98f00b204e9800998ecf8427e		/img_F/_48320C.tmp
ppt6DE8.tmp				d41d8cd98f00b204e9800998ecf8427e		/img_F/ppt6DE8.tmp
fat32_ppt_perfect.pptx				d41d8cd98f00b204e9800998ecf8427e		/img_F/fat32_ppt_perfect.pptx
새 Microsoft PowerPoint 프레젠테이션.pptx				d41d8cd98f00b204e9800998ecf8427e		/img_F/새 Microsoft PowerPoint 프레젠테이션.pptx
새 텍스트 문서.txt				d41d8cd98f00b204e9800998ecf8427e		/img_F/새 텍스트 문서.txt

THANK YOU
for Listening!



Digital Forensic Research Center

고 려 대 학 교 정 보 보 호 연 구 원
디 지 털 포 렌 식 연 구 센 터

dfrc.korea.ac.kr

Questions?

